

Facilitator-Member Difference Identification

Creating the csv file for the following:

Name-conference-year

Also for the following:

- facilitator vs. non-facilitator
- ppl-team vs. ppl-not-team
- ppl-funded-team vs. ppl-not-funded-team

Regression Analysis

Figuring out the question:

By analyzing at the individual level, do facilitators act differently than team members?

Also, considering again, the classification from above:

- facilitator vs. non-facilitator
- ppl-team vs. ppl-not-team
- ppl-funded-team vs. ppl-not-funded-team

Running the features alone (without existing knowledge)

Determining if people are (or not) facilitator, funded-team, on-team.

Probably would just run cross-validation on this.

Fine-tuning stuff (09/29/2025)

floating points

```
In [4]: import json, pandas as pd, numpy as np
        from pathlib import Path
        from collections import defaultdict, Counter

        # ===== CONFIG =====
        DATA_DIR = Path("/Users/maxchalekson/Desktop/gemini_data_analysis/da")
        OUTPUT_DIR = Path("/Users/maxchalekson/Desktop/outputs")
        OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

        WRITE_PERSON_SESSION = True # set False to skip writing the audit csv

        # ===== HELPERS =====
        def _load_json(fp: Path):
            with open(fp, "r") as f:
                return json.load(f)

        def extract_person_metrics_from_session_data(conf_path: Path, session_id: str):
            """
            Compute per-person p_* metrics for a single session from /session_
            Returns dict: person -> {p_*: value}
            """
            f = conf_path / "session_data" / f"{session_id}.json"
            if not f.exists():
                return {}

            data = _load_json(f)
            rows = data.get("all_data", [])
            by_person = defaultdict(lambda: Counter())
            for r in rows:
                speaker = r.get("speaker")
                if not speaker:
                    continue
                dur = r.get("speaking_duration", 0) or 0
                by_person[speaker]["p_speaking_duration_sec"] += float(dur)
                by_person[speaker]["p_turns"] += 1

                if str(r.get("interruption", "")).strip().lower() == "yes":
                    by_person[speaker]["p_interruptions_made"] += 1
                if str(r.get("overlap", "")).strip().lower() == "yes":
                    by_person[speaker]["p_overlaps"] += 1
                if str(r.get("screenshare", "")).strip().lower() == "yes":
                    by_person[speaker]["p_screenshare_segments"] += 1

                by_person[speaker]["p_smile_self_total"] += float(r.get("smile_self", 0))
                by_person[speaker]["p_smile_other_total"] += float(r.get("smile_other", 0))
                by_person[speaker]["p_nods_received"] += float(r.get("nods_received", 0))

            # OPTIONAL: collect annotation counts/scores at utterance level
            ann = r.get("annotations") or {}
            for cat, info in ann.items():
```

```

        # normalize category name to snake-ish
        key = (
            cat.lower()
            .replace("&", "and")
            .replace("/", "_")
            .replace(" ", "_")
            .replace("-", "_")
        )
        score = info.get("score", None)
        if score is not None:
            by_person[speaker][f"ann_{key}_count"] += 1
            by_person[speaker][f"ann_{key}_sum_score"] += float(sc

# return dict of dicts
return {p: dict(cnt) for p, cnt in by_person.items()}

def load_conference(conf_path: Path):
    conf_name = conf_path.name # e.g., 2021MZT
    year = int(conf_name[:4])
    conference = conf_name[4:]

    session_outcomes = _load_json(conf_path / f"{conf_name}_session_outcomes.json")

    # features_*.json (session-level context; we'll prefix ctx_*)
    features = {}
    for fp in conf_path.glob("features_*.json"):
        sid = fp.stem.replace("features_", "")
        features[sid] = _load_json(fp)

    return year, conference, session_outcomes, features

# ===== BUILD PERSON-SESSION =====
all_ps = []
for conf_path in sorted(DATA_DIR.iterdir()):
    if not conf_path.is_dir():
        continue
    conf_name = conf_path.name
    core = conf_path / f"{conf_name}_session_outcomes.json"
    if not core.exists():
        continue

    year, conference, session_outcomes, features = load_conference(conf_path)
    special = {"missing_names", "people_not_in_any_team"}
    sessions = [sid for sid in session_outcomes.keys() if sid not in special]

    for sid in sessions:
        so = session_outcomes[sid]
        facilitators = set(so.get("facilitators", []) or [])
        speakers = set(so.get("all_speakers", []) or [])

        # team members from teams{}
        members = set()

```

```

for _, tinfo in (so.get("teams", {}) or {}).items():
    for m in tinfo.get("members", []) or []:
        members.add(m)

people = sorted(facilitators | speakers | members)
ctx = {f"ctx_{k}": v for k, v in (features.get(sid, {}) or {})}
pmet = extract_person_metrics_from_session_data(conf_path, sid)

for person in people:
    if person in facilitators:
        role_in_session = "facilitator"
    elif person in members:
        role_in_session = "member"
    elif person in speakers:
        role_in_session = "participant"
    else:
        role_in_session = "unknown"

    row = {
        "person_name": person,
        "conference": conference,
        "year": year,
        "session_id": sid,
        "role_in_session": role_in_session,
        **ctx
    }
    row.update(pmet.get(person, {}))
    all_ps.append(row)

ps_df = pd.DataFrame(all_ps)

# write person-session (audit)
if WRITE_PERSON_SESSION and not ps_df.empty:
    ps_out = OUTPUT_DIR / "ALL_person_session.csv"
    ps_df.to_csv(ps_out, index=False)
    print(f"Wrote person-session audit: {ps_out} ({len(ps_df)} rows)")

# ===== AGGREGATE → PERSON-YEAR (SUM COUNTS; WEIGHTED MEANS FOR an
if ps_df.empty:
    raise SystemExit("No person-session rows. Check DATA_DIR structure

keys = ["person_name", "conference", "year"]

# identify columns
p_cols = [c for c in ps_df.columns if c.startswith("p_")]
ctx_cols = [c for c in ps_df.columns if c.startswith("ctx_")]
ann_count_cols = [c for c in ps_df.columns if c.startswith("ann_") and
ann_sum_cols = [c for c in ps_df.columns if c.startswith("ann_") and

# base aggregations
agg_map = {}
# p_* sum (counts/segments) and duration sum

```

```

for c in p_cols:
    agg_map[c] = "sum"
# ctx_* mean across sessions
for c in ctx_cols:
    agg_map[c] = "mean"
# annotations: sum counts and sum_scores
for c in ann_count_cols + ann_sum_cols:
    agg_map[c] = "sum"

py_base = ps_df.groupby(keys, as_index=False).agg(agg_map)

# add sessions_total (unique sessions attended)
sesh_counts = (
    ps_df.groupby(keys, as_index=False)["session_id"]
        .nunique()
        .rename(columns={"session_id": "sessions_total"})
)
py = py_base.merge(sesh_counts, on=keys, how="left")

# derive role tallies per person-year from person-session
role_tallies = (
    ps_df.assign(one=1)
        .pivot_table(index=keys, columns="role_in_session", values="one")
        .reset_index()
        .rename(columns={"facilitator": "facilitator", "member": "member"})
)
# ensure all role columns exist
for col in ["facilitator", "member", "participant", "unknown"]:
    if col not in role_tallies.columns:
        role_tallies[col] = 0
py = py.merge(role_tallies, on=keys, how="left")

# weighted means for each annotation mean_score = sum_score / count
ann_categories = set([c.replace("ann_", "").replace("_count", "") for c in ann_count_cols])
for cat in sorted(ann_categories):
    cnt_col = f"ann_{cat}_count"
    sum_col = f"ann_{cat}_sum_score"
    mean_col = f"ann_{cat}_mean_score"
    if cnt_col in py.columns and sum_col in py.columns:
        py[mean_col] = np.where(py[cnt_col].fillna(0) > 0,
                                pd.to_numeric(py[sum_col], errors="coerce"),
                                np.nan)

# ===== dtype enforcement: ints for counts; floats for durations/score
# counts (discrete)
count_like = [
    "p_turns", "p_interruptions_made", "p_overlaps", "p_screenshare_segme",
    "sessions_total", "facilitator", "member", "participant", "unknown"
] + ann_count_cols

for c in count_like:
    if c in py.columns:

```

```

        py[c] = pd.to_numeric(py[c], errors="coerce").fillna(0).round(
# sums of scores are numeric; they'll typically be floats but are sums
for c in ann_sum_cols:
    if c in py.columns:
        s = pd.to_numeric(py[c], errors="coerce").fillna(0)
        py[c] = np.where(np.isclose(s, np.round(s)), s.round().astype(

# speaking duration is continuous; keep float (round for readability)
if "p_speaking_duration_sec" in py.columns:
    py["p_speaking_duration_sec"] = pd.to_numeric(py["p_speaking_durat

# ===== OPTIONAL: derive role flags =====
py["role_primary"] = py[["facilitator", "member", "participant", "unknown
py["role_facilitator"] = (py["facilitator"] > 0).astype(int)
py["role_member"] = (py["member"] > 0).astype(int)
py["role_participant"] = (py["participant"] > 0).astype(int)
py["role_nonfacilitator"] = (py["role_facilitator"] == 0).astype(int)

# If you have team files to compute on_team / funded, merge here (skip

# write person-year
py_out = OUTPUT_DIR / "ALL_person_year_FIXED.csv"
py.sort_values(["person_name", "year", "conference"]).to_csv(py_out, ind
print(f"Wrote person-year: {py_out} ({len(py)} rows)")

# ===== AGGREGATE → PERSON (across all years) =====
sum_cols_person = [
    "p_turns", "p_interruptions_made", "p_overlaps", "p_screenshare_segme
    "sessions_total", "facilitator", "member", "participant", "unknown"
] + ann_count_cols + ann_sum_cols

mean_cols_person = [c for c in py.columns if c.startswith(("ctx_"))]
mean_cols_person += [c for c in py.columns if c.endswith("_mean_score")

agg_person = {}
for c in sum_cols_person:
    if c in py.columns: agg_person[c] = "sum"
for c in mean_cols_person:
    if c in py.columns: agg_person[c] = "mean"

person = py.groupby("person_name", as_index=False).agg(agg_person)

# enforce ints again for counts
for c in sum_cols_person:
    if c in person.columns:
        person[c] = pd.to_numeric(person[c], errors="coerce").fillna(0

# keep durations and mean scores as floats (rounded)
for c in [x for x in person.columns if x.endswith("_mean_score") or x.
    person[c] = pd.to_numeric(person[c], errors="coerce").round(3)

```

```

person_out = OUTPUT_DIR / "all_data_df-agg-ppl_FIXED.csv"
person.sort_values(["person_name"]).to_csv(person_out, index=False)
print(f"Wrote person-level: {person_out} ({len(person)} rows)")

# ===== QUICK SANITY: show any columns that are float but expected
def check_int_columns(df, cols, label):
    bad = []
    for c in cols:
        if c in df.columns and not pd.api.types.is_integer_dtype(df[c]):
            bad.append(c)
    if bad:
        print(f"[WARN] {label} columns not integer as expected:", bad)
    else:
        print(f"[OK] All {label} columns are integer.")

int_cols_py = ["p_turns", "p_interruptions_made", "p_overlaps", "p_screen",
               "sessions_total", "facilitator", "member", "participant",
               "person-year count-like"]
check_int_columns(py, int_cols_py, "person-year count-like")

int_cols_person = [c for c in int_cols_py + ann_sum_cols if c in person]
check_int_columns(person, int_cols_person, "person-level count-like")

```

Wrote person-session audit: /Users/maxchalekson/Desktop/outputs/ALL_per
son_session.csv (2073 rows)

Wrote person-year: /Users/maxchalekson/Desktop/outputs/ALL_person_year_
FIXED.csv (790 rows)

Wrote person-level: /Users/maxchalekson/Desktop/outputs/all_data_df-agg-
ppl_FIXED.csv (670 rows)

[OK] All person-year count-like columns are integer.

[OK] All person-level count-like columns are integer.

fixing the names

```

In [7]: import re, unicodedata
import pandas as pd
import numpy as np
from pathlib import Path
from collections import Counter

# ===== PATHS =====
IN_DIR = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-g
PS_IN = IN_DIR / "ALL_person_session.csv"
PY_IN = IN_DIR / "ALL_person_year_FIXED.csv"
P_IN = IN_DIR / "all_data_df-agg-ppl_FIXED.csv"

OUT_DIR = IN_DIR
MAP_OUT = OUT_DIR / "name_clean_map.csv"
PS_OUT = OUT_DIR / "ALL_person_session_CLEANED.csv"
PY_OUT = OUT_DIR / "ALL_person_year_FIXED_CLEANED.csv"
P_OUT = OUT_DIR / "all_data_df-agg-ppl_FIXED_CLEANED.csv"

# ===== LOAD =====

```

```

ps = pd.read_csv(PS_IN)
py = pd.read_csv(PY_IN)
p = pd.read_csv(P_IN)

# ===== NAME NORMALIZATION =====
DEGREE_SUFFIXES = [
    r"\bph\.?d\.?b", r"\bmd\b", r"\bmd-ph\.?d\.?b", r"\bscd\b", r"\b
    r"\bba\b", r"\bma\b", r"\bmba\b", r"\bdvm\b", r"\bdd\b"
]
ORG_DELIMS = [r"\s*-\s*", r"\s*-\s*", r"\s*-\s*", r"\s*\|\s*", r"\s*@\"
TRAILERS = [r"\(.?*\)", r"\[.*?\]", r"\{.*?\}"]

# If you know any manual fixes, put them here (raw or cleaned key -> c
alias_map = {
    # "Max Chalekson - UCLA": "Max Chalekson",
    # "Evey Huang, PhD": "Evey Huang",
}

def strip_accents(text: str) -> str:
    return ''.join(c for c in unicodedata.normalize('NFKD', text) if n

def basic_name_clean(s: str) -> str:
    if not isinstance(s, str) or not s.strip():
        return ""
    x = s.strip()

    # remove emails
    x = re.sub(r"\b\S+@\S+\.\S+\b", " ", x)
    # drop anything in (), [], {}
    for patt in TRAILERS:
        x = re.sub(patt, " ", x)
    # drop degree suffixes
    for deg in DEGREE_SUFFIXES:
        x = re.sub(deg, " ", x, flags=re.IGNORECASE)
    # split off affiliation/org
    for delim in ORG_DELIMS:
        x = re.split(delim, x)[0]

    # remove titles
    x = re.sub(r"^s*(dr|prof|mr|ms|mrs)\.?s+", " ", x, flags=re.IGNO

    # tidy punctuation/whitespace
    x = re.sub(r"^[^w\s'-.]", " ", x) # keep letters/digits
    x = re.sub(r"\s+", " ", x).strip()

    # accents -> ascii, title-case words
    x = strip_accents(x)
    x = " ".join(w.capitalize() for w in x.split())

    # length guard
    return x[:120].strip()

```

```

def final_clean(raw: str) -> str:
    if not isinstance(raw, str): return ""
    if raw in alias_map: return alias_map[raw]
    cleaned = basic_name_clean(raw)
    return alias_map.get(cleaned, cleaned)

# Build mapping from the *rawest* table (person-session)
raw_names = ps["person_name"].astype(str).fillna("")
freq = Counter(raw_names)
unique_raw = pd.Series(sorted(set(raw_names)))
cleaned = unique_raw.apply(final_clean)

def keyize(s: str) -> str:
    s2 = s.lower()
    s2 = re.sub(r"^[a-z]", "", s2) # letters only for a coarse cluster
    return s2

map_df = pd.DataFrame({
    "raw_name": unique_raw,
    "clean_name": cleaned,
})
map_df["raw_freq"] = map_df["raw_name"].map(freq)
map_df["cluster_key"] = map_df["clean_name"].apply(keyize)

# choose canonical per cluster (most frequent raw -> its cleaned)
canon_by_key = {}
for key, grp in map_df.groupby("cluster_key"):
    idx = grp["raw_freq"].idxmax()
    canon_by_key[key] = grp.loc[idx, "clean_name"]
map_df["canonical_name"] = map_df["cluster_key"].map(canon_by_key)

# Save mapping for review
map_df.sort_values(["cluster_key", "clean_name", "raw_freq"], ascending=
print(f"[OK] Wrote name mapping: {MAP_OUT}")

# Helper to apply mapping (raw->canonical) with fallback to cleaner
raw_to_canon = dict(zip(map_df["raw_name"], map_df["canonical_name"]))

def apply_canonical(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["person_name"] = df["person_name"].map(raw_to_canon).fillna(df[
    return df

# ===== APPLY TO PERSON-SESSION (and write) =====
ps_clean = apply_canonical(ps)
ps_clean.to_csv(PS_OUT, index=False)
print(f"[OK] Wrote cleaned person-session: {PS_OUT} (rows={len(ps_clean)}

# ===== RE-AGG LOGIC (keeps ints for counts, floats for means) =====
def build_agg_dict(df: pd.DataFrame):
    sum_cols = []
    mean_cols = []

```

```

for c in df.columns:
    if c in ["person_name", "conference", "year", "session_id", "role_
        continue
    # counts / sums (discrete events) -> SUM
    if (c.endswith("_count") or c.endswith("_sum_score") or
        c in ["p_turns", "p_interruptions_made", "p_overlaps", "p_scr
            "sessions_total", "facilitator", "member", "participant
            "teams_funded", "teams_unfunded", "teams_total",
            "role_facilitator", "role_nonfacilitator", "role_on_te
            "role_member", "role_participant"]):
        sum_cols.append(c)
    # means / continuous -> MEAN
    elif (c.endswith("_mean_score") or c.endswith("_mean_score_we
        c.startswith("ctx_") or c in ["p_speaking_duration_sec"])
        mean_cols.append(c)
    else:
        # default: if numeric and not a known count, treat as mean
        if pd.api.types.is_numeric_dtype(df[c]):
            mean_cols.append(c)
agg = {**{c:"sum" for c in sum_cols}, **{c:"mean" for c in mean_co
return agg, sum_cols

def cast_int_counts(df: pd.DataFrame, count_cols):
    for c in count_cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0).ro
    return df

# ===== RE-AGG PERSON-YEAR =====
py_clean = apply_canonical(py)
agg_py, sum_cols_py = build_agg_dict(py_clean)
py_final = (py_clean
    .groupby(["person_name", "conference", "year"], as_index=False)
    .agg(agg_py)
)
py_final = cast_int_counts(py_final, sum_cols_py + ["year"])
py_final.to_csv(PY_OUT, index=False)
print(f"[OK] Wrote cleaned person-year: {PY_OUT} (rows={len(py_final)})

# ===== RE-AGG PERSON-LEVEL =====
p_clean = apply_canonical(p)
agg_p, sum_cols_p = build_agg_dict(p_clean)
p_final = (p_clean
    .groupby(["person_name"], as_index=False)
    .agg(agg_p)
)
p_final = cast_int_counts(p_final, sum_cols_p)
p_final.to_csv(P_OUT, index=False)
print(f"[OK] Wrote cleaned person-level: {P_OUT} (rows={len(p_final)})

# ===== QUICK DIAGNOSTICS =====
sus = map_df[ (map_df["clean_name"].eq("")) | (~map_df["clean_name"].s

```

```

if not sus.empty:
    print("\n[Heads-up] Some names look empty or single-token; consider
    print(sus.sort_values("raw_freq", ascending=False).head(15)[["raw_
else:
    print("\n[OK] No obviously empty/single-token names in mapping.")

```

[OK] Wrote name mapping: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/name_clean_map.csv

[OK] Wrote cleaned person-session: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/ALL_person_session_CLEANED.csv (rows=2073)

[OK] Wrote cleaned person-year: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/ALL_person_year_FIXED_CLEANED.csv (rows=771)

[OK] Wrote cleaned person-level: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/all_data_df-aggr-ppl_FIXED_CLEANED.csv (rows=637)

[Heads-up] Some names look empty or single-token; consider alias_map fixes. Examples:

raw_name	clean_name	canonical_name	raw_freq
Marie-Claire Arrieta	Marie	Marie	4
John-Paul Yu	John	John	4
Gang-yu Liu	Gang	Gang	4
Anna-Karin Gustavsson	Anna	Anna	3
Yang-Yu Liu	Yang	Yang	3
Li-Chiang Lin	Li	Li	3
Jaeyun	Jaeyun	Jaeyun	2
Andrew	Andrew	Andrew	2
Soragni	Soragni	Soragni	2
Ying-hui Chou	Ying	Ying	2
BotB1	Botb1	Botb1	2
Wan	Wan	Wan	1
[Unintelligible Speaker]			1
Yevgenia	Yevgenia	Yevgenia	1
bot1	Bot1	Bot 5	1

rebuilding the analysis part

```

In [11]: # =====
# REBUILD ANALYSIS: Charts + Regressions + ROC (robust to collinearity
# Requires: pandas, numpy, matplotlib, statsmodels, scikit-learn, xlsx
# =====

import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import statsmodels.api as sm

```

```

import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_fa
from statsmodels.tools.sm_exceptions import PerfectSeparationError
from patsy import dmatrices
import numpy.linalg as npl

from sklearn.model_selection import StratifiedKFold, cross_validate, c
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, classification_report
from sklearn.impute import SimpleImputer

# ----- CONFIG -----
DATA_CSV = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human
OUT_DIR = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human
OUT_DIR.mkdir(parents=True, exist_ok=True)

RANDOM_STATE = 42
N_SPLITS = 5
INCLUDE_CTX = True # include ctx_* features along with p_* and ann_

# ----- LOAD -----
df = pd.read_csv(DATA_CSV)

# ---- Ensure role flags exist (derive if missing) ----
def ensure_role_flags(df):
    df = df.copy()

    # role_facilitator
    if "role_facilitator" not in df.columns:
        if "facilitator" in df.columns:
            df["role_facilitator"] = (pd.to_numeric(df["facilitator"],
        elif "role_primary" in df.columns:
            df["role_facilitator"] = (df["role_primary"].astype(str).s
        else:
            df["role_facilitator"] = 0

    # role_on_team
    if "role_on_team" not in df.columns:
        if "teams_total" in df.columns:
            df["role_on_team"] = (pd.to_numeric(df["teams_total"], err
        elif "member" in df.columns:
            df["role_on_team"] = (pd.to_numeric(df["member"], errors="
        else:
            df["role_on_team"] = 0

    # role_in_funded
    if "role_in_funded" not in df.columns:
        if "teams_funded" in df.columns:
            df["role_in_funded"] = (pd.to_numeric(df["teams_funded"],

```

```

        else:
            df["role_in_funded"] = 0

    for c in ["role_facilitator", "role_on_team", "role_in_funded"]:
        df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0).astype
    return df

df = ensure_role_flags(df)

# Which flags actually exist?
role_flags_available = [c for c in ["role_facilitator", "role_on_team",

# ----- FEATURE SETS -----
# Person-level behavioral (transcript-derived)
p_feats = [c for c in df.columns if c.startswith("p_") and pd.api.ty
# Annotation categories
ann_feats = [c for c in df.columns if c.startswith("ann_") and pd.api.
# Session-level context (if present)
ctx_feats = [c for c in df.columns if c.startswith("ctx_") and pd.api.
if not INCLUDE_CTX:
    ctx_feats = []

num_feats = sorted(list(set(p_feats + ann_feats + ctx_feats)))

# Drop zero-variance / all-missing features
good_feats = []
for c in num_feats:
    s = pd.to_numeric(df[c], errors="coerce")
    if s.notna().any() and s.std(skipna=True) > 0:
        good_feats.append(c)
num_feats = sorted(good_feats)

print(f"Using {len(num_feats)} numeric features.")

# ----- CHARTS: Helper Plots -----
def bar_mean_with_ci(data, by_flag, metric, title, out_png):
    tmp = data[[by_flag, metric]].dropna()
    tmp[by_flag] = tmp[by_flag].astype(int)
    groups = tmp.groupby(by_flag)[metric]
    means = groups.mean()
    ns = groups.size()
    sds = groups.std()
    cis = 1.96 * sds / np.sqrt(ns.clip(lower=1)) # 95% CI

    fig, ax = plt.subplots(figsize=(5,4))
    ax.bar(["No", "Yes"], [means.get(0, np.nan), means.get(1, np.nan)])
    ax.errorbar([0,1], [means.get(0, np.nan), means.get(1, np.nan)],
                yerr=[cis.get(0,0), cis.get(1,0)], fmt='none', capsize
    ax.set_title(title)
    ax.set_ylabel(metric)
    ax.set_xlabel(by_flag)
    plt.tight_layout()

```

```

fig.savefig(out_png, dpi=180)
plt.close(fig)

def box_by_flag(data, by_flag, metric, title, out_png):
    tmp = data[[by_flag, metric]].dropna()
    tmp[by_flag] = tmp[by_flag].astype(int)
    fig, ax = plt.subplots(figsize=(5,4))
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==1, metric]],
                labels=["No", "Yes"], showfliers=False)
    ax.set_title(title)
    ax.set_ylabel(metric)
    ax.set_xlabel(by_flag)
    plt.tight_layout()
    fig.savefig(out_png, dpi=180)
    plt.close(fig)

# Pick a concise subset of metrics to visualize (add more if you like)
bar_metrics = [
    "p_interruptions_made", "p_overlaps", "p_screenshare_segments", "p_no",
    "ann_knowledge_sharing_count", "ann_evaluation_practices_count",
    "ann_coordination_and_decision_practices_count", "ann_relational_cl",
    "ann_participation_dynamics_count"
]
box_metrics = ["p_speaking_duration_sec", "p_turns"]

# Make charts for each contrast flag that exists
for flag in role_flags_available:
    for m in bar_metrics:
        if m in df.columns:
            fn = OUT_DIR / f"bar_{m}_{flag}.png"
            bar_mean_with_ci(df, flag, m, f"{m} by {flag}", fn)
    for m in box_metrics:
        if m in df.columns:
            fn = OUT_DIR / f"box_{m}_{flag}.png"
            box_by_flag(df, flag, m, f"{m} by {flag}", fn)

print("[OK] Wrote bar/box charts.")

# ----- REGRESSIONS (Logit with FE, full-rank safe) -----
def _drop_problem_cols(X_df):
    """Drop zero-variance, duplicate columns; if still rank-deficient,
    X = X_df.copy()

    # Drop all-constant / zero-variance
    keep = [c for c in X.columns if X[c].std(ddof=0) > 0]
    X = X[keep]

    # Drop exact-duplicate columns
    seen = {}
    dedup_keep = []
    for c in X.columns:
        key = tuple(np.asarray(X[c]).round(12))

```

```

        if key in seen:
            continue
        seen[key] = c
        dedup_keep.append(c)
X = X[dedup_keep]

# Full-rank via QR pivot if needed
A = X.values
rank = np.linalg.matrix_rank(A)
if rank == A.shape[1]:
    return X

Q, R, piv = npl.qr(A, mode='reduced', pivoting=True)
indep_idx = piv[:rank]
keep_cols = [X.columns[i] for i in indep_idx]
return X[keep_cols]

def logit_with_fe(data, target, predictors, fe_conf=True, fe_year=True)
    """
    Build design via patsy, drop collinear columns, fit Logit.
    If MLE fails (singular/complete separation), fall back to fit_regu
    Returns: (data_used, model_or_result, robust_or_none, formula_stri
    """
    # Build formula
    rhs = []
    if add_sess and "sessions_total" in data.columns:
        rhs.append("sessions_total")
    rhs += [f"`{x}`" if (" " in x or "-" in x) else x for x in predict
    if fe_conf and "conference" in data.columns:
        rhs.append("C(conference)")
    if fe_year and "year" in data.columns:
        rhs.append("C(year)")
    formula = f"{target} ~ " + " + ".join(rhs)

    # Design matrices
    try:
        y, X = dmatrices(formula, data=data, return_type="dataframe",
    except Exception as e:
        print(f"[SKIP] Patsy failed for {target}: {e}")
        return None, None, None, formula

    y_vec = np.asarray(y).ravel().astype(int)
    if np.unique(y_vec).size < 2:
        print(f"[SKIP] {target}: only one class after NA drop.")
        return None, None, None, formula

    # Hold intercept, clean the rest
    const = None
    if "Intercept" in X.columns:
        const = X["Intercept"].copy()
        X = X.drop(columns=["Intercept"])
    X = _drop_problem_cols(X)

```

```

if const is not None:
    X.insert(0, "Intercept", const.values)

# Fit (request robust SEs directly); fallback to regularized if ne
try:
    model = sm.Logit(y_vec, X)
    result = model.fit(dis=0, cov_type="HC3") # <-- robust SEs
    robust = result # already robust
    used = pd.concat([y.reset_index(drop=True), X.reset_index(drop
    return used, result, robust, formula
except (np.linalg.LinAlgError, PerfectSeparationError) as e:
    print(f"[Info] MLE failed for {target} ({type(e).__name__}); t
    model = sm.Logit(y_vec, X)
    result = model.fit_regularized(alpha=1.0, L1_wt=0.5, dis=0)
    robust = None # robust SEs not available here
    used = pd.concat([y.reset_index(drop=True), X.reset_index(drop
    return used, result, robust, formula

def tidy_from_result(result, robust_or_none, model_label):
    params = (robust_or_none.params if robust_or_none is not None else
    bse = (robust_or_none.bse if robust_or_none is not None else
    pvals = (robust_or_none.pvalues if robust_or_none is not None els
    conf = (robust_or_none.conf_int() if robust_or_none is not None

    out = pd.DataFrame({
        "predictor": params.index,
        "coef": params.values,
        "std_err": bse.values,
        "p_value": pvals.values,
        "ci_low": conf[0].values,
        "ci_high": conf[1].values,
        "model": model_label,
        "n_obs": int(result.nobs),
        "llf": result.llf,
        "llnull": getattr(result, "llnull", np.nan),
        "pseudo_r2": (1 - (result.llf / result.llnull)) if getattr(res
        "llr_pvalue": getattr(result, "llr_pvalue", np.nan)
    })
    return out

tidy_list = []
summaries = []

for tgt, desc in [
    ("role_facilitator", "Facilitator vs Non-facilitator"),
    ("role_on_team", "On-team vs Not-on-team"),
    ("role_in_funded", "Funded vs Not-funded")
]:
    if tgt not in role_flags_available:
        continue
    used, res, rob, form = logit_with_fe(df, tgt, predictors=num_feats
    if res is None:

```

```

        continue

    # summary text
    summ = [f"=== {desc} ({tgt}) ===", f"Formula: {form}", res.summary]
    if rob is None:
        summ.append("\nNote: Regularized fit used (robust SEs not available)")
    else:
        summ.append("\nRobust (HC3) SEs used in tidy table.")
    summaries.append("\n".join(summ))

    tidy_list.append(tidy_from_result(res, rob, desc))

# Save tidy combined and summaries
if tidy_list:
    tidy_df = pd.concat(tidy_list, ignore_index=True)
    tidy_csv = OUT_DIR / "regression_tidy_results.csv"
    tidy_df.to_csv(tidy_csv, index=False)

    with open(OUT_DIR / "regression_model_summaries.txt", "w") as f:
        f.write("\n\n".join(summaries))

# Nicely formatted per-model tables
def tidy_to_display(df_tidy, model_name):
    sub = df_tidy[df_tidy["model"]==model_name].copy()
    sub.loc[sub["predictor"].str.startswith("C(conference)"), "predictor"] = "C(conference)"
    sub.loc[sub["predictor"].str.startswith("C(year)"), "predictor"] = "C(year)"
    sub["Coef (SE)"] = sub["coef"].round(3).astype(str) + " (" + sub["se"].round(3).astype(str) + ")"
    sub["p"] = sub["p_value"].apply(lambda x: f"{x:.3g}")
    keep = ["predictor", "Coef (SE)", "ci_low", "ci_high", "p", "n_obs"]
    sub = sub[keep].rename(columns={"predictor": "Term", "ci_low": "CI_Lower", "ci_high": "CI_Upper"})
    return sub

tables = {}
for model_name in tidy_df["model"].unique():
    disp = tidy_to_display(tidy_df, model_name)
    safe = "".join(ch for ch in model_name if ch.isalnum() or ch in " . _")
    disp.to_csv(OUT_DIR / f"reg_table_{safe}.csv", index=False)
    try:
        disp.to_markdown(OUT_DIR / f"reg_table_{safe}.md", index=False)
    except Exception:
        pass
    tables[model_name] = disp

xlsx_path = OUT_DIR / "regression_tables_by_model.xlsx"
with pd.ExcelWriter(xlsx_path, engine="xlsxwriter") as xw:
    for model_name, disp in tables.items():
        sheet = model_name[:31]
        disp.to_excel(xw, index=False, sheet_name=sheet)

print(f"[OK] Regressions saved: {tidy_csv} and {xlsx_path}")
else:
    print("[SKIP] No regression models were fit (missing targets or si")

```

```

# ----- MULTICOLLINEARITY (VIF on key behaviors) -----
vif_feats = [c for c in ["p_speaking_duration_sec", "p_turns", "p_interr",
                        "p_screenshare_segments", "p_smile_self_total",
                        "p_nods_received"] if c in df.columns]

if vif_feats:
    X_vif = df[vif_feats].fillna(0)
    X_vif = sm.add_constant(X_vif)
    vif_table = pd.DataFrame({
        "feature": X_vif.columns,
        "VIF": [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1]-1)]
    })
    vif_table.to_csv(OUT_DIR / "vif_behaviors.csv", index=False)
    print("[OK] VIF table written.")
else:
    print("[SKIP] No VIF – none of the key behavior features found.")

# ----- PREDICTIVE: ROC Curves with behavior-only -----
targets = [(t, d) for (t, d) in [
    ("role_facilitator", "Facilitator vs Non-facilitator"),
    ("role_on_team", "On-team vs Not-on-team"),
    ("role_in_funded", "Funded vs Not-funded")
] if t in role_flags_available]

X_cols = num_feats # p_, ann_, and (ctx_ if INCLUDE_CTX)
if targets and X_cols:
    numeric_transform = Pipeline(steps=[
        ("impute", SimpleImputer(strategy="constant", fill_value=0.0)),
        ("scale", StandardScaler())
    ])
    preprocess = ColumnTransformer(transformers=[("num", numeric_transform, X_cols)])

    logit = LogisticRegression(
        penalty="l2",
        solver="liblinear",
        class_weight="balanced",
        max_iter=2000,
        random_state=RANDOM_STATE
    )
    clf = Pipeline(steps=[("prep", preprocess), ("clf", logit)])
    cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)

    summary_rows = []
    for target, desc in targets:
        dsub = df.dropna(subset=[target]).copy()
        if dsub[target].nunique() < 2:
            print(f"[SKIP] ROC for {target}: single class present.")
            continue

        y = dsub[target].astype(int).values
        X = dsub[X_cols]

```

```

# CV probs + ROC
y_prob = cross_val_predict(clf, X, y, cv=cv, method="predict_p
fpr, tpr, _ = roc_curve(y, y_prob)
roc_auc = auc(fpr, tpr)

# Classification report at 0.5
y_hat = (y_prob >= 0.5).astype(int)
cr = classification_report(y, y_hat, output_dict=True, zero_di
pd.DataFrame(cr).to_csv(OUT_DIR / f"clf_classification_report_

# CV metric summary
scoring = {"roc_auc": "roc_auc", "accuracy": "accuracy", "precisi
cv_res = cross_validate(clf, X, y, cv=cv, scoring=scoring, ret
row = {"target": target, "description": desc}
for m in scoring:
    row[f"{m}_mean"] = float(cv_res[f"test_{m}"].mean())
    row[f"{m}_sd"] = float(cv_res[f"test_{m}"].std())
summary_rows.append(row)

# ROC plot
fig, ax = plt.subplots(figsize=(5,4))
ax.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
ax.plot([0,1],[0,1], linestyle="--")
ax.set_title(f"ROC - {desc}")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.legend(loc="lower right")
plt.tight_layout()
fig.savefig(OUT_DIR / f"clf_roc_{target}.png", dpi=180)
plt.close(fig)

if summary_rows:
    summary_df = pd.DataFrame(summary_rows).sort_values("target")
    summary_df.to_csv(OUT_DIR / "clf_results_summary.csv", index=F
    print("[OK] Predictive ROC + summaries written.")
else:
    print("[SKIP] ROC - no targets available or no numeric features.")

print("\nOutputs written to:", OUT_DIR)

```

Using 33 numeric features.

```

/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_68986/346889
039.py:124: MatplotlibDeprecationWarning: The 'labels' parameter of box
plot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==
1, metric]],
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_68986/346889
039.py:124: MatplotlibDeprecationWarning: The 'labels' parameter of box
plot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==
1, metric]],
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_68986/346889
039.py:124: MatplotlibDeprecationWarning: The 'labels' parameter of box
plot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==
1, metric]],
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_68986/346889
039.py:124: MatplotlibDeprecationWarning: The 'labels' parameter of box
plot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==
1, metric]],
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_68986/346889
039.py:124: MatplotlibDeprecationWarning: The 'labels' parameter of box
plot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
    ax.boxplot([tmp.loc[tmp[by_flag]==0, metric], tmp.loc[tmp[by_flag]==
1, metric]],
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered i
n exp
    return 1/(1+np.exp(-X))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2443: RuntimeWarning: divide by zero encount
ered in log
    return np.sum(np.log(self.cdf(q * linpred)))

```

```

[OK] Wrote bar/box charts.
[Info] MLE failed for role_facilitator (LinAlgError); trying regularize
d fit.
[SKIP] role_in_funded: only one class after NA drop.
[OK] Regressions saved: /Users/maxchalekson/Projects/NICO-Research/NICO
_human-gemini/facilitator-identification/outputs_rebuild/regression_tid
y_results.csv and /Users/maxchalekson/Projects/NICO-Research/NICO_human
-gemini/facilitator-identification/outputs_rebuild/regression_tables_by
_model.xlsx
[OK] VIF table written.
[SKIP] ROC for role_in_funded: single class present.
[OK] Predictive ROC + summaries written.

```

Outputs written to: /Users/maxchalekson/Projects/NICO-Research/NICO_hum
an-gemini/facilitator-identification/outputs_rebuild

analysis - 3 categorical

```

In [18]: # =====
# Meeting-level analysis (person-year data)
# - Outlier checks for controls
# - Single-feature OLS (HC3 SE) on num_teams / num_funded_teams
# - Tidy CSVs + Excel
# - Coef barplots with 95% CI
# - Big grid image: hist + scatters for each feature
# =====

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import math

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_fa

# ----- CONFIG -----
DATA_CSV = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human
OUT_DIR = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Which outcomes to model (must exist in the CSV)
OUTCOMES = ["num_teams", "num_funded_teams"]

# Which controls to include in every single-feature regression
CONTROLS = ["p_speaking_duration_sec", "p_turns"] # (Evey's suggestio

# Number of bars to show (by smallest p) in coef barplots
TOP_N_FOR_PLOT = 15

RANDOM_STATE = 42

```

```

# ----- LOAD -----
df = pd.read_csv(DATA_CSV)

# Basic hygiene: make sure outcomes exist & are numeric
for c in OUTCOMES:
    if c not in df.columns:
        raise ValueError(f"Outcome '{c}' not found in {DATA_CSV}.")
    df[c] = pd.to_numeric(df[c], errors="coerce")

# Predictors = p_* and ann_* numeric columns
p_feats = [c for c in df.columns if c.startswith("p_") and pd.api.
ann_feats = [c for c in df.columns if c.startswith("ann_") and pd.api.
predictors = sorted(list(set(p_feats + ann_feats)))

# Drop outcomes / predictors that are entirely missing
df = df.copy()
df[predictors + OUTCOMES] = df[predictors + OUTCOMES].apply(pd.to_nume

# ----- Quick outlier checks for controls -----
def quick_outlier_plots(data: pd.DataFrame, cols: list, out_dir: Path)
    for c in cols:
        if c not in data.columns:
            continue
        s = pd.to_numeric(data[c], errors="coerce")
        fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
        # Histogram (log x if long tail)
        ax[0].hist(s.dropna(), bins=40)
        ax[0].set_title(f"{c} - histogram")
        ax[0].set_xlabel(c); ax[0].set_ylabel("count")
        # Boxplot (shows extreme points)
        ax[1].boxplot(s.dropna(), vert=True, showfliers=True)
        ax[1].set_title(f"{c} - boxplot")
        ax[1].set_ylabel(c)
        plt.tight_layout()
        fig.savefig(out_dir / f"outlier_{c}.png", dpi=180)
        plt.close(fig)

# scatter against outcomes (to see leverage)
def scatter_vs_outcomes(data: pd.DataFrame, cols: list, outcomes: list
    for c in cols:
        if c not in data.columns:
            continue
        for y in outcomes:
            sub = data[[c, y]].dropna()
            if sub.empty:
                continue
            fig, ax = plt.subplots(figsize=(4.8, 3.6))
            ax.scatter(sub[c], sub[y], s=10, alpha=0.6)
            ax.set_xlabel(c); ax.set_ylabel(y)
            ax.set_title(f"{y} vs {c}")
            plt.tight_layout()

```

```

        fig.savefig(out_dir / f"scatter_{y}_vs_{c}.png", dpi=180)
        plt.close(fig)

quick_outlier_plots(df, CONTROLS, OUT_DIR)
scatter_vs_outcomes(df, CONTROLS, OUTCOMES, OUT_DIR)

# ----- Utility: single-feature OLS with HC3 robust SE -----
def run_single_feature_ols(data: pd.DataFrame, outcome: str, feature:
    cols_base = [outcome, feature]
    # exclude the focal feature from the controls to avoid duplicates
    controls_eff = [c for c in controls if (c in data.columns and c !=
    cols = cols_base + controls_eff

    d = data[cols].dropna()
    if d[outcome].nunique() <= 1:
        return None # cannot regress on constant outcome

    y = d[outcome].astype(float).values
    X = d[[feature] + controls_eff].astype(float)
    X = sm.add_constant(X, has_constant="add")

    model = sm.OLS(y, X).fit(cov_type="HC3") # HC3 robust SE

    # now `feature` is unique in X, so these are scalars
    out = {
        "outcome": outcome,
        "feature": feature,
        "coef": float(model.params[feature]),
        "std_err": float(model.bse[feature]),
        "t": float(model.tvalues[feature]),
        "p_value": float(model.pvalues[feature]),
        "ci_low": float(model.conf_int().loc[feature, 0]),
        "ci_high": float(model.conf_int().loc[feature, 1]),
        "N": int(model.nobs),
        "R2": float(model.rsquared),
        "Adj_R2": float(model.rsquared_adj),
    }
    return out

# ----- Run regressions for each outcome -----
all_rows = []
for outcome in OUTCOMES:
    for feat in predictors:
        res = run_single_feature_ols(df, outcome, feat, CONTROLS)
        if res is not None:
            all_rows.append(res)

if not all_rows:
    raise RuntimeError("No models were fit – check your outcomes and p

results_df = pd.DataFrame(all_rows)
results_df = results_df.sort_values(["outcome", "p_value", "feature"])

```

```

# Write CSVs per outcome + combined + Excel workbook
results_df.to_csv(OUT_DIR / "single_feature_regressions_ALL.csv", index=False)
for outcome in OUTCOMES:
    sub = results_df[results_df["outcome"] == outcome].copy()
    sub.to_csv(OUT_DIR / f"single_feature_regressions_{outcome}.csv", index=False)

with pd.ExcelWriter(OUT_DIR / "single_feature_regressions_by_outcome.xlsx"):
    for outcome in OUTCOMES:
        sub = results_df[results_df["outcome"] == outcome].copy()
        sub.to_excel(xw, sheet_name=outcome, index=False)

print(f"[OK] wrote regression tables to: {OUT_DIR}")

# ----- Coefficient barplots with 95% CI -----
def coef_barplot(results: pd.DataFrame, outcome: str, topn: int, out_dir: str):
    sub = results[results["outcome"] == outcome].copy()
    if sub.empty:
        return
    # Choose topn by smallest p-value
    sub = sub.nsmallest(topn, "p_value")
    sub = sub.sort_values("coef") # nicest for bars

    y = np.arange(len(sub))
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.barh(y, sub["coef"], xerr=1.96*sub["std_err"], alpha=0.8)
    ax.set_yticks(y, labels=sub["feature"])
    ax.set_xlabel("Coefficient (OLS, HC3 SE)")
    ax.set_title(f"Coefficients with 95% CI - {outcome}")
    for i, (coef) in enumerate(sub["coef"].round(3)):
        ax.text(coef, i, f"b = {coef:.3f}", va="center")
    plt.tight_layout()
    fig.savefig(out_dir / f"coef_bar_{outcome}.png", dpi=200)
    plt.close(fig)

for outcome in OUTCOMES:
    coef_barplot(results_df, outcome, TOP_N_FOR_PLOT, OUT_DIR)

# ----- Big "feature grid" image (hist + two scatters) -----
def feature_grid_plot(data: pd.DataFrame, feats: list, outcomes: list,
    """
    For each feature: col0 = histogram, col1 = scatter vs outcomes[0],
    Paginates across multiple JPEGs if needed.
    """
    feats = [f for f in feats if f in data.columns]
    pages = math.ceil(len(feats) / per_page)
    for page in range(pages):
        chunk = feats[page*per_page : (page+1)*per_page]
        nrows = len(chunk)
        ncols = 3
        fig, axes = plt.subplots(nrows, ncols, figsize=(12, 3.2*nrows))
        for r, f in enumerate(chunk):

```

```

s = pd.to_numeric(data[f], errors="coerce").dropna()
# 0) histogram
axes[r,0].hist(s, bins=40)
axes[r,0].set_title(f"{f} - distribution")
axes[r,0].set_ylabel("count")
# 1) scatter vs first outcome
if outcomes[0] in data.columns:
    sub = data[[f, outcomes[0]]].dropna()
    axes[r,1].scatter(sub[f], sub[outcomes[0]], s=8, alpha
    axes[r,1].set_title(f"{outcomes[0]} vs {f}")
    axes[r,1].set_ylabel(outcomes[0])
# 2) scatter vs second outcome
if len(outcomes) > 1 and outcomes[1] in data.columns:
    sub = data[[f, outcomes[1]]].dropna()
    axes[r,2].scatter(sub[f], sub[outcomes[1]], s=8, alpha
    axes[r,2].set_title(f"{outcomes[1]} vs {f}")
    axes[r,2].set_ylabel(outcomes[1])

for c in range(ncols):
    axes[r,c].set_xlabel(f)

plt.tight_layout(h_pad=1.2, w_pad=1.2)
out_path = out_dir / f"feature_analysis_grid_p{page+1}.jpeg"
fig.savefig(out_path, dpi=160, pil_kwargs={"quality": 90})
plt.close(fig)

# Build the grid over all predictors
feature_grid_plot(df, predictors, OUTCOMES, OUT_DIR, per_page=12)

# ----- VIF for controls (sanity check) -----
vif_feats = [c for c in CONTROLS if c in df.columns]
if len(vif_feats) >= 2:
    X_vif = df[vif_feats].dropna().astype(float)
    X_vif = sm.add_constant(X_vif)
    vif_tbl = pd.DataFrame({
        "feature": X_vif.columns,
        "VIF": [variance_inflation_factor(X_vif.values, i) for i in ra
    })
    vif_tbl.to_csv(OUT_DIR / "vif_controls.csv", index=False)
    print("[OK] VIF written:", OUT_DIR / "vif_controls.csv")
else:
    print("[SKIP] VIF - need at least 2 control variables present.")

print("\nDone. Outputs in:", OUT_DIR)

```

[OK] wrote regression tables to: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level

[OK] VIF written: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/vif_controls.csv

Done. Outputs in: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level

fixing the graph visually (so names do not run onto each other on title)

In []:

```
[OK] Wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean/coef_bar_num_teams_clean.png
[OK] Wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean/coef_bar_num_funded_clean.png
[OK] Wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean/coef_bar_facilitator_logit_clean.png
[OK] Wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean/feature_analysis_grid_clean.png
```

All set! Figure pack at: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean/FigurePack.pdf

Cleaned figures in: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/outputs_meeting_level/figures_clean

```
In [22]: import matplotlib.pyplot as plt
import textwrap
from matplotlib.backends.backend_pdf import PdfPages

def safe_title(text, width=25, fontsize=7):
    """Wrap long titles and shrink fontsize if needed."""
    return "\n".join(textwrap.wrap(text, width)), fontsize

# Collect features (p_* and ann_*)
features_to_plot = [c for c in df.columns
                    if (c.startswith("ann_") or c.startswith("p_"))
                    and pd.api.types.is_numeric_dtype(df[c])]

pdf_out = OUT_DIR / "feature_analysis_all.pdf"
with PdfPages(pdf_out) as pdf:
    # batch 9 features per page (3 rows x 3 cols)
    batch_size = 9
    for i in range(0, len(features_to_plot), batch_size):
        batch = features_to_plot[i:i+batch_size]

        fig, axes = plt.subplots(3, 3, figsize=(12, 12))
        axes = axes.flatten()

        for ax, feat in zip(axes, batch):
            if feat not in df.columns:
                continue
```

```

        # Example: histogram for distribution
        ax.hist(df[feat].dropna(), bins=30, color="steelblue", alp

        # safe wrapped title
        t, fs = safe_title(f"{feat} - distribution")
        ax.set_title(t, fontsize=fs)

        # Remove any unused subplots in the grid
        for ax in axes[len(batch):]:
            ax.axis("off")

        plt.tight_layout()
        pdf.savefig(fig)
        plt.close(fig)

    print(f"[OK] Combined plots saved to: {pdf_out}")

```

[OK] Combined plots saved to: outputs_meeting_level_cleaned_titles/feature_analysis_all.pdf

regression analysis - w/ & w/out controls

```

In [1]: # =====
# REGRESSION COMPARISON: with vs. without controls
# File assumptions:
#   - Person-level rows
#   - Outcomes: num_teams, num_funded_teams (already created)
#   - Annotation features start with "ann_"
#   - Controls (per Evey's hunch): p_speaking_duration_sec, p_turns
#   - Optional FE: conference, year
# Outputs:
#   - ./outputs_meeting_level/compare_controls_*.csv
#   - ./outputs_meeting_level/compare_controls.xlsx
#   - ./outputs_meeting_level/compare_controls_summary.txt
#   - ./outputs_meeting_level/coef_compare_<outcome>.png
# =====

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from patsy import dmatrices

# ----- paths -----
DATA_CSV = Path("ALL_person_year_WITH_OUTCOMES.csv") # use your latest
OUT_DIR = Path("outputs_meeting_level")
OUT_DIR.mkdir(parents=True, exist_ok=True)

# ----- load -----

```

```

df = pd.read_csv(DATA_CSV)

# keep only numeric ann_ features (drop all-NA or zero-variance later)
ann_feats = [c for c in df.columns if c.startswith("ann_") and pd.api.
controls = [c for c in ["p_speaking_duration_sec", "p_turns"] if c in
fe_conf = "conference" in df.columns
fe_year = "year" in df.columns

# small hygiene
for c in ["num_teams", "num_funded_teams"]:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

# drop ann_ features that are all missing or constant
good_ann = []
for c in ann_feats:
    s = pd.to_numeric(df[c], errors="coerce")
    if s.notna().sum() > 1 and s.std(skipna=True) > 0:
        good_ann.append(c)
ann_feats = sorted(good_ann)

print(f"Annotation predictors: {len(ann_feats)}; Controls present: {co

def fit_ols_hc3(data, outcome, rhs_terms):
    """Fit OLS with HC3 robust SE via patsy design; returns result and
    rhs = " + ".join(rhs_terms)
    formula = f"{outcome} ~ {rhs}"
    y, X = dmatrixes(formula, data=data, return_type="dataframe", NA_a
    if y.shape[0] < 20:
        return None, None, formula # too few rows to be meaningful
    model = sm.OLS(y, X).fit(cov_type="HC3")
    # tidy
    ci = model.conf_int()
    tidy = pd.DataFrame({
        "term": model.params.index,
        "coef": model.params.values,
        "std_err": model.bse.values,
        "p_value": model.pvalues.values,
        "ci_low": ci[0].values,
        "ci_high": ci[1].values,
    })
    tidy["N"] = int(model.nobs)
    tidy["R2"] = float(model.rsquared)
    tidy["Adj_R2"] = float(model.rsquared_adj)
    return model, tidy, formula

def make_rhs(include_controls: bool):
    rhs = ann_feats.copy()
    if include_controls:
        rhs += controls
    if fe_conf: rhs += ["C(conference)"]
    if fe_year: rhs += ["C(year)"]

```

```

    return rhs

outcomes = [c for c in ["num_teams", "num_funded_teams"] if c in df.col

all_tables = {}
summary_lines = []

for y in outcomes:
    # (A) annotations only
    rhs_A = make_rhs(include_controls=False)
    modA, tidyA, formA = fit_ols_hc3(df, y, rhs_A)

    # (B) annotations + controls
    rhs_B = make_rhs(include_controls=True)
    modB, tidyB, formB = fit_ols_hc3(df, y, rhs_B)

    if (modA is None) or (modB is None):
        print(f"[SKIP] Not enough rows for {y}")
        continue

    # Save raw tidy tables
    a_csv = OUT_DIR / f"compare_controls_{y}_A_ann_only.csv"
    b_csv = OUT_DIR / f"compare_controls_{y}_B_ann_plus_controls.csv"
    tidyA.to_csv(a_csv, index=False)
    tidyB.to_csv(b_csv, index=False)

    # Build a merged comparison for overlapping terms (esp. ann_terms
    dispA = tidyA[["term", "coef", "std_err", "p_value", "ci_low", "ci_high",
        columns={"coef": "coef_A", "std_err": "se_A", "p_value": "p_A", "ci_
            "R2": "R2_A", "Adj_R2": "Adj_R2_A", "N": "N_A"}
    )
    dispB = tidyB[["term", "coef", "std_err", "p_value", "ci_low", "ci_high",
        columns={"coef": "coef_B", "std_err": "se_B", "p_value": "p_B", "ci_
            "R2": "R2_B", "Adj_R2": "Adj_R2_B", "N": "N_B"}
    )
    merged = dispA.merge(dispB, on="term", how="outer")
    # annotate deltas
    merged["delta_coef"] = merged["coef_B"] - merged["coef_A"]
    merged["abs_pct_change_coef"] = np.where(
        merged["coef_A"].abs() > 1e-12,
        (merged["coef_B"] - merged["coef_A"]).abs() / merged["coef_A"]
        np.nan
    )
    # Add overall model deltas (repeat on every row for convenience)
    merged["delta_R2"] = merged["R2_B"].iloc[0] - merged["R2_A"].iloc[0]
    merged["delta_Adj_R2"] = merged["Adj_R2_B"].iloc[0] - merged["Adj_R2_A"].iloc[0]

    comp_csv = OUT_DIR / f"compare_controls_{y}_MERGED.csv"
    merged.to_csv(comp_csv, index=False)
    all_tables[y] = merged

    # Summary text line

```

```

summary_lines.append(
    f"[{y}] R²: {merged['R2_A'].iloc[0]:.3f} → {merged['R2_B'].iloc[0]:.3f}; "
    f"(Δ {merged['delta_R2'].iloc[0]:+.3f}); "
    f"Adj.R²: {merged['Adj_R2_A'].iloc[0]:.3f} → {merged['Adj_R2_B'].iloc[0]:.3f}; "
    f"(Δ {merged['delta_Adj_R2'].iloc[0]:+.3f})."
)

# Quick coefficient comparison plot (controls + top 10 ann_ by |coef|)
plot_terms = []
# controls first (if present in B)
for c in controls:
    if (merged["term"]==c).any():
        plot_terms.append(c)
# then top 10 ann_ by absolute coef in model B
ann_in_B = merged[merged["term"].str.startswith("ann_")].copy()
ann_in_B["rank"] = ann_in_B["coef_B"].abs().rank(ascending=False,
top_ann = ann_in_B.sort_values("rank").head(10)["term"].tolist()
plot_terms += [t for t in top_ann if t not in plot_terms]

sub = merged[merged["term"].isin(plot_terms)].copy()
sub = sub.sort_values("coef_B")

fig, ax = plt.subplots(figsize=(10, 6))
y_pos = np.arange(len(sub))
ax.errorbar(sub["coef_A"], y_pos+0.15, xerr=sub["se_A"]*1.96, fmt="k", lw=1)
ax.errorbar(sub["coef_B"], y_pos-0.15, xerr=sub["se_B"]*1.96, fmt="k", lw=1)
ax.axvline(0, color="k", lw=1)
ax.set_yticks(y_pos)
ax.set_yticklabels(sub["term"])
ax.set_xlabel("Coefficient (HC3 SE)")
ax.set_title(f"Coefficient comparison - {y}")
ax.legend(loc="lower right")
plt.tight_layout()
fig.savefig(OUT_DIR / f"coef_compare_{y}.png", dpi=200)
plt.close(fig)

# one Excel with both merged tables (if available)
if all_tables:
    xlsx = OUT_DIR / "compare_controls.xlsx"
    with pd.ExcelWriter(xlsx, engine="xlsxwriter") as xw:
        for y, tab in all_tables.items():
            sheet = y[:31]
            tab.to_excel(xw, index=False, sheet_name=sheet)

# summary text
with open(OUT_DIR / "compare_controls_summary.txt", "w") as f:
    f.write("Controls tested: " + ", ".join(controls) + "\n")
    f.write("\n".join(summary_lines) if summary_lines else "No models")

print("Done. See:", OUT_DIR)

```

Annotation predictors: 25; Controls present: ['p_speaking_duration_sec', 'p_turns']
Done. See: outputs_meeting_level

I should have most the information for the analysis EV was mentioning on 09/29 (refer to notes if needbe)- combining all together now

```
In [6]: # =====  
# CONSOLIDATE & SUMMARIZE (robust logit)  
# =====  
from pathlib import Path  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import textwrap  
import statsmodels.api as sm  
import statsmodels.formula.api as smf  
from patsy import dmatrices  
from statsmodels.tools.sm_exceptions import PerfectSeparationError  
import numpy.linalg as npl  
  
from sklearn.model_selection import StratifiedKFold, cross_val_predict  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import StandardScaler  
from sklearn.pipeline import Pipeline  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import roc_curve, auc  
from sklearn.impute import SimpleImputer  
  
# ----- CONFIG -----  
DATA_CSV = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human  
OUT_DIR = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human  
OUT_DIR.mkdir(parents=True, exist_ok=True)  
  
INCLUDE_CTX = True  
RANDOM_STATE = 42  
N_SPLITS = 5  
  
OUTCOMES = [  
    ("role_facilitator", "Facilitator vs Non-facilitator"), # binary  
    ("num_teams", "Number of teams (count)", # numeri  
    ("num_funded_teams", "Number of funded teams (count)" # numeri  
]  
CONTROLS = ["p_speaking_duration_sec", "p_turns"]  
  
# ----- LOAD -----  
df = pd.read_csv(DATA_CSV)  
for col in ["role_facilitator", "num_teams", "num_funded_teams", "year", "  
    if col in df.columns:
```

```

        df[col] = pd.to_numeric(df[col], errors="coerce")
    if "role_facilitator" not in df.columns and "facilitator" in df.columns:
        df["role_facilitator"] = (pd.to_numeric(df["facilitator"], errors="coerce")
                                   .fillna(0).astype(int))

    p_feats = [c for c in df.columns if c.startswith("p_") and pd.api.
                is_numeric_dtype(df[c])]
    ann_feats = [c for c in df.columns if c.startswith("ann_") and pd.api.
                 is_numeric_dtype(df[c])]
    ctx_feats = [c for c in df.columns if c.startswith("ctx_") and pd.api.
                 is_numeric_dtype(df[c])]
    if not INCLUDE_CTX: ctx_feats = []

    def nonconstant(cols):
        out = []
        for c in cols:
            s = pd.to_numeric(df[c], errors="coerce")
            if s.notna().any() and s.std(skipna=True) > 0:
                out.append(c)
        return out
    p_feats, ann_feats, ctx_feats = map(nonconstant, [p_feats, ann_feats,
                                                       ctx_feats])

    # ----- helpers -----
    def wrap_label(txt, width=24):
        import textwrap
        return "\n".join(textwrap.wrap(str(txt), width=width))

    def build_formula(outcome, predictors, controls=None, add_fe=True):
        rhs = []
        if controls: rhs += controls
        rhs += predictors
        if add_fe:
            if "conference" in df.columns: rhs.append("C(conference)")
            if "year" in df.columns: rhs.append("C(year)")
            if "sessions_total" in df.columns: rhs.append("sessions_total")
        return f"{outcome} ~ " + (" + ".join(rhs) if rhs else "1")

    def iqr_mask(x, k=1.5):
        x = pd.to_numeric(x, errors="coerce")
        if not x.notna().any(): return pd.Series(True, index=x.index)
        q1, q3 = np.nanpercentile(x.dropna(), [25, 75])
        lo, hi = q1 - k*(q3-q1), q3 + k*(q3-q1)
        return (x >= lo) & (x <= hi)

    def trim_outliers_on_controls(df_in, controls):
        if not controls:
            return df_in.copy(), pd.Series(False, index=df_in.index)
        m = pd.Series(True, index=df_in.index)
        for c in controls:
            if c in df_in.columns: m &= iqr_mask(df_in[c])
        return df_in.loc[m].copy(), ~m

    def roc_cv_auc(data, target, X_cols):
        d = data.dropna(subset=[target]).copy()
        if d[target].nunique() < 2: return None, None, None
        y = d[target].astype(int).values

```

```

X = d[X_cols].copy()
numeric_transform = Pipeline(steps=[
    ("impute", SimpleImputer(strategy="constant", fill_value=0.0))
    ("scale", StandardScaler())
])
preprocess = ColumnTransformer([("num", numeric_transform, X_cols)]
clf = Pipeline([("prep", preprocess),
    ("clf", LogisticRegression(penalty="l2", solver="lbfgs",
                               class_weight="balanced",
                               max_iter=2000, random_state=0))
])
cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=0)
y_prob = cross_val_predict(clf, X, y, cv=cv, method="predict_proba")
fpr, tpr, _ = roc_curve(y, y_prob)
return auc(fpr, tpr), fpr, tpr

def tidy_from_result(result, model_label):
    params = result.params; bse = result.bse; pvals = result.pvalues;
    out = pd.DataFrame({
        "model": model_label, "term": params.index,
        "coef": params.values, "std_err": bse.values, "p_value": pvals,
        "ci_low": conf[0].values, "ci_high": conf[1].values,
        "N": int(result.nobs), "llf": getattr(result, "llf", np.nan),
        "llnull": getattr(result, "llnull", np.nan)
    })
    out["pseudo_r2"] = np.where(out["llnull"].notna() & (out["llnull"]
        1 - (out["llf"]/out["llnull"])), np.nan)

    return out

# --- NEW: full-rank safe logit with regularized fallback ---
def fit_logit_safe(formula, data):
    """
    1) Use patsy to create y, X.
    2) Drop collinear columns via QR pivoting to full rank.
    3) Fit Logit; if singular / separation, fall back to fit_regularized
    Returns a statsmodels results object compatible with .params, .bse
    """
    # build design
    y, X = dmatrixes(formula, data=data, return_type="dataframe", NA_as_nan=True)
    y = np.asarray(y).ravel().astype(int)
    if np.unique(y).size < 2:
        raise ValueError("Binary target has a single class after NA drop")
    # keep intercept aside
    const = None
    if "Intercept" in X.columns:
        const = X["Intercept"].copy()
        X = X.drop(columns=["Intercept"])
    # QR select independent columns
    A = X.values
    if A.size == 0:
        X_full = pd.DataFrame({"Intercept": const}) if const is not None
    else:
        rank = np.linalg.matrix_rank(A)

```

```

    if rank < A.shape[1]:
        Q, R, piv = npl.qr(A, mode="reduced", pivoting=True)
        keep_idx = piv[:rank]
        X = X.iloc[:, keep_idx]
        # reattach intercept as first col
    if const is not None:
        X.insert(0, "Intercept", const.values)
    else:
        X.insert(0, "Intercept", 1.0)
    X_full = X

    try:
        model = sm.Logit(y, X_full)
        return model.fit(dis=0)
    except (np.linalg.LinAlgError, PerfectSeparationError):
        # fallback: elastic-net style regularization
        model = sm.Logit(y, X_full)
        res = model.fit_regularized(alpha=1.0, L1_wt=0.5, dis=0)
        return res

# ----- outliers on controls -----
controls_present = [c for c in CONTROLS if c in df.columns]
trimmed_df, outlier_mask = trim_outliers_on_controls(df, controls_pres

# ----- models + plots -----
all_tables = []
for outcome, desc in OUTCOMES:
    if outcome not in df.columns:
        continue

    is_binary = df[outcome].dropna().isin([0,1]).all()
    ann_pred = ann_feats.copy()
    if not ann_pred:
        continue

    ctl = [c for c in CONTROLS if c in df.columns]
    f_noctl = build_formula(outcome, ann_pred, controls=None, add_fe
    f_withctl = build_formula(outcome, ann_pred, controls=ctl, add_fe

    if is_binary:
        try:
            m1 = fit_logit_safe(f_noctl, df)
            m2 = fit_logit_safe(f_withctl, df)
            m3 = fit_logit_safe(f_noctl, trimmed_df)
            m4 = fit_logit_safe(f_withctl, trimmed_df)
        except ValueError:
            # single-class after filtering - skip
            continue
    else:
        m1 = smf.ols(f_noctl, data=df).fit(cov_type="HC3")
        m2 = smf.ols(f_withctl, data=df).fit(cov_type="HC3")
        m3 = smf.ols(f_noctl, data=trimmed_df).fit(cov_type="HC3")

```

```

m4 = smf.ols(f_withctl, data=trimmed_df).fit(cov_type="HC3")

label = lambda s: f"{desc} - {s}"
for res, lbl in [(m1,"ANN only, full"),(m2,"ANN+controls, full"),
                 (m3,"ANN only, trimmed"),(m4,"ANN+controls, trimm
t = tidy_from_result(res, label(lbl))
t.insert(0, "outcome", outcome)
all_tables.append(t)

# save tidy
if all_tables:
    pd.concat(all_tables, ignore_index=True).to_csv(OUT_DIR / "regress

print("✓ Finished without singular-matrix errors. Outputs in:", OUT_DI

```

```

/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered i
n exp
    return 1/(1+np.exp(-X))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2443: RuntimeWarning: divide by zero encount
ered in log
    return np.sum(np.log(self.cdf(q * linpred)))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered i
n exp
    return 1/(1+np.exp(-X))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2443: RuntimeWarning: divide by zero encount
ered in log
    return np.sum(np.log(self.cdf(q * linpred)))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered i
n exp
    return 1/(1+np.exp(-X))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2443: RuntimeWarning: divide by zero encount
ered in log
    return np.sum(np.log(self.cdf(q * linpred)))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered i
n exp
    return 1/(1+np.exp(-X))
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
discrete/discrete_model.py:2443: RuntimeWarning: divide by zero encount
ered in log
    return np.sum(np.log(self.cdf(q * linpred)))
✓ Finished without singular-matrix errors. Outputs in: /Users/maxchalek
son/Projects/NICO-Research/NICO_human-gemini/facilitator-identificatio
n/outputs_summary

```

10/01 work - continuing (fixing the plotting of the

figures)

note: file path updated cause working off of MBP

```
In [5]: # =====
# PERSON-LEVEL: single-feature OLS with controls (+ publication-ready)
# =====

from pathlib import Path
import re, math, textwrap
import numpy as np, pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# ----- PATHS (MBP) -----
BASE_DIR = Path("/Users/maxchalekson/Northwestern University/Summer-20
DATA_CSV = BASE_DIR / "facilitator-identification" / "ALL_person_year_
OUT_DIR = BASE_DIR / "facilitator-identification" / "outputs_person_l
OUT_DIR.mkdir(parents=True, exist_ok=True)

# ----- CONFIG -----
CONTROLS = ["p_speaking_duration_sec", "p_turns"] # Evey's
OUTCOMES = ["num_teams", "num_funded_teams"]
KEEP_PREFIXES = ("ann_", "mean_", "num_", "positive_", "meeting_length
EXCLUDE_COLS = set(OUTCOMES) | set(CONTROLS) | {
    "person_id", "participant_id", "meeting_id", "session_id", "year", "con
}
MAX_FEATURES = 40
WRAP = 26 # wrap width (tightens label blocks)
FS_BASE = 12 # base font size
mpl.rcParams.update({"font.size": FS_BASE})

# ----- LOAD -----
df = pd.read_csv(DATA_CSV)

for c in set(OUTCOMES) | set(CONTROLS):
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

# ----- NAME PRETTIFIER -----
def pretty_label(col: str, wrap=WRAP) -> str:
    """
    Make long machine names human-friendly and short.
    Rules:
    - drop leading {ann_, mean_score_, num_}
    - *_sum_score or *_sum -> "(sum)" ; *_count -> "(count)"
    - replace underscores with spaces; compact double spaces
    - lower-case body; keep acronyms; then sentence-case first word
    - wrap into multiple lines of <= wrap chars
    """
    s = col
```

```

# outcome/controls: keep readable forms
if s in OUTCOMES:
    return "\n".join(textwrap.wrap(s.replace("_", " "), wrap))

# strip common prefixes once
s = re.sub(r"^(ann_|mean_score_|num_)", "", s)

# collapse well-known tails to compact tags
tail_tag = None
for pat, tag in [
    (r"_sum_score$", " (sum)"),
    (r"_sum$", " (sum)"),
    (r"_count$", " (count)"),
    (r"_score$", ""), # drop bare _score
    (r"^meeting_length$", "meeting length"),
]:
    if re.search(pat, s):
        s = re.sub(pat, "", s)
        if tag and tag.strip():
            tail_tag = tag
        break

# readability
s = s.replace("_", " ")
s = re.sub(r"\s+", " ", s).strip()

# compact common phrases
s = (s
    .replace("coordination and decision practices", "coordination
    .replace("participation dynamics", "participation dynamics")
    .replace("knowledge sharing", "knowledge sharing")
    .replace("relational climate", "relational climate")
    .replace("idea management", "idea management")
    )

if tail_tag:
    s += tail_tag

# Title case, but keep inside parentheses lower
def smart_title(u):
    if "(" in u:
        head, tail = re.match(r"^(.*?)(\s*(.*\s*))?$", u).groups()
        head = head.title() if head else ""
        return (head + (tail or "")).strip()
    return u.title()

s = smart_title(s)

# wrap
return "\n".join(textwrap.wrap(s, wrap))

```

```

# cache pretty names
PRETTY = {c: pretty_label(c) for c in df.columns}

# ----- FEATURE PICKER -----
cand = []
for c in df.columns:
    if c in EXCLUDE_COLS:
        continue
    if not any(c.startswith(p) for p in KEEP_PREFIXES):
        continue
    if not pd.api.types.is_numeric_dtype(df[c]):
        continue
    if pd.to_numeric(df[c], errors="coerce").std(skipna=True) <= 0:
        continue
    if df[c].nunique(dropna=True) <= 2:
        continue
    cand.append(c)

# fallback if empty
if not cand:
    cand = [c for c in df.columns
             if c not in EXCLUDE_COLS
             and pd.api.types.is_numeric_dtype(df[c])
             and pd.to_numeric(df[c], errors="coerce").std(skipna=True)
             and df[c].nunique(dropna=True) > 2]

# rank by variance
cand = sorted(cand, key=lambda x: pd.to_numeric(df[x], errors="coerce")
features = cand
print(f"[Info] Using {len(features)} features.")

# ----- helpers -----
def iqr_mask(s, k=1.5):
    s = pd.to_numeric(s, errors="coerce")
    if s.notna().sum() == 0: return pd.Series(True, index=s.index)
    q1, q3 = np.nanpercentile(s.dropna(), [25, 75])
    iqr = q3 - q1
    lo, hi = q1 - k*iqr, q3 + k*iqr
    return (s >= lo) & (s <= hi)

def star(p):
    return "***" if p < 0.001 else "**" if p < 0.01 else "*" if p < 0

def run_ols(y, x, controls, data):
    rhs = [c for c in controls if c in data.columns]
    rhs.append(x)
    fml = f"{y} ~ " + " + ".join(rhs)
    try:
        m = smf.ols(fml, data=data).fit(cov_type="HC3")
    except Exception as e:
        print(f"[WARN] {y} ~ {x} failed: {e}")
        return None

```

```

    if x not in m.params.index:
        return None
    return dict(
        outcome=y, feature=x,
        coef=float(m.params[x]),
        se=float(m.bse[x]),
        p=float(m.pvalues[x]),
        ci_low=float(m.conf_int().loc[x,0]),
        ci_high=float(m.conf_int().loc[x,1]),
        r2=float(m.rsquared),
        n=int(m.nobs)
    )

# ----- outlier trimming on controls (diagnostic plots only) -----
present_ctrls = [c for c in CONTROLS if c in df.columns]
mask = pd.Series(True, index=df.index)
for c in present_ctrls: mask &= iqr_mask(df[c])
df_trim = df.loc[mask].copy()
print(f"[Info] IQR trim on controls removed {len(df)-len(df_trim)} of

# boxplots for controls (bigger labels, no overlap)
for c in present_ctrls:
    s = pd.to_numeric(df[c], errors="coerce")
    fig, ax = plt.subplots(figsize=(3.8,3.2), constrained_layout=True)
    ax.boxplot([s.dropna().values], tick_labels=[pretty_label(c, wrap=
    ax.set_title("Outlier check")
    fig.savefig(OUT_DIR / f"outlier_{c}.png", dpi=180)
    plt.close(fig)

# ----- run all regressions (full data; with controls) -----
rows = []
for y in OUTCOMES:
    if y not in df.columns:
        print(f"[SKIP] {y} not in data");
        continue
    for x in features:
        res = run_ols(y, x, CONTROLS, df)
        if res: rows.append(res)

res_df = pd.DataFrame(rows)
if res_df.empty:
    print("[ERROR] No models were fit.")
else:
    res_df.to_csv(OUT_DIR / "single_feature_with_controls_FULL.csv", i
    print(f"[OK] wrote {OUT_DIR / 'single_feature_with_controls_FULL.c

# ----- convenience tables -----
if res_df.empty: raise SystemExit

wide_r2    = res_df.pivot(index="feature", columns="outcome", values="r
wide_coef  = res_df.pivot(index="feature", columns="outcome", values="c
wide_p     = res_df.pivot(index="feature", columns="outcome", values="p

```

```

display_feats = (wide_r2.fillna(0)
                  .assign(maxR=wide_r2.max(axis=1))
                  .sort_values("maxR", ascending=False)
                  .head(16)
                  .index.tolist())

# ----- small helpers for plotting -----
def yticks_pretty(ax, feats):
    labels = [PRETTY.get(f, f) for f in feats]
    ax.set_yticks(range(len(feats)))
    ax.set_yticklabels(labels, ha="right")

def add_right_gutter(ax, feats, text_list, xpad=0.02):
    """Put a neat right-column with text (e.g., coef+stars or p-values)
    ylim = ax.get_ylim()
    xlim = ax.get_xlim()
    xr = xlim[1] + (xlim[1]-xlim[0]) * xpad
    for i, t in enumerate(text_list):
        ax.text(xr, i, t, va="center", ha="left", fontsize=FS_BASE-1)
    # expand axes to make room
    ax.set_xlim(xlim[0], xlim[1] + (xlim[1]-xlim[0]) * (xpad*10))

# ----- FIG 1: R-squared comparison -----
fig, ax = plt.subplots(figsize=(18,10), constrained_layout=True)
X = np.arange(len(display_feats)); w = 0.38
for i, y in enumerate(OUTCOMES):
    vals = [wide_r2.loc[f, y] if (f in wide_r2.index and y in wide_r2.
                                for f in display_feats)]
    bars = ax.bar(X + (i-0.5)*w, vals, width=w, label=y)
    for xi, v in zip(X + (i-0.5)*w, vals):
        if np.isfinite(v): ax.text(xi, v+(0.002 if v>=0 else -0.002),
ax.set_xticks(X)
ax.set_xticklabels([PRETTY.get(f, f) for f in display_feats], rotation
ax.set_ylabel("R-squared")
ax.set_title("R-squared Comparison – Sorted by Maximum R2", loc="left"
ax.legend()
fig.savefig(OUT_DIR / "r_squared_comparison.png", dpi=200)
plt.close(fig)

# ----- FIG 2: R-squared heatmap -----
fig, ax = plt.subplots(figsize=(16,10), constrained_layout=True)
sub = wide_r2.loc[display_feats, OUTCOMES]
im = ax.imshow(sub.values, cmap="viridis", aspect="auto")
ax.set_yticks(range(len(display_feats)))
ax.set_yticklabels([PRETTY.get(f, f) for f in display_feats])
ax.set_xticks(range(len(OUTCOMES)))
ax.set_xticklabels(OUTCOMES)
for i in range(sub.shape[0]):
    for j in range(sub.shape[1]):
        v = sub.iloc[i,j]
        if np.isfinite(v):

```

```

        ax.text(j, i, f"{v:.4f}", ha="center", va="center",
                color="white" if v>0.06 else "black", fontsize=FS_)
ax.set_title("R-squared by Feature and Outcome", loc="left", pad=8)
fig.colorbar(mpl.cm.ScalarMappable(
    norm=mpl.colors.Normalize(vmin=np.nanmin(sub.values), vmax=np.nanmax(sub.values)),
    cmap="viridis"), ax=ax, label="R-squared")
fig.savefig(OUT_DIR / "r_squared_heatmap.png", dpi=200)
plt.close(fig)

```

----- FIG 3: side-by-side coefficient comparison (with significance)

```

def plot_side_by_side(outcomes, feats):
    fig, axs = plt.subplots(1, len(outcomes), figsize=(22,10), constrained_layout=True)
    if len(outcomes)==1: axs = [axs]
    colors = {"***":"tab:green", "**":"tab:orange", "*":"gold", "ns":""}
    for ax, y in zip(axs, outcomes):
        sub = res_df[(res_df["outcome"]==y) & (res_df["feature"].isin(feats))]
        sub["sig"] = sub["p"].apply(star)
        sub = sub.sort_values("coef")
        ax.barh(range(len(sub)), sub["coef"], color=[colors[s] for s in sub["sig"]])
        yticks_pretty(ax, sub["feature"].tolist())
        texts = [f"{c:.3f} {star(p)}" for c,p in zip(sub["coef"], sub["sig"])]
        add_right_gutter(ax, sub["feature"].tolist(), texts, xpad=0.03)
        ax.axvline(0, color="k", lw=1)
        ax.set_title(f"{y}\nCoefficient Comparison with Significance")
        ax.set_xlabel("Coefficient (effect size)")
    fig.suptitle("Side-by-Side Coefficient Comparison by Outcome", fontweight="bold")
    fig.savefig(OUT_DIR / "side_by_side_coefficient_comparison.png", dpi=200)
    plt.close(fig)

```

```

plot_side_by_side(OUTCOMES, display_feats)

```

----- FIG 4: significance matrix -----

```

fig, ax = plt.subplots(figsize=(16,10), constrained_layout=True)
show = wide_p.loc[display_feats, OUTCOMES]
txt = show.applymap(star)
img = ax.imshow((show<0.05).astype(float).values, cmap="Reds", aspect="equal")
ax.set_yticks(range(len(display_feats)))
ax.set_yticklabels([PRETTY.get(f, f) for f in display_feats])
ax.set_xticks(range(len(OUTCOMES)))
ax.set_xticklabels(OUTCOMES)
for i in range(show.shape[0]):
    for j in range(show.shape[1]):
        ax.text(j, i, txt.iloc[i,j], ha="center", va="center", color="white" if txt[i,j]!=" " else "black")
ax.set_title("Significant Results Only - *** p<0.001, ** p<0.01, * p<0.05, . p<0.1, ns = not significant")
fig.savefig(OUT_DIR / "significance_matrix.png", dpi=200)
plt.close(fig)

```

----- FIG 5: coefficient barplots with 95% CI per outcome -----

```

for y in OUTCOMES:
    sub = res_df[(res_df["outcome"]==y) & (res_df["feature"].isin(display_feats))]
    sub = sub.sort_values("coef")
    fig, ax = plt.subplots(figsize=(22,10), constrained_layout=True)

```


The output file path is

- for MBP: '/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz'
- for Studio: '/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz'

```
In [23]: # =====
# PERSON x TEMPORAL THIRDS (first/middle/last) – controls + annotation
# Outputs:
#   - ALL_person_session_segment.csv
#   - ALL_person_year_segment.csv
#   - ALL_person_year_segment_WITH_OUTCOMES.csv (if outcomes file found)
# Plots:
#   - seg_controls_overview.png
#   - seg_ann_counts_heatmap.png
#   - seg_ann_means_heatmap.png
#   - seg_role_facilitator_coefbars.png (only if outcomes available)
# =====

import json, re, textwrap
from pathlib import Path
from collections import defaultdict
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import statsmodels.formula.api as smf

# ----- PATHS -----
DATA_ROOT = Path("/Users/maxchalekson/Desktop/gemini_data_analysis/data")

OUT_MBP = Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO")
OUT_STUDIO = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini")
OUT_DIR = OUT_MBP if OUT_MBP.exists() else OUT_STUDIO
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Outcomes (person-year) to merge if present
OUTCOME_CANDIDATES = [
    Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO"),
    Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini")
]
OUTCOME_PATH = next((p for p in OUTCOME_CANDIDATES if p.exists()), None)

print(f"[INFO] DATA_ROOT = {DATA_ROOT}")
print(f"[INFO] OUT_DIR = {OUT_DIR}")
```

```

print(f"[INFO] Outcomes merge: {OUTCOME_PATH if OUTCOME_PATH else '- ('

# ----- CONFIG -----
WRAP = 28
SEGMENTS = ["first", "middle", "last"]

# Map handwritten/JSON labels → tidy families you've been using
ANN_ALIAS = {
    "relational_climate" : "ann_relational_climate",
    "participation_dynamics" : "ann_participation_dynamics",
    "coordination_and_decision" : "ann_coordination_and_decis",
    "coordination_and_decision_practices" : "ann_coordination_and_decis",
    "evaluation_practices" : "ann_evaluation_practices",
    "knowledge_sharing" : "ann_knowledge_sharing",
    "information_seeking" : "ann_information_seeking",
    "integration_practices" : "ann_integration_practices",
    "idea_management" : "ann_idea_management",
    "none" : "ann_none",
}

# ----- HELPERS -----
def wrap_label(s, w=WRAP):
    return "\n".join(textwrap.wrap(str(s), width=w))

_time_pat = re.compile(r"^(?:(\d+):)?(\d{1,2}):(\d{2})(?:\.\d+)?$")
def time_to_sec(v):
    if v is None: return None
    if isinstance(v, (int, float)): return float(v)
    s = str(v).strip()
    if s.isdigit():
        return float(int(s))
    m = _time_pat.match(s)
    if m:
        h = int(m.group(1) or 0); mm = int(m.group(2)); ss = int(m.group(3))
        frac = m.group(4)
        sec = h*3600 + mm*60 + ss
        if frac: sec += float("0."+frac)
        return float(sec)
    # "MM:SS-MM:SS"
    if "-" in s and s.count(":") >= 2:
        return None
    # "12m34s", "95s", "45m"
    s2 = s.lower()
    try:
        if s2.endswith("s") and "m" in s2:
            parts = s2.replace("s", "").split("m")
            return float(int(parts[0])*60 + int(parts[1]))
        if s2.endswith("s"):
            return float(s2[:-1])
        if s2.endswith("m"):
            return float(int(s2[:-1])*60)
    except Exception:

```

```

        pass
    return None

def parse_range(s):
    if not s or "-" not in str(s): return (None, None)
    a, b = [t.strip() for t in str(s).split("-", 1)]
    return (time_to_sec(a), time_to_sec(b))

def canon_name(s):
    if not isinstance(s, str): return ""
    s = s.strip()
    if not s: return ""
    s = s.replace(", PhD", "").replace(" PhD", "")
    if " - " in s: s = s.split(" - ")[0]
    return s

def which_third(t, T):
    if not (isinstance(t, (int, float)) and isinstance(T, (int, float)))
        return None
    third = T/3.0
    return "first" if t < third else ("middle" if t < 2*third else "la

def session_duration(J):
    # Prefer explicit if present
    for k in ["total_speaking_length", "meeting_duration_sec", "duration
        v = J.get(k)
        if isinstance(v, str):
            v2 = time_to_sec(v)
            if v2 and v2>0: return v2
        if isinstance(v, (int, float)) and v>0:
            return float(v)
    # Fallback: max end over all_data
    mx = 0.0
    ad = J.get("all_data", [])
    if isinstance(ad, list):
        for ev in ad:
            if not isinstance(ev, dict): continue
            e = time_to_sec(ev.get("end_time"))
            if e is None:
                _, e = parse_range(ev.get("timestamp"))
            if e is not None:
                mx = max(mx, float(e))
    return mx if mx>0 else None

def parse_path_info(path: Path):
    conf = path.parent.parent.name
    base = path.stem
    sid = None
    for tok in base.split("_"):
        if tok.upper().startswith("S") and tok[1:].isdigit():
            sid = tok.upper(); break
    year = None

```

```

for tok in base.split("_"):
    if len(tok)==4 and tok.isdigit():
        year = int(tok); break
return conf, year, sid or base

# Extract speaking segments (speaker + time + duration) from all_data
def iter_speaking_segments(J):
    segs = []
    ad = J.get("all_data", [])
    if not isinstance(ad, list):
        return segs
    for ev in ad:
        if not isinstance(ev, dict):
            continue
        who = canon_name(ev.get("speaker") or ev.get("speaker_name") or
        if not who:
            continue
        s = time_to_sec(ev.get("start_time"))
        e = time_to_sec(ev.get("end_time"))
        if (s is None or e is None) and isinstance(ev.get("timestamp"),
        s2, e2 = parse_range(ev.get("timestamp"))
        s = s if s is not None else s2
        e = e if e is not None else e2
        d = ev.get("speaking_duration")
        d = float(d) if isinstance(d, (int, float)) else None
        # resolve 2-of-3
        if d is None and s is not None and e is not None:
            d = max(0.0, float(e)-float(s))
        if s is None and e is not None and d is not None:
            s = float(e)-float(d)
        if e is None and s is not None and d is not None:
            e = float(s)+float(d)
        if d is None:
            continue
        # midpoint
        if s is not None and e is not None:
            mid = 0.5*(float(s)+float(e))
        elif s is not None:
            mid = float(s)+float(d)/2.0
        elif e is not None:
            mid = float(e)-float(d)/2.0
        else:
            mid = float(d)/2.0
        segs.append((who, float(d), float(mid)))
    return segs

# Extract annotation items attached to all_data rows: {"annotations":
def iter_annotations(J):
    """
    Yields (who, family, score, mid_sec)
    Accepts "annotations": { "Relational Climate": {"score":2, ...}, .
    Family keys are normalized via ANN_ALIAS.

```

```

"""
out = []
ad = J.get("all_data", [])
if not isinstance(ad, list):
    return out
for ev in ad:
    if not isinstance(ev, dict):
        continue
    anns = ev.get("annotations")
    if not isinstance(anns, dict) or not anns:
        continue
    who = canon_name(ev.get("speaker") or ev.get("speaker_name")) or
    # time midpoints (same logic as speaking segs)
    s = time_to_sec(ev.get("start_time"))
    e = time_to_sec(ev.get("end_time"))
    if (s is None or e is None) and isinstance(ev.get("timestamp")):
        s2, e2 = parse_range(ev.get("timestamp"))
        s = s if s is not None else s2
        e = e if e is not None else e2
    d = ev.get("speaking_duration")
    d = float(d) if isinstance(d, (int, float)) else None
    if s is not None and e is not None:
        mid = 0.5*(float(s)+float(e))
    elif s is not None and d is not None:
        mid = float(s)+float(d)/2.0
    elif e is not None and d is not None:
        mid = float(e)-float(d)/2.0
    else:
        mid = None

    # iterate categories
    for raw_cat, payload in anns.items():
        cat = str(raw_cat).strip().lower().replace(" ", "_")
        fam = ANN_ALIAS.get(cat)
        if fam is None:
            continue
        score = None
        if isinstance(payload, dict):
            sc = payload.get("score")
            if isinstance(sc, (int, float)):
                score = float(sc)
        out.append((who, fam, score, mid))
return out

# ----- SCAN -----
rows = []
files_seen = 0
skipped_no_duration = 0
skipped_no_segments = 0

conf_dirs = [p for p in DATA_ROOT.glob("*") if (p.is_dir() and (p/"ses
print("[INFO] conferences:", [d.name for d in conf_dirs])

```

```

for conf_dir in conf_dirs:
    for js in (conf_dir/"session_data").glob("*.json"):
        files_seen += 1
        try:
            J = json.loads(js.read_text())
        except Exception as e:
            print(f"[WARN] read error {js.name}: {e}")
            continue

        T = session_duration(J)
        if not T or T <= 0:
            skipped_no_duration += 1
            if skipped_no_duration <= 5:
                print(f"[SKIP:no-duration] {js.name} keys={list(J.keys)}")
            continue

        conference, year, session_id = parse_path_info(js)

        # Controls (speaking) per person×third
        speak_segs = iter_speaking_segments(J)
        if not speak_segs:
            skipped_no_segments += 1
            if skipped_no_segments <= 8:
                print(f"[SKIP:no-speaking-segments] {js.name}")
            # we still might have annotations with timestamps—so don't

        controls = defaultdict(lambda: {"p_speaking_duration_sec": 0.0,
        for who, dur, mid in speak_segs:
            seg = which_third(mid, T) if mid is not None else "middle"
            if not seg: seg = "middle"
            controls[(who, seg)]["p_speaking_duration_sec"] += float(dur)
            controls[(who, seg)]["p_turns"] += 1

        # Annotation families per person×third
        anns = iter_annotations(J)
        ann_aggs = defaultdict(lambda: defaultdict(list)) # {(who, seg)
        for who, fam, score, mid in anns:
            if not who or fam is None:
                continue
            seg = which_third(mid, T) if mid is not None else "middle"
            if not seg: seg = "middle"
            ann_aggs[(who, seg)][fam].append(score)

        # union of people×segment appearing in either controls or anns
        keys = set(controls.keys()) | set(k for k in ann_aggs.keys())
        if not keys:
            continue

        for (who, seg) in keys:
            row = {
                "person_name": who,

```

```

        "conference": conference,
        "year": year,
        "session_id": session_id,
        "segment": seg,
        "session_duration_sec": float(T),
        "p_speaking_duration_sec": float(controls[(who,seg)]["p_speaking_duration_sec"]),
        "p_turns": int(controls[(who,seg)]["p_turns"]),
    }
    # explode annotation families
    fams = ann_aggs.get((who,seg), {})
    for fam, scores in fams.items():
        clean = [s for s in scores if s is not None]
        row[f"{fam}_count"] = int(len(scores))
        row[f"{fam}_sum_score"] = float(np.nansum(clean)) if c
        row[f"{fam}_mean_score"] = (float(np.nanmean(clean)) if c
    rows.append(row)

print(f"[INFO] scanned={files_seen} | skipped no-duration={skipped_no_

# ----- WRITE SESSION-SEGMENT -----
pss = pd.DataFrame(rows)
if pss.empty:
    raise RuntimeError("No personxsessionxthird rows built. Check that

pss["year"] = pd.to_numeric(pss["year"], errors="coerce").astype("Int6

# fill numeric family cols
for c in pss.columns:
    if c.endswith("_count"):
        pss[c] = pd.to_numeric(pss[c], errors="coerce").fillna(0).asty
    elif c.endswith("_sum_score"):
        pss[c] = pd.to_numeric(pss[c], errors="coerce").fillna(0.0).as
    elif c.endswith("_mean_score"):
        pss[c] = pd.to_numeric(pss[c], errors="coerce")

pss_path = OUT_DIR / "ALL_person_session_segment.csv"
pss.to_csv(pss_path, index=False)
print(f"[OK] wrote {pss_path} rows={len(pss)}")

# ----- WRITE YEAR-SEGMENT (aggregate sessions) -----
sum_cols = [c for c in pss.columns if c.endswith("_count") or c.endsw
mean_cols = [c for c in pss.columns if c.endswith("_mean_score")]

def agg_mean_ignore_na(x):
    x = pd.to_numeric(x, errors="coerce")
    return float(np.nanmean(x)) if np.isfinite(x).any() else np.nan

agg_dict = {**{c:"sum" for c in sum_cols}, **{c:agg_mean_ignore_na for

pys = (pss.groupby(["person_name","conference","year","segment"], drop
    .agg(agg_dict))

```

```

pys_path = OUT_DIR / "ALL_person_year_segment.csv"
pys.to_csv(pys_path, index=False)
print(f"[OK] wrote {pys_path} rows={len(pys)}")

# ----- OPTIONAL MERGE OUTCOMES -----
pys_out = pys.copy()
if OUTCOME_PATH and OUTCOME_PATH.exists():
    base = pd.read_csv(OUTCOME_PATH)
    keep = [c for c in ["role_facilitator", "num_teams", "num_funded_tea
    if {"person_name", "year"}.issubset(base.columns) and keep:
        merged = base[["person_name", "year"] + keep].drop_duplicates(["p
        pys_out = pys_out.merge(merged, on=["person_name", "year"], how
        print(f"[OK] merged outcomes: {keep}")
    else:
        print("[WARN] outcomes file found but keys/cols missing; skippe
else:
    print("[INFO] no outcomes file; skipping merge.")

pys_out_path = OUT_DIR / "ALL_person_year_segment_WITH_OUTCOMES.csv"
pys_out.to_csv(pys_out_path, index=False)
print(f"[OK] wrote {pys_out_path} rows={len(pys_out)}")

# ----- QUICK VISUALS -----
def maybe_cols(df, cols):
    return [c for c in cols if c in df.columns]

# 1) Controls overview by segment
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ctrl = pys_out.groupby("segment")[["p_speaking_duration_sec", "p_turns"]
ctrl["p_speaking_duration_hours"] = ctrl["p_speaking_duration_sec"] / 36
ax[0].bar(ctrl.index, ctrl["p_speaking_duration_hours"])
ax[0].set_title("Total speaking duration by segment"); ax[0].set_ylabel("
ax[1].bar(ctrl.index, ctrl["p_turns"])
ax[1].set_title("Total speaking turns by segment"); ax[1].set_ylabel("
fig.tight_layout()
fig.savefig(OUT_DIR / "seg_controls_overview.png", dpi=170)
plt.close(fig)

# 2) Heatmaps of annotation counts/means by segment (top N families)
families = sorted({c.rsplit("_", 2)[0] for c in pys_out.columns if c.en
topN = min(12, len(families))
fam_for_plot = families[:topN]

def heat(data, value_col, fname, title):
    if not fam_for_plot:
        return
    mat = []
    for fam in fam_for_plot:
        col = f"{fam}_{value_col}"
        if col not in data.columns:
            mat.append([np.nan] * len(SEGMENTS))
        else:

```

```

        s = data.groupby("segment")[col].mean().reindex(SEGMENTS)
        mat.append(list(s.values))
M = np.array(mat, dtype=float)
fig, ax = plt.subplots(figsize=(12, 0.45*len(fam_for_plot)+2))
im = ax.imshow(M, aspect="auto", cmap="viridis")
ax.set_yticks(range(len(fam_for_plot)))
ax.set_yticklabels([wrap_label(f, 36) for f in fam_for_plot])
ax.set_xticks(range(len(SEGMENTS))); ax.set_xticklabels(SEGMENTS)
for i in range(M.shape[0]):
    for j in range(M.shape[1]):
        if np.isfinite(M[i,j]):
            ax.text(j, i, f"{M[i,j]:.2f}", ha="center", va="center")
ax.set_title(title)
fig.colorbar(mpl.cm.ScalarMappable(norm=mpl.colors.Normalize(vmin=
    ax=ax, label=value_col))
fig.tight_layout()
fig.savefig(OUT_DIR / fname, dpi=170)
plt.close(fig)

```

```

heat(pys_out, "count", "seg_ann_counts_heatmap.png", "Annotation count
heat(pys_out, "mean_score", "seg_ann_means_heatmap.png", "Annotation m

```

```

# 3) If role_facilitator exists: simple logit with segment interaction

```

```

if "role_facilitator" in pys_out.columns:
    rows_l = []
    for fam in families:
        fam_col = f"{fam}_count"
        if fam_col not in pys_out.columns:
            continue
        dfm = pys_out.dropna(subset=["role_facilitator"]).copy()
        dfm["role_facilitator"] = pd.to_numeric(dfm["role_facilitator"])
        formula = f"role_facilitator ~ {fam_col} * C(segment)"
        if "conference" in dfm.columns:
            formula += " + C(conference)"
        if "year" in dfm.columns:
            formula += " + C(year)"
        try:
            m = smf.logit(formula, data=dfm).fit(dis=0)
        except Exception:
            continue
        # gather the family term & its segment interactions
        coefs = m.params
        ses = m.bse
        take = [idx for idx in coefs.index if idx.startswith(fam_col)]
        for idx in take:
            rows_l.append({
                "family": fam,
                "term": idx.replace(fam_col, fam),
                "coef": float(coefs[idx]),
                "se": float(ses[idx]),
                "p": float(m.pvalues[idx])
            })

```

```

if rows_l:
    coe_df = pd.DataFrame(rows_l)
    # keep the most informative terms per family (main + interaction)
    coe_df["abscoef"] = coe_df["coef"].abs()
    keep = coe_df.sort_values(["family", "abscoef"], ascending=[True, False])
    keep = keep.sort_values("coef")
    fig, ax = plt.subplots(figsize=(12, max(3, 0.5*keep.shape[0])))
    ylabels = [wrap_label(f"{r['family']} • {r['term']}", 36) for r in keep.itertuples()]
    ax.barh(ylabels, keep["coef"], xerr=1.96*keep["se"], alpha=0.9)
    ax.axvline(0, color="k", lw=1)
    for yy, (c, p) in enumerate(zip(keep["coef"], keep["p"])):
        ax.text(c + (0.02 if c >= 0 else -0.02), yy, f"{c:.3f} ({'**' if p < 0.01 else ''})",
                va="center", ha="left" if c >= 0 else "right", fontstyle="italic")
    ax.set_title("Facilitation logit – family counts × segment (coefficient)")
    ax.set_xlabel("Log-odds")
    fig.tight_layout()
    fig.savefig(OUT_DIR / "seg_role_facilitator_coefbars.png", dpi=300)
    plt.close(fig)

print("\n[DONE] Temporal thirds (person-level) complete.")
print("CSV outputs:")
print("  -", OUT_DIR / "ALL_person_session_segment.csv")
print("  -", OUT_DIR / "ALL_person_year_segment.csv")
print("  -", OUT_DIR / "ALL_person_year_segment_WITH_OUTCOMES.csv")
print("Plots:")
print("  -", OUT_DIR / "seg_controls_overview.png")
print("  -", OUT_DIR / "seg_ann_counts_heatmap.png")
print("  -", OUT_DIR / "seg_ann_means_heatmap.png")
print("  -", OUT_DIR / "seg_role_facilitator_coefbars.png", "(if outcomes available)")

```

```

[INFO] DATA_ROOT = /Users/maxchalekson/Desktop/gemini_data_analysis/data
[INFO] OUT_DIR = /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz
[INFO] Outcomes merge: – (not found)
[INFO] conferences: ['2021ABI', '2021SLU', '2022MND', '2020NES', '2021CMC', '2021MND', '2021MZT', '2021NES']
[INFO] scanned=157 | skipped no-duration=0 | skipped no-speaking-seg=0
[OK] wrote /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz/ALL_person_session_segment.csv rows=4450
[OK] wrote /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz/ALL_person_year_segment.csv rows=1788
[INFO] no outcomes file; skipping merge.
[OK] wrote /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal-facilitator-outputs-viz/ALL_person_year_segment_WITH_OUTCOMES.csv rows=1788

```

```

/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_12432/411900
9273.py:417: RuntimeWarning: All-NaN slice encountered
  fig.colorbar(mpl.cm.ScalarMappable(norm=mpl.colors.Normalize(vmin=np.
nanmin(M), vmax=np.nanmax(M)), cmap="viridis"),
/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_12432/411900
9273.py:417: RuntimeWarning: All-NaN slice encountered
  fig.colorbar(mpl.cm.ScalarMappable(norm=mpl.colors.Normalize(vmin=np.
nanmin(M), vmax=np.nanmax(M)), cmap="viridis"),
[DONE] Temporal thirds (person-level) complete.
CSV outputs:
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/ALL_person_session_segmen
t.csv
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/ALL_person_year_segmen
t.csv
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/ALL_person_year_segment
_WITH_OUTCOMES.csv
Plots:
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/seg_controls_overview.p
ng
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/seg_ann_counts_heatmap.
png
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/seg_ann_means_heatmap.p
ng
  - /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO R
research/NICO_human-gemini/facilitator-identification/temporal-facilitat
or-significant/temporal-facilitator-outputs-viz/seg_role_facilitator_co
efbars.png (if outcomes present)

```

confirming csv layouts (sanity check)

```

In [24]: import pandas as pd
         from pathlib import Path

         base = Path("/Users/maxchalekson/Northwestern University/Summer-2025/N

         pss = pd.read_csv(base / "ALL_person_session_segment.csv")
         pys = pd.read_csv(base / "ALL_person_year_segment.csv")
         pysw = pd.read_csv(base / "ALL_person_year_segment_WITH_OUTCOMES.csv")

```

```
def peek(df, name):
    ann_cols = sorted([c for c in df.columns if c.startswith("ann_")])
    print(f"\n=== {name} ===")
    print("rows:", len(df))
    print("has segment?:", "segment" in df.columns, "unique:", df["seg
    print("has outcomes?:", [c for c in ["role_facilitator", "num_teams
    print("sample ann cols:", ann_cols[:8])
    print("\n ann cols:", len(ann_cols))

peek(pss, "session-level (person×session×segment)")
peek(pys, "year-level (person×year×segment)")
peek(pysw, "year-level WITH outcomes")
```

```
=== session-level (person×session×segment) ===
rows: 4450
has segment?: True unique: ['middle' 'last' 'first']
has outcomes?: []
sample ann cols: ['ann_coordination_and_decision_practices_count', 'ann
_coordination_and_decision_practices_mean_score', 'ann_coordination_and
_decision_practices_sum_score', 'ann_evaluation_practices_count', 'ann_
evaluation_practices_mean_score', 'ann_evaluation_practices_sum_score',
'ann_idea_management_count', 'ann_idea_management_mean_score']
n ann cols: 27
```

```
=== year-level (person×year×segment) ===
rows: 1788
has segment?: True unique: ['first' 'last' 'middle']
has outcomes?: []
sample ann cols: ['ann_coordination_and_decision_practices_count', 'ann
_coordination_and_decision_practices_mean_score', 'ann_coordination_and
_decision_practices_sum_score', 'ann_evaluation_practices_count', 'ann_
evaluation_practices_mean_score', 'ann_evaluation_practices_sum_score',
'ann_idea_management_count', 'ann_idea_management_mean_score']
n ann cols: 27
```

```
=== year-level WITH outcomes ===
rows: 1788
has segment?: True unique: ['first' 'last' 'middle']
has outcomes?: []
sample ann cols: ['ann_coordination_and_decision_practices_count', 'ann
_coordination_and_decision_practices_mean_score', 'ann_coordination_and
_decision_practices_sum_score', 'ann_evaluation_practices_count', 'ann_
evaluation_practices_mean_score', 'ann_evaluation_practices_sum_score',
'ann_idea_management_count', 'ann_idea_management_mean_score']
n ann cols: 27
```

temporal analysis -- running the graphs

```
In [8]: # =====
# TEMPORAL (person-year-segment) – single-feature OLS w/ controls
# – Finds temporal CSVs in the *parent* folder you specified or the ol
```

```

# - Robustly merges outcomes
# - Produces results CSV + plots ( $R^2$  heatmaps, coef±CI bars, sig matri
# =====

from pathlib import Path
import numpy as np, pandas as pd, textwrap
import matplotlib.pyplot as plt
import matplotlib as mpl
import statsmodels.formula.api as smf

# ----- PATHS -----
ROOTS = [
    Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini
    Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO
]

# New parent folder (where you moved the CSVs)
TEMP_PARENTS = [
    Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini
    Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO
]

# Old viz folder (still searched just in case)
TEMP_VIZES = [
    Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini
    Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO
]

OUTCOME_REL = "facilitator-identification/facilitator-identification-s

def find_first(*candidates):
    for p in candidates:
        if p and p.exists():
            return p
    return None

def find_file(name, roots):
    for r in roots:
        if r and r.exists():
            hits = list(r.rglob(name))
            if hits:
                # Prefer a hit inside "temporal-facilitator-significan
                hits.sort(key=lambda x: 0 if "temporal-facilitator-sig
                return hits[0]
    return None

# Look in the new parent first, then old viz folder
SEARCH_ROOTS = [*TEMP_PARENTS, *TEMP_VIZES]

PATH_WITH = find_file("ALL_person_year_segment_WITH_OUTCOMES.csv", SEAR
PATH_YEAR = find_file("ALL_person_year_segment.csv", SEAR
PATH_SESS = find_file("ALL_person_session_segment.csv", SEAR

```

```

OUTCOME_PATH = None
for base in ROOTS:
    cand = base / OUTCOME_REL
    if cand.exists():
        OUTCOME_PATH = cand
        break

OUT_DIR = find_first(*TEMP_PARENTS) or find_first(*TEMP_VIZES) or ROOT
OUT_DIR.mkdir(parents=True, exist_ok=True)

print("[INFO] OUT_DIR:", OUT_DIR)
print("[INFO] located:")
print(" - YEAR_SEGMENT_WITH_OUTCOMES:", PATH_WITH)
print(" - YEAR_SEGMENT:", PATH_YEAR)
print(" - SESSION_SEG :", PATH_SESS)
print(" - OUTCOMES      :", OUTCOME_PATH)

# ----- LOAD / MERGE -----
def load_temporal_base():
    if PATH_WITH is not None:
        return pd.read_csv(PATH_WITH)
    if PATH_YEAR is not None:
        return pd.read_csv(PATH_YEAR)
    if PATH_SESS is not None:
        pss = pd.read_csv(PATH_SESS)
        # aggregate to person-year-segment (keep speaking controls)
        return (pss.groupby(["person_name", "conference", "year", "segment"]
                             .sum(numeric_only=True))
                .sum(numeric_only=True))
    raise FileNotFoundError("Could not find any temporal CSVs (WITH, Y

merged = load_temporal_base()

# If outcomes missing/empty, try to merge them
needs_merge = not set(["num_teams", "num_funded_teams"]).intersection(merged[["num_teams", "num_funded_teams"]].isna().all().all())
set(["num_teams", "num_funded_teams"]).issubset(merged.columns)

if needs_merge and OUTCOME_PATH is not None:
    out = pd.read_csv(OUTCOME_PATH)
    if {"person_name", "year"}.issubset(out.columns):
        keep_out = [c for c in ["person_name", "year", "num_teams", "num_funded_teams", "role_facilitator", "sessions_total"] if c in out.columns]
        out_small = out[keep_out].drop_duplicates(subset=["person_name", "year"])
        merged = merged.merge(out_small, on=["person_name", "year"], how="left")
        print("[INFO] merged outcomes into temporal table.")
    else:
        print("[WARN] outcomes file missing person_name/year; proceeding")

# hygiene
for c in ["num_teams", "num_funded_teams", "p_turns", "p_speaking_duration"]:
    if c in merged.columns:

```

```

merged[c] = pd.to_numeric(merged[c], errors="coerce")

if "segment" in merged.columns:
    merged["segment"] = merged["segment"].astype(str).str.lower().str.strip()
    merged = merged[merged["segment"].isin(["first", "middle", "last"])]

if "person_name" in merged.columns:
    merged["person_name"] = merged["person_name"].astype(str).str.strip()

print("[INFO] temporal rows:", len(merged))

# ----- CONFIG -----
CONTROLS = [c for c in ["p_speaking_duration_sec", "p_turns"] if c in merged.columns]
OUTCOMES = [c for c in ["num_teams", "num_funded_teams"] if c in merged.columns]
if not OUTCOMES:
    cols = list(merged.columns)
    raise RuntimeError(f"No numeric outcomes after merge. Columns present: {cols}")

SEGMENTS = ["first", "middle", "last"]
WRAP = 26
MIN_NON_NULL = 10

# ----- FEATURES -----
ann_counts = sorted([c for c in merged.columns if c.startswith("ann_")])
ann_means = sorted([c for c in merged.columns if c.startswith("ann_")])

def fam_of(col):
    if col.endswith("_count"): return col[:-len("_count")]
    if col.endswith("_mean_score"): return col[:-len("_mean_score")]
    return col

families = sorted(set(map(fam_of, ann_counts + ann_means)))

def has_variance(s, min_n=MIN_NON_NULL):
    s = pd.to_numeric(s, errors="coerce")
    return (s.notna().sum() >= min_n) and (s.std(skipna=True) > 0)

def choose_feature_for_family(df_seg, fam, min_n=MIN_NON_NULL):
    cnt = f"{fam}_count"
    mean = f"{fam}_mean_score"
    # Prefer counts; if too sparse, fall back to mean_score
    if cnt in df_seg.columns and has_variance(df_seg[cnt], min_n): return cnt
    if mean in df_seg.columns and has_variance(df_seg[mean], min_n): return mean
    return None

# ----- MODEL -----
def run_ols(y, x, data):
    rhs = [x] + [c for c in CONTROLS if c in data.columns]
    if "conference" in data.columns: rhs.append("C(conference)")
    if "year" in data.columns: rhs.append("C(year)")
    formula = f"{y} ~ " + " + ".join(rhs)
    try:

```

```

        m = smf.ols(formula, data=data).fit(cov_type="HC3")
    except Exception:
        return None
    if x not in m.params.index:
        return None
    ci = m.conf_int().loc[x]
    return dict(
        outcome=y, feature=x,
        coef=float(m.params[x]), se=float(m.bse[x]), p=float(m.pvalues[x]),
        ci_low=float(ci.iloc[0]), ci_high=float(ci.iloc[1]),
        r2=float(m.rsquared), n=int(m.nobs)
    )

# ----- RUN -----
all_rows = []
for seg in SEGMENTS:
    dseg = merged[merged["segment"] == seg].copy()
    if dseg.empty:
        print(f"[SKIP] {seg}: no rows."); continue

    chosen = []
    for fam in families:
        xcol = choose_feature_for_family(dseg, fam, min_n=MIN_NON_NULL)
        if xcol:
            chosen.append(xcol)
    print(f"[INFO] {seg}: eligible predictors = {len(chosen)} / {len(families)}")

    for y in OUTCOMES:
        dy = dseg.dropna(subset=[y]).copy()
        if dy[y].nunique(dropna=True) <= 1:
            print(f"[SKIP] {seg} - {y}: no variance in outcome."); continue
        elig = [x for x in chosen if has_variance(dy[x], min_n=MIN_NON_NULL)]
        print(f"    {y}: usable predictors = {len(elig)}")
        for x in elig:
            res = run_ols(y, x, dy)
            if res:
                res["segment"] = seg
                all_rows.append(res)

res = pd.DataFrame(all_rows)
if res.empty:
    raise RuntimeError("No models fit. Lower MIN_NON_NULL to 5, or increase n")

out_csv = OUT_DIR / "temporal_single_feature_with_controls.csv"
res.to_csv(out_csv, index=False)
print(f"[OK] wrote {out_csv} (rows={len(res)})")

# ----- PLOTS -----
def wrap_lab(s, width=WRAP):
    return "\n".join(textwrap.wrap(str(s), width=width))

def top_features_for_segment(res_df, seg, top_k=18):

```

```

sub = res_df[res_df["segment"]==seg]
if sub.empty: return []
r2w = sub.pivot(index="feature", columns="outcome", values="r2").f
r2w["maxR"] = r2w.max(axis=1)
return r2w.sort_values("maxR", ascending=False).head(top_k).index.

# 1) R2 heatmaps per segment
for seg in SEGMENTS:
    sub = res[res["segment"]==seg]
    if sub.empty: continue
    r2w = sub.pivot(index="feature", columns="outcome", values="r2").f
    feats = top_features_for_segment(res, seg, top_k=18)
    r2w = r2w.loc[[f for f in feats if f in r2w.index]]
    if r2w.empty: continue
    fig, ax = plt.subplots(figsize=(12, max(6, 0.4*len(r2w))))
    im = ax.imshow(r2w.values, cmap="viridis", aspect="auto")
    ax.set_yticks(range(len(r2w.index)))
    ax.set_yticklabels([wrap_lab(f, 28) for f in r2w.index])
    ax.set_xticks(range(len(r2w.columns)))
    ax.set_xticklabels(r2w.columns)
    for i in range(r2w.shape[0]):
        for j in range(r2w.shape[1]):
            val = r2w.values[i,j]
            ax.text(j, i, f"{val:.3f}", ha="center", va="center",
                    color="white" if val>0.04 else "black", fontsize=8)
    ax.set_title(f"R2 by Feature × Outcome – {seg.title()}")
    fig.colorbar(mpl.cm.ScalarMappable(norm=mpl.colors.Normalize(vmin=
                                                                    vmax=
                                                                    cmap="viridis"),
                                ax=ax, label="R2"))
    fig.tight_layout()
    fig.savefig(OUT_DIR / f"r2_heatmap_{seg}.png", dpi=170)
    plt.close(fig)

# 2) Coefficient barplots with 95% CI per outcome × segment
for seg in SEGMENTS:
    for y in OUTCOMES:
        sub = res[(res["segment"]==seg) & (res["outcome"]==y)].copy()
        if sub.empty: continue
        feats = top_features_for_segment(res, seg, top_k=18)
        sub = sub[sub["feature"].isin(feats)].sort_values("coef")
        fig, ax = plt.subplots(figsize=(15, max(6, 0.35*len(sub))))
        ylabels = [wrap_lab(f, 30) for f in sub["feature"]]
        ax.barh(ylabels, sub["coef"], xerr=1.96*sub["se"], alpha=0.9)
        ax.axvline(0, color="k", lw=1)
        ax.set_xlabel("Coefficient (±95% CI)")
        ax.set_title(f"Coef ±95% CI – {y} – {seg.title()}")
        fig.tight_layout()
        fig.savefig(OUT_DIR / f"coef_bar_{y}_{seg}.png", dpi=170)
        plt.close(fig)

# 3) Significance matrix (***, **, *, ns) per segment

```

```

def p_to_star(p):
    return ("***" if p < 0.001 else "**" if p < 0.01 else "*" if p < 0.05 else "")

for seg in SEGMENTS:
    sub = res[res["segment"]==seg].copy()
    if sub.empty: continue
    feats = top_features_for_segment(res, seg, top_k=18)
    show = sub[sub["feature"].isin(feats)]
    if show.empty: continue
    pw = show.pivot(index="feature", columns="outcome", values="p")
    txt = pw.applymap(p_to_star)
    fig, ax = plt.subplots(figsize=(10, max(5, 0.35*len(txt))))
    img = ax.imshow((pw < 0.05).astype(float).values, cmap="Reds", aspect="auto")
    ax.set_yticks(range(len(txt.index))); ax.set_yticklabels([wrap_label(i) for i in range(len(txt.index))])
    ax.set_xticks(range(len(txt.columns))); ax.set_xticklabels(txt.columns)
    for i in range(txt.shape[0]):
        for j in range(txt.shape[1]):
            ax.text(j, i, txt.values[i,j], ha="center", va="center", color="white")
    ax.set_title(f"Significance ( $\alpha=0.05$ ) - {seg.title()}")
    fig.tight_layout()
    fig.savefig(OUT_DIR / f"significance_matrix_{seg}.png", dpi=170)
    plt.close(fig)

print("[DONE] Wrote:")
print(f" - {out_csv.name}")
print(" - r2_heatmap_first/middle/last.png")
print(" - coef_bar_<outcome>_<segment>.png")
print(" - significance_matrix_<segment>.png")

```

```

[INFO] OUT_DIR: /Users/maxchalekson/Projects/NICO-Research/NICO_human-g
emini/facilitator-identification/temporal-facilitator-significant
[INFO] located:
  - YEAR_SEGMENT_WITH_OUTCOMES: /Users/maxchalekson/Projects/NICO-Resea
rch/NICO_human-gemini/facilitator-identification/temporal-facilitator-s
ignificant/ALL_person_year_segment_WITH_OUTCOMES.csv
  - YEAR_SEGMENT: /Users/maxchalekson/Projects/NICO-Research/NICO_human
-gemini/facilitator-identification/temporal-facilitator-significant/ALL
_person_year_segment.csv
  - SESSION_SEG : /Users/maxchalekson/Projects/NICO-Research/NICO_human
-gemini/facilitator-identification/temporal-facilitator-significant/ALL
_person_session_segment.csv
  - OUTCOMES      : /Users/maxchalekson/Projects/NICO-Research/NICO_human
-gemini/facilitator-identification/facilitator-identification-significa
nt/ALL_person_year_WITH_OUTCOMES.csv
[INFO] merged outcomes into temporal table.
[INFO] temporal rows: 1788
[INFO] first: eligible predictors = 9 / 9
      num_teams: usable predictors = 9
      num_funded_teams: usable predictors = 9
[INFO] middle: eligible predictors = 9 / 9
      num_teams: usable predictors = 9
      num_funded_teams: usable predictors = 9
[INFO] last: eligible predictors = 9 / 9
      num_teams: usable predictors = 9
      num_funded_teams: usable predictors = 9
[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemin
i/facilitator-identification/temporal-facilitator-significant/temporal_
single_feature_with_controls.csv (rows=54)
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_18475/135386
5037.py:274: FutureWarning: DataFrame.applymap has been deprecated. Use
DataFrame.map instead.
      txt = pw.applymap(p_to_star)
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_18475/135386
5037.py:274: FutureWarning: DataFrame.applymap has been deprecated. Use
DataFrame.map instead.
      txt = pw.applymap(p_to_star)
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_18475/135386
5037.py:274: FutureWarning: DataFrame.applymap has been deprecated. Use
DataFrame.map instead.
      txt = pw.applymap(p_to_star)
[DONE] Wrote:
  - temporal_single_feature_with_controls.csv
  - r2_heatmap_first/middle/last.png
  - coef_bar_<outcome>_<segment>.png
  - significance_matrix_<segment>.png

```

**quick confirming differences b/w roles
(temporally)**

- facilitators or not
- in-teams or funded teams?
 - (remember there's three ways to parse through this)

```
In [1]: # =====
# TEMPORAL ROLE/TEAM DIFFERENCES (answers Evey's question)
# - Compare facilitators vs non-facilitators within each segment
# - Compare people with any funded team vs none within each segment
# - Uses annotation *_count features (same families as before)
# - Controls: speaking duration, turns, conf/year FE in regressions
# - Outputs CSVs + barplots highlighting biggest effects
# =====

from pathlib import Path
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# ----- PATHS -----
ROOTS = [
    Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini"),
    Path("/Users/maxchalekson/Northwestern University/Summer-2025/NICO")
]

TEMP_REL = "facilitator-identification/temporal-facilitator-significan
OUTCOME_REL = "facilitator-identification/facilitator-identification-s

BASE = next((b for b in ROOTS if b.exists()), ROOTS[0])
TEMP_DIR = BASE / TEMP_REL
OUT_DIR = TEMP_DIR # write outputs here

# Prefer WITH_OUTCOMES; fall back to YEAR_SEGMENT + merge
CANDS = [
    TEMP_DIR / "ALL_person_year_segment_WITH_OUTCOMES.csv",
    TEMP_DIR / "ALL_person_year_segment.csv",
]

TEMP_PATH = next((p for p in CANDS if p.exists()), None)
OUTCOME_PATH = next((BASE / OUTCOME_REL for _ in [0] if (BASE / OUTCOM

if TEMP_PATH is None:
    raise FileNotFoundError("Could not find temporal CSVs in\n" + "\n"

df = pd.read_csv(TEMP_PATH)

# If outcomes not present, merge them
needed_outs = {"num_teams", "num_funded_teams"}
if not needed_outs.intersection(df.columns) and OUTCOME_PATH is not No
    outs = pd.read_csv(OUTCOME_PATH)
    keep = [c for c in ["person_name", "year", "role_facilitator", "num_t
    outs_small = outs[keep].drop_duplicates(subset=["person_name", "yea
```

```

df = df.merge(outs_small, on=["person_name","year"], how="left")

# ----- HYGIENE -----
for c in ["num_teams","num_funded_teams","p_speaking_duration_sec","p_
    if c in df.columns: df[c] = pd.to_numeric(df[c], errors="coerce")
if "segment" in df.columns:
    df["segment"] = df["segment"].astype(str).str.lower().str.strip()

# Flags
df["is_fac"] = (df.get("role_facilitator", 0).fillna(0) > 0).asty
df["any_team"] = (df.get("num_teams", 0).fillna(0) > 0).astype(int)
df["any_funded"] = (df.get("num_funded_teams", 0).fillna(0) > 0).asty

# Annotation feature set (counts)
ann_counts = sorted([c for c in df.columns if c.startswith("ann_") and

# Small helper: Cohen's d for two groups
def cohens_d(a, b):
    a = pd.to_numeric(a, errors="coerce").dropna()
    b = pd.to_numeric(b, errors="coerce").dropna()
    if len(a) < 3 or len(b) < 3: return np.nan
    na, nb = len(a), len(b)
    va, vb = a.var(ddof=1), b.var(ddof=1)
    pooled = np.sqrt(((na-1)*va + (nb-1)*vb) / (na+nb-2)) if (na+nb-2)
    if pooled == 0 or np.isnan(pooled): return np.nan
    return (a.mean() - b.mean()) / pooled

# ----- COMPARISONS -----
SEGMENTS = ["first","middle","last"]
CONTROLS = [c for c in ["p_speaking_duration_sec","p_turns"] if c in d

rows_role = []
rows_fund = []

for seg in SEGMENTS:
    dseg = df[df["segment"]==seg].copy()
    if dseg.empty: continue

    # ---- Facilitator vs Non-facilitator ----
    for f in ann_counts:
        g1 = dseg[dseg["is_fac"]==1][f]
        g0 = dseg[dseg["is_fac"]==0][f]
        mu1, mu0 = g1.mean(skipna=True), g0.mean(skipna=True)
        d = cohens_d(g1, g0)

    # regression with controls + FE
    rhs = [f] # (we're testing mean difference, so model f ~ is_f
    # We'll predict the feature from is_fac to ask "do facilitator
    # (OLS is fine for quick directional tests; counts aren't huge
    formula = f"{f} ~ is_fac"
    if CONTROLS: formula += " + " + " + " + ".join(CONTROLS
    if "conference" in dseg.columns: formula += " + C(conference)"

```

```

        if "year" in dseg.columns:            formula += " + C(year)"
        m = None
        try:
            m = smf.ols(formula, data=dseg).fit(cov_type="HC3")
            p_isfac = float(m.pvalues.get("is_fac", np.nan))
        except Exception:
            p_isfac = np.nan

        rows_role.append(dict(segment=seg, feature=f, mean_fac=mu1, me
                               diff=mu1-mu0, cohens_d=d, p_is_fac=p_isf
                               n_fac=int((dseg["is_fac"]==1).sum()), n_

# ---- Funded vs Not (person-year with any funded team) ----
for f in ann_counts:
    g1 = dseg[dseg["any_funded"]==1][f]
    g0 = dseg[dseg["any_funded"]==0][f]
    mu1, mu0 = g1.mean(skipna=True), g0.mean(skipna=True)
    d = cohens_d(g1, g0)

    formula = f"{f} ~ any_funded"
    if CONTROLS:            formula += " + " + " + " + ".join(CONTROLS)
    if "conference" in dseg.columns: formula += " + C(conference)"
    if "year" in dseg.columns:      formula += " + C(year)"
    m = None
    try:
        m = smf.ols(formula, data=dseg).fit(cov_type="HC3")
        p_funded = float(m.pvalues.get("any_funded", np.nan))
    except Exception:
        p_funded = np.nan

    rows_fund.append(dict(segment=seg, feature=f, mean_funded=mu1,
                           diff=mu1-mu0, cohens_d=d, p_any_funded=
                           n_funded=int((dseg["any_funded"]==1).su

role_df = pd.DataFrame(rows_role)
fund_df = pd.DataFrame(rows_fund)

# Save tidy results
role_csv = OUT_DIR / "temporal_role_differences.csv"
fund_csv = OUT_DIR / "temporal_funded_differences.csv"
role_df.to_csv(role_csv, index=False)
fund_df.to_csv(fund_csv, index=False)
print(f"[OK] wrote {role_csv} (rows={len(role_df)})")
print(f"[OK] wrote {fund_csv} (rows={len(fund_df)})")

# ----- PLOTS -----
def nice_name(s):
    # trim long family names for plotting
    return s.replace("ann_", "").replace("_count", "")

def plot_top_diffs(df_sub, value_col, p_col, seg, title_prefix, fname_
    if df_sub.empty: return

```

```

# rank by absolute difference; keep top
show = (df_sub
        .assign(absdiff=lambda x: x[value_col].abs())
        .sort_values("absdiff", ascending=False)
        .head(top)
        .sort_values(value_col))
if show.empty: return
fig, ax = plt.subplots(figsize=(12, max(4.5, 0.45*len(show))))
ylab = [nice_name(f) for f in show["feature"]]
ax.barh(ylab, show[value_col].values, alpha=0.9)
# mark significance stars at bar ends
for yi, (val, p) in enumerate(zip(show[value_col].values, show[p_col])):
    star = "***" if p < 0.001 else "**" if p < 0.01 else "*" if p < 0.05 else ""
    ax.text(val + (0.02*show["absdiff"].max() if val>=0 else -0.02),
            yi, star, va="center", ha="left" if val>=0 else "right")
ax.axvline(0, color="k", lw=1)
ax.set_xlabel("Difference in mean counts (group A - group B)")
ax.set_title(f"{title_prefix} - {seg.title()} (bars = group difference)")
fig.tight_layout()
out = OUT_DIR / f"{fname_prefix}_{seg}.png"
fig.savefig(out, dpi=170)
plt.close(fig)
print(f"[OK] wrote {out}")

# Role: facilitators (mean_fac - mean_nonfac)
for seg in SEGMENTS:
    sub = role_df[role_df["segment"]==seg].copy()
    plot_top_diffs(sub, value_col="diff", p_col="p_is_fac",
                   seg=seg,
                   title_prefix="Facilitator vs Non-facilitator (mean difference)",
                   fname_prefix="role_diff_bars")

# Funded: funded - nonfunded
for seg in SEGMENTS:
    sub = fund_df[fund_df["segment"]==seg].copy()
    plot_top_diffs(sub, value_col="diff", p_col="p_any_funded",
                   seg=seg,
                   title_prefix="Funded vs Non-funded (mean annotation difference)",
                   fname_prefix="funded_diff_bars")

# ----- QUICK READ-OUT -----
def quick_summary(tab, group_cols, diff_col, p_col, k=5):
    lines = []
    for seg in SEGMENTS:
        sub = tab[tab["segment"]==seg].copy()
        if sub.empty: continue
        top = (sub.assign(absdiff=lambda x: x[diff_col].abs())
               .sort_values([p_col, "absdiff"], ascending=[True, False])
               .head(k))
        if top.empty: continue
        lines.append(f"\n[{seg.upper()}] strongest differences:")
        for _, r in top.iterrows():

```

```

        feat = nice_name(r["feature"])
        star = "***" if r[p_col] < 0.001 else "*" if r[p_col] < 0
        lines.append(f" - {feat:35s} diff={r[diff_col]: .3f} d={
return "\n".join(lines)

print(quick_summary(role_df, ["segment"], "diff", "p_is_fac"))
print(quick_summary(fund_df, ["segment"], "diff", "p_any_funded"))

```

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal_role_differences.csv (rows=27)

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal_funded_differences.csv (rows=27)

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/role_diff_bars_first.png

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/role_diff_bars_middle.png

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/role_diff_bars_last.png

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/funded_diff_bars_first.png

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/funded_diff_bars_middle.png

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/funded_diff_bars_last.png

[FIRST] strongest differences:

- knowledge_sharing	diff= 4.505	d= 0.82	p=4.76e-05 (***)
- participation_dynamics	diff= 17.061	d= 3.28	p=5.6e-05 (***)
- coordination_and_decision_practices	diff= 21.493	d= 3.30	p=7.94e-05 (***)
- relational_climate	diff= 20.457	d= 3.06	p=0.00208 (**)
- none	diff= 0.927	d= 0.54	p=0.00309 (**)

[MIDDLE] strongest differences:

- participation_dynamics	diff= 5.736	d= 2.22	p=2.59e-10 (***)
- coordination_and_decision_practices	diff= 3.668	d= 2.35	p=5.6e-10 (***)
- knowledge_sharing	diff= 3.394	d= 0.58	p=8.78e-06 (***)
- integration_practices	diff= 1.263	d= 1.39	p=1.08e-0

```

5 (***)
  - none                                diff= 0.306  d= 0.11  p=0.00401
(**)

[LAST] strongest differences:
  - participation_dynamics              diff= 8.346  d= 2.44  p=5.68e-0
8 (***)
  - knowledge_sharing                  diff= 7.511  d= 0.88  p=0.00030
4 (***)
  - coordination_and_decision_practices diff= 11.906  d= 1.99  p=0.0006
31 (***)
  - integration_practices              diff= 3.135  d= 1.57  p=0.004
(**)
  - none                                diff= 1.768  d= 0.55  p=0.00461
(**)

[FIRST] strongest differences:
  - coordination_and_decision_practices diff=-2.799  d=-0.31  p=0.00061
8 (***)
  - relational_climate                 diff=-2.399  d=-0.27  p=0.00184
(**)
  - participation_dynamics            diff=-2.017  d=-0.28  p=0.00238
(**)
  - idea_management                   diff= 0.645  d= 0.26  p=0.0167
(*)
  - knowledge_sharing                 diff= 1.128  d= 0.20  p=0.0195
(*)

[MIDDLE] strongest differences:
  - coordination_and_decision_practices diff=-0.539  d=-0.28  p=8.53e-0
6 (***)
  - participation_dynamics            diff=-0.758  d=-0.24  p=1.12e-0
5 (***)
  - knowledge_sharing                 diff= 1.893  d= 0.32  p=0.00342
(**)
  - relational_climate                 diff=-0.150  d=-0.04  p=0.193 (
ns)
  - evaluation_practices              diff= 0.572  d= 0.24  p=0.205 (
ns)

[LAST] strongest differences:
  - participation_dynamics            diff=-0.744  d=-0.18  p=1.93e-0
6 (***)
  - coordination_and_decision_practices diff=-0.705  d=-0.10  p=3.61e-0
5 (***)
  - knowledge_sharing                 diff= 4.073  d= 0.47  p=0.00313
(**)
  - relational_climate                 diff= 0.762  d= 0.09  p=0.0167
(*)
  - integration_practices             diff= 0.230  d= 0.11  p=0.27 (n
s)

```

In [2]: # =====

```

# ON-TEAM vs NOT-ON-TEAM (new comparison)
# =====

rows_team = []
for seg in SEGMENTS:
    dseg = df[df["segment"]==seg].copy()
    if dseg.empty: continue

    for f in ann_counts:
        g1 = dseg[dseg["any_team"]==1][f]
        g0 = dseg[dseg["any_team"]==0][f]
        mu1, mu0 = g1.mean(skipna=True), g0.mean(skipna=True)
        d = cohens_d(g1, g0)

        formula = f"{f} ~ any_team"
        if CONTROLS: formula += " + " + " + ".join(CONTROLS)
        if "conference" in dseg.columns: formula += " + C(conference)"
        if "year" in dseg.columns: formula += " + C(year)"
        try:
            m = smf.ols(formula, data=dseg).fit(cov_type="HC3")
            p_team = float(m.pvalues.get("any_team", np.nan))
        except Exception:
            p_team = np.nan

        rows_team.append(dict(segment=seg, feature=f,
                               mean_team=mu1, mean_nonteam=mu0,
                               diff=mu1-mu0, cohens_d=d, p_any_team=p_team,
                               n_team=int((dseg["any_team"]==1).sum()),
                               n_nonteam=int((dseg["any_team"]==0).sum()))

team_df = pd.DataFrame(rows_team)
team_csv = OUT_DIR / "temporal_team_differences.csv"
team_df.to_csv(team_csv, index=False)
print(f"[OK] wrote {team_csv} (rows={len(team_df)})")

for seg in SEGMENTS:
    sub = team_df[team_df["segment"]==seg].copy()
    plot_top_diffs(sub, value_col="diff", p_col="p_any_team",
                   seg=seg,
                   title_prefix="On-Team vs Not-On-Team (mean annotation)",
                   fname_prefix="team_diffBars")

print(quick_summary(team_df, ["segment"], "diff", "p_any_team"))

```

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/temporal_team_differences.csv (rows=27)
 [OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/team_diff_bars_first.png
 [OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/team_diff_bars_middle.png
 [OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/temporal-facilitator-significant/team_diff_bars_last.png

[FIRST] strongest differences:

- coordination_and_decision_practices	diff=-3.517	d=-0.40	p=6.64e-08
8 (***)			
- participation_dynamics	diff=-2.697	d=-0.38	p=1.45e-06
6 (***)			
- knowledge_sharing	diff= 1.846	d= 0.33	p=1.01e-05
5 (***)			
- relational_climate	diff=-2.915	d=-0.33	p=2.36e-05
5 (***)			
- idea_management	diff= 0.711	d= 0.29	p=0.00274
(**)			

[MIDDLE] strongest differences:

- participation_dynamics	diff=-1.149	d=-0.38	p=1.63e-09
9 (***)			
- coordination_and_decision_practices	diff=-0.721	d=-0.38	p=4.55e-09
9 (***)			
- knowledge_sharing	diff= 1.366	d= 0.23	p=0.00144
(**)			
- evaluation_practices	diff= 0.598	d= 0.25	p=0.0153
(*)			
- integration_practices	diff=-0.083	d=-0.08	p=0.0245
(*)			

[LAST] strongest differences:

- participation_dynamics	diff=-1.150	d=-0.28	p=2.67e-13
3 (***)			
- coordination_and_decision_practices	diff=-1.330	d=-0.20	p=3.58e-13
3 (***)			
- none	diff= 1.183	d= 0.37	p=0.00010
9 (***)			
- knowledge_sharing	diff= 4.173	d= 0.49	p=0.00047
(***)			
- idea_management	diff= 2.846	d= 0.50	p=0.00174
(**)			

reconsolidating code (refer to 10/08 meeting notes w/ evey)

```

In [4]: # =====
# PERSON × TEMPORAL THIRDS – updated for new JSONs (handles list-or-di
# Consolidates annotation metrics → ann_<family>_score
#   – numeric score present → add that score
#   – no numeric score      → add 1 (presence)
# Outputs:
#   – ALL_person_session_segment_CONSOLIDATED.csv
#   – ALL_person_year_segment_CONSOLIDATED.csv
# =====

import json, re
from pathlib import Path
from collections import defaultdict
import numpy as np
import pandas as pd

# ----- PATHS -----
DATA_ROOT = Path("/Users/maxchalekson/Desktop/gemini_data_analysis/dat

OUT_MBP    = Path("/Users/maxchalekson/Northwestern University/Summer-
OUT_STUDIO = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_hum
OUT_DIR     = OUT_MBP if OUT_MBP.exists() else OUT_STUDIO
OUT_DIR.mkdir(parents=True, exist_ok=True)

print(f"[INFO] DATA_ROOT = {DATA_ROOT}")
print(f"[INFO] OUT_DIR    = {OUT_DIR}")

# ----- HELPERS -----
_time_pat = re.compile(r"^(?:(\d+):)?(\d{1,2}):(\d{2})(?:\.(\d+))?$")

def time_to_sec(v):
    if v is None: return None
    if isinstance(v, (int, float)): return float(v)
    s = str(v).strip()
    if s.isdigit(): return float(int(s))
    m = _time_pat.match(s)
    if m:
        h = int(m.group(1) or 0); mm = int(m.group(2)); ss = int(m.gro
        sec = h*3600 + mm*60 + ss
        if m.group(4): sec += float("0."+m.group(4))
        return float(sec)
    return None

def parse_range(s):
    if not s or "-" not in str(s): return (None, None)
    a, b = [t.strip() for t in str(s).split("-", 1)]
    return (time_to_sec(a), time_to_sec(b))

def canon_name(s):
    if not isinstance(s, str): return ""
    s = s.replace(" ", "PhD", "").replace(" PhD", "")
    if " – " in s: s = s.split(" – ")[0]

```

```

    return s.strip()

def which_third(t, T):
    if not (isinstance(t, (int, float)) and isinstance(T, (int, float))):
        return None
    third = T / 3.0
    return "first" if t < third else ("middle" if t < 2 * third else "last")

def normalize_json(J_raw):
    """
    Accept:
    - dict (maybe with 'all_data'), or
    - top-level list (assume that's all_data)
    Return dict with 'all_data' always present (list).
    """
    if isinstance(J_raw, list):
        return {"all_data": J_raw}
    if isinstance(J_raw, dict):
        if "all_data" not in J_raw and isinstance(J_raw.get("data"), list):
            J_raw = {**J_raw, "all_data": J_raw["data"]}
        if "all_data" not in J_raw:
            for k, v in J_raw.items():
                if isinstance(v, list) and v and isinstance(v[0], dict):
                    J_raw = {**J_raw, "all_data": v}
                    break
        if "all_data" not in J_raw:
            J_raw["all_data"] = []
        return J_raw
    return {"all_data": []}

def session_duration(J):
    v = J.get("total_speaking_length") or J.get("meeting_duration_sec")
    if isinstance(v, str): v = time_to_sec(v)
    if isinstance(v, (int, float)) and v > 0: return float(v)
    # fallback: max end_time in all_data
    mx = 0.0
    for ev in J.get("all_data", []):
        if not isinstance(ev, dict): continue
        e = time_to_sec(ev.get("end_time"))
        if e is None:
            _, e = parse_range(ev.get("timestamp"))
        if e is not None:
            mx = max(mx, float(e))
    return mx if mx > 0 else None

def parse_path_info(path: Path):
    conf = path.parent.parent.name if path.parent and path.parent.parent
    base = path.stem
    sid = None
    for tok in base.split("_"):
        if tok.upper().startswith("S") and tok[1:].isdigit():
            sid = tok.upper(); break

```

```

    yr = None
    for tok in base.split("_"):
        if tok.isdigit() and len(tok) == 4:
            yr = int(tok); break
    return conf, yr, (sid or base)

# Annotation alias → canonical family column names
ANN_ALIAS = {
    "relational_climate" : "ann_relational_climate_sco
    "participation_dynamics" : "ann_participation_dynamics
    "coordination_and_decision" : "ann_coordination_and_decis
    "coordination_and_decision_practices": "ann_coordination_and_decis
    "evaluation_practices" : "ann_evaluation_practices_s
    "knowledge_sharing" : "ann_knowledge_sharing_scor
    "information_seeking" : "ann_information_seeking_sc
    "integration_practices" : "ann_integration_practices_
    "idea_management" : "ann_idea_management_score"
    "none" : "ann_none_score",
}

def canonical_fam(raw_cat: str) -> str | None:
    if not raw_cat: return None
    key = str(raw_cat).strip().lower().replace(" ", " ").replace("_", " ")
    return ANN_ALIAS.get(key, None)

# ----- SCAN & BUILD -----
rows = []
files = sorted(DATA_ROOT.glob("*/*/*.json"))
print(f"[INFO] Found {len(files)} JSON files")

for js in files:
    # ignore feature-summary files (no temporal info)
    if js.name.startswith("features_"):
        print(f"[SKIP:no-duration] {js.name}")
        continue

    try:
        J_raw = json.loads(js.read_text())
    except Exception as e:
        print(f"[WARN] Skipping {js.name}: read error {e}")
        continue

    J = normalize_json(J_raw)
    T = session_duration(J)
    if not T or T <= 0:
        print(f"[SKIP:no-duration] {js.name}")
        continue

    conference, year, session_id = parse_path_info(js)

    controls = defaultdict(lambda: {"p_speaking_duration_sec": 0.0, "p
    anns = defaultdict(lambda: defaultdict(float)) # {(who, seg)[fam_

```

```

fams_seen = set()

for ev in J.get("all_data", []):
    if not isinstance(ev, dict):
        continue

    who = canon_name(ev.get("speaker") or ev.get("speaker_name")) or 0
    if not who:
        continue

    # time fields
    s = time_to_sec(ev.get("start_time"))
    e = time_to_sec(ev.get("end_time"))
    if (s is None or e is None) and isinstance(ev.get("timestamp"), str):
        s2, e2 = parse_range(ev.get("timestamp"))
        s = s if s is not None else s2
        e = e if e is not None else e2

    # duration
    if isinstance(ev.get("speaking_duration"), (int, float)):
        d = float(ev["speaking_duration"])
    elif s is not None and e is not None:
        d = max(0.0, float(e) - float(s))
    else:
        d = 0.0

    # midpoint for thirds
    if s is not None and e is not None:
        mid = 0.5*(float(s)+float(e))
    elif s is not None:
        mid = float(s) + d/2.0
    elif e is not None:
        mid = float(e) - d/2.0
    else:
        mid = None

    seg = which_third(mid, T) if mid is not None else "middle"

    # controls
    controls[(who, seg)]["p_speaking_duration_sec"] += d
    controls[(who, seg)]["p_turns"] += 1

    # annotations (consolidated)
    a = ev.get("annotations")
    if isinstance(a, dict) and a:
        for raw_cat, payload in a.items():
            fam_col = canonical_fam(raw_cat)
            if not fam_col:
                continue
            fams_seen.add(fam_col)
            score = None
            if isinstance(payload, dict):

```

```

        sc = payload.get("score", None)
        if isinstance(sc, (int, float)):
            score = float(sc)
        # If there is a numeric score, add it; else add 1 for
        anns[(who, seg)][fam_col] += (score if score is not None else 1)

# materialize rows for each (person, segment)
keys = set(controls.keys()) | set((k for k in anns.keys()))
for (who, seg) in keys:
    row = {
        "person_name": who,
        "conference": conference,
        "year": year,
        "session_id": session_id,
        "segment": seg,
        "session_duration_sec": float(T),
        "p_speaking_duration_sec": float(controls[(who, seg)]["p_speaking_duration_sec"]),
        "p_turns": int(controls[(who, seg)]["p_turns"]),
    }
    for fam_col, total_score in anns[(who, seg)].items():
        row[fam_col] = float(total_score)
    rows.append(row)

# ----- WRITE (session-level) -----
pss = pd.DataFrame(rows)

# ensure numeric for *_score
for c in [c for c in pss.columns if c.endswith("_score")]:
    pss[c] = pd.to_numeric(pss[c], errors="coerce")

print(f"[OK] Built temporal dataset - rows={pss.shape[0]}, cols={pss.shape[1]}")
out_session = OUT_DIR / "ALL_person_session_segment_CONSOLIDATED.csv"
pss.to_csv(out_session, index=False)
print(f"[SAVED] {out_session}")

# ----- Aggregate to person-year-segment -----
group_keys = ["person_name", "conference", "year", "segment"]
num_cols = [c for c in pss.columns if c not in group_keys]
pys = (pss.groupby(group_keys, dropna=False, as_index=False)[num_cols]
        .sum(numeric_only=True))

out_year = OUT_DIR / "ALL_person_year_segment_CONSOLIDATED.csv"
pys.to_csv(out_year, index=False)
print(f"[SAVED] {out_year}")

# quick sanity ping
ann_cols = [c for c in pys.columns if c.endswith("_score")]
print(f"\n[CHECK] annotation columns ({len(ann_cols)}):", sorted(ann_cols))
print("[CHECK] segments:", pys["segment"].value_counts(dropna=False).sort_index())
print("[CHECK] speaking-time by segment (hrs):",
      (pys.groupby('segment')['p_speaking_duration_sec'].sum()/3600.0))

```

[INFO] DATA_ROOT = /Users/maxchalekson/Desktop/gemini_data_analysis/dat

a

```
[INFO] OUT_DIR    = /Users/maxchalekson/Projects/NICO-Research/NICO_huma
n-gemini/facilitator-identification/3-reconsolidating-features-temporal
[INFO] Found 476 JSON files
[SKIP:no-duration] features_2020_11_05_NES_S1.json
[SKIP:no-duration] features_2020_11_05_NES_S2.json
[SKIP:no-duration] features_2020_11_05_NES_S3.json
[SKIP:no-duration] features_2020_11_05_NES_S4.json
[SKIP:no-duration] features_2020_11_05_NES_S5.json
[SKIP:no-duration] features_2020_11_05_NES_S6.json
[SKIP:no-duration] features_2020_11_06_NES_S1.json
[SKIP:no-duration] features_2020_11_06_NES_S10.json
[SKIP:no-duration] features_2020_11_06_NES_S11.json
[SKIP:no-duration] features_2020_11_06_NES_S12.json
[SKIP:no-duration] features_2020_11_06_NES_S2.json
[SKIP:no-duration] features_2020_11_06_NES_S3.json
[SKIP:no-duration] features_2020_11_06_NES_S4.json
[SKIP:no-duration] features_2020_11_06_NES_S7.json
[SKIP:no-duration] features_2020_11_06_NES_S8.json
[SKIP:no-duration] features_2020_11_06_NES_S9.json
[SKIP:no-duration] features_2020_11_05_NES_S1_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S1_end.json
[SKIP:no-duration] features_2020_11_05_NES_S1_middle.json
[SKIP:no-duration] features_2020_11_05_NES_S2_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S2_end.json
[SKIP:no-duration] features_2020_11_05_NES_S2_middle.json
[SKIP:no-duration] features_2020_11_05_NES_S3_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S3_end.json
[SKIP:no-duration] features_2020_11_05_NES_S3_middle.json
[SKIP:no-duration] features_2020_11_05_NES_S4_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S4_end.json
[SKIP:no-duration] features_2020_11_05_NES_S4_middle.json
[SKIP:no-duration] features_2020_11_05_NES_S5_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S5_end.json
[SKIP:no-duration] features_2020_11_05_NES_S5_middle.json
[SKIP:no-duration] features_2020_11_05_NES_S6_beginning.json
[SKIP:no-duration] features_2020_11_05_NES_S6_end.json
[SKIP:no-duration] features_2020_11_05_NES_S6_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S10_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S10_end.json
[SKIP:no-duration] features_2020_11_06_NES_S10_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S11_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S11_end.json
[SKIP:no-duration] features_2020_11_06_NES_S11_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S12_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S12_end.json
[SKIP:no-duration] features_2020_11_06_NES_S12_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S1_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S1_end.json
[SKIP:no-duration] features_2020_11_06_NES_S1_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S2_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S2_end.json
```

[SKIP:no-duration] features_2020_11_06_NES_S2_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S3_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S3_end.json
[SKIP:no-duration] features_2020_11_06_NES_S3_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S4_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S4_end.json
[SKIP:no-duration] features_2020_11_06_NES_S4_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S7_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S7_end.json
[SKIP:no-duration] features_2020_11_06_NES_S7_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S8_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S8_end.json
[SKIP:no-duration] features_2020_11_06_NES_S8_middle.json
[SKIP:no-duration] features_2020_11_06_NES_S9_beginning.json
[SKIP:no-duration] features_2020_11_06_NES_S9_end.json
[SKIP:no-duration] features_2020_11_06_NES_S9_middle.json
[SKIP:no-duration] features_2021_05_20_ABI_S1.json
[SKIP:no-duration] features_2021_05_20_ABI_S2.json
[SKIP:no-duration] features_2021_05_20_ABI_S3.json
[SKIP:no-duration] features_2021_05_20_ABI_S4.json
[SKIP:no-duration] features_2021_05_20_ABI_S5.json
[SKIP:no-duration] features_2021_05_20_ABI_S6.json
[SKIP:no-duration] features_2021_05_20_ABI_S7.json
[SKIP:no-duration] features_2021_05_20_ABI_S8.json
[SKIP:no-duration] features_2021_05_21_ABI_S1.json
[SKIP:no-duration] features_2021_05_21_ABI_S10.json
[SKIP:no-duration] features_2021_05_21_ABI_S11.json
[SKIP:no-duration] features_2021_05_21_ABI_S12.json
[SKIP:no-duration] features_2021_05_21_ABI_S13.json
[SKIP:no-duration] features_2021_05_21_ABI_S14.json
[SKIP:no-duration] features_2021_05_21_ABI_S15.json
[SKIP:no-duration] features_2021_05_21_ABI_S16.json
[SKIP:no-duration] features_2021_05_21_ABI_S2.json
[SKIP:no-duration] features_2021_05_21_ABI_S3.json
[SKIP:no-duration] features_2021_05_21_ABI_S4.json
[SKIP:no-duration] features_2021_05_21_ABI_S5.json
[SKIP:no-duration] features_2021_05_21_ABI_S6.json
[SKIP:no-duration] features_2021_05_21_ABI_S7.json
[SKIP:no-duration] features_2021_05_21_ABI_S8.json
[SKIP:no-duration] features_2021_05_21_ABI_S9.json
[SKIP:no-duration] features_output-2021-05-20-ABI-S1.json
[SKIP:no-duration] features_2021_10_07_CMC_S1.json
[SKIP:no-duration] features_2021_10_07_CMC_S2.json
[SKIP:no-duration] features_2021_10_07_CMC_S3.json
[SKIP:no-duration] features_2021_10_07_CMC_S4.json
[SKIP:no-duration] features_2021_10_07_CMC_S5.json
[SKIP:no-duration] features_2021_10_07_CMC_S6.json
[SKIP:no-duration] features_2021_10_07_CMC_S7.json
[SKIP:no-duration] features_2021_10_08_CMC_S1.json
[SKIP:no-duration] features_2021_10_08_CMC_S10.json
[SKIP:no-duration] features_2021_10_08_CMC_S11.json
[SKIP:no-duration] features_2021_10_08_CMC_S12.json

[illegible]

[illegible]

```

[SKIP:no-duration] features_2022_04_08_MND_S9.json
[SKIP:no-duration] 2020NES_outcome.json
[SKIP:no-duration] 2020NES_session_outcomes.json
[SKIP:no-duration] 2021ABI_outcome.json
[SKIP:no-duration] 2021ABI_person_to_team.json
[SKIP:no-duration] 2021ABI_session_outcomes.json
[SKIP:no-duration] 2021MND_outcome.json
[SKIP:no-duration] 2021MND_person_to_team.json
[SKIP:no-duration] 2021MND_session_outcomes.json
[SKIP:no-duration] 2021MZT_outcome.json
[SKIP:no-duration] 2021MZT_person_to_team.json
[SKIP:no-duration] 2021MZT_session_outcomes.json
[SKIP:no-duration] 2021_10_01_MZT_S5.json
[SKIP:no-duration] 2021SLU_outcome.json
[SKIP:no-duration] 2021SLU_person_to_team.json
[SKIP:no-duration] 2021SLU_session_outcomes.json
[OK] Built temporal dataset – rows=7382, cols=17
[SAVED] /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_session_segment_CONSOLIDATED.csv
[SAVED] /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED.csv

[CHECK] annotation columns (9): ['ann_coordination_and_decision_practices_score', 'ann_evaluation_practices_score', 'ann_idea_management_score', 'ann_information_seeking_score', 'ann_integration_practices_score', 'ann_knowledge_sharing_score', 'ann_none_score', 'ann_participation_dynamics_score', 'ann_relational_climate_score']
[CHECK] segments: {'last': 1157, 'first': 1090, 'middle': 962}
[CHECK] speaking-time by segment (hrs): {'first': 559.94, 'last': 741.74, 'middle': 561.04}

```

regression model + control

```

In [10]: from pathlib import Path
import pandas as pd
import numpy as np

# --- paths ---
FEAT = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_session_segment_CONSOLIDATED.csv")
OUT = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED.csv")

assert FEAT.exists(), f"Missing features: {FEAT}"
assert OUT.exists(), f"Missing outcomes: {OUT}"

# --- load ---
features = pd.read_csv(FEAT)
outcomes = pd.read_csv(OUT)

# --- normalize join keys ---

```

```

for df in (features, outcomes):
    if "person_name" in df.columns:
        df["person_name"] = df["person_name"].astype(str).str.strip()
    if "year" in df.columns:
        df["year"] = pd.to_numeric(df["year"], errors="coerce").astype(

# keep only the outcomes we use downstream (add more if needed)
keep_out = [c for c in ["person_name", "year", "role_facilitator", "num_t
outcomes_small = outcomes[keep_out].drop_duplicates(subset=["person_na

# --- merge ---
merged = features.merge(outcomes_small, on=["person_name", "year"], how

# --- light hygiene ---
num_cols = ["num_teams", "num_funded_teams", "sessions_total"]
for c in num_cols:
    if c in merged.columns:
        merged[c] = pd.to_numeric(merged[c], errors="coerce")

if "role_facilitator" in merged.columns:
    merged["role_facilitator"] = pd.to_numeric(merged["role_facilitato

# confirm segment labeling is clean
if "segment" in merged.columns:
    merged["segment"] = merged["segment"].astype(str).str.lower().str.
    merged = merged[merged["segment"].isin(["first", "middle", "last"])]

print("[OK] features:", FEAT, "rows=", len(features))
print("[OK] outcomes:", OUT, "rows=", len(outcomes_small))
print("[OK] merged rows:", len(merged))

# quick diagnostics
present = [c for c in ["num_teams", "num_funded_teams", "role_facilitato
print("[CHECK] outcomes present:", present)
print(merged[present].describe(include="all") if present else "- no ou

# --- save regression-ready table ---
OUT_MERGED = FEAT.with_name("ALL_person_year_segment_CONSOLIDATED_WITH
merged.to_csv(OUT_MERGED, index=False)
print("[SAVED]", OUT_MERGED)

```

```
[OK] features: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED.csv rows= 3209
[OK] outcomes: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/1-facilitator-identification-significant/ALL_person_year_WITH_OUTCOMES.csv rows= 732
[OK] merged rows: 3209
[CHECK] outcomes present: ['num_teams', 'num_funded_teams', 'role_facilitator']
```

	num_teams	num_funded_teams	role_facilitator
count	2389.000000	2389.000000	3209.000000
mean	0.790707	0.259104	0.076348
std	1.205333	0.702536	0.265595
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	1.000000	0.000000	0.000000
max	7.000000	5.000000	1.000000

```
[SAVED] /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.csv
```

running the different regression models

```
In [11]: import pandas as pd, numpy as np, matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from pathlib import Path

# ===== paths =====
CSV = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED.csv")
OUT = CSV.parent

df = pd.read_csv(CSV)

# --- light cleaning ---
for c in ["num_teams", "num_funded_teams", "sessions_total", "year", "role_facilitator"]:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

df["segment"] = df["segment"].astype(str).str.lower().str.strip()
df = df[df["segment"].isin(["first", "middle", "last"])]

# feature families (consolidated sums you built)
features = [c for c in df.columns if c.endswith("_score") and c.startswith("num_")]
controls = [c for c in df.columns if c.endswith("_sec") or c.endswith("_turns") or c.endswith("_sessions")]

print("[INFO] rows:", len(df))
print("[INFO] features:", features)
print("[INFO] controls:", controls)
print(df[features + controls].describe())
```

```

# ===== distribution check -> modeling guidance =====
def check_dispersion(y):
    y = pd.to_numeric(y, errors="coerce").dropna()
    return {"mean": y.mean(), "var": y.var(ddof=1), "zeros": (y==0).me

for y in [c for c in ["num_teams", "num_funded_teams"] if c in df.columns]:
    stats = check_dispersion(df[y])
    print(f"\n{y}: mean={stats['mean']:.2f} var={stats['var']:.2f} zer
    if stats["zeros"] > 0.5:
        print(" → Many zeros: consider Zero-Inflated / hurdle model l
    elif stats["var"] > stats["mean"]:
        print(" → Overdispersion: Negative Binomial preferred over Po
    else:
        print(" → Poisson could be OK; OLS is still readable as basel

# ===== helpers =====
def safe_fit(formula, data, model="ols"):
    try:
        if model=="ols":
            m = smf.ols(formula, data=data).fit(cov_type="HC3") # rob
        elif model=="poisson":
            m = smf.glm(formula, data=data, family=sm.families.Poisson
        elif model=="negbin":
            m = smf.glm(formula, data=data, family=sm.families.Negativ
        elif model=="logit":
            m = smf.logit(formula, data=data).fit(dis=0, maxiter=200)
        else:
            return None
        return m
    except Exception as e:
        print("[SKIP]", formula, "|", e)
        return None

def tidy_model(m, outcome, feature, segment=None, model_name="OLS"):
    if m is None or feature not in m.params.index:
        return None
    ci = m.conf_int().loc[feature]
    return {
        "outcome": outcome,
        "feature": feature,
        "segment": segment,
        "model": model_name,
        "coef": float(m.params[feature]),
        "se": float(m.bse[feature]),
        "p": float(m.pvalues[feature]),
        "ci_low": float(ci.iloc[0]),
        "ci_high": float(ci.iloc[1]),
        "r2_like": float(getattr(m, "rsquared", np.nan) if hasattr(m,
        "n": int(m.nobs)
    }

```

```

# ===== 1) OLS by segment (simple, readable) =====
ols_rows = []
for seg in ["first", "middle", "last"]:
    dseg = df[df["segment"]==seg].copy()
    if dseg.empty:
        continue
    for y in [c for c in ["num_teams", "num_funded_teams"] if c in dseg]:
        if dseg[y].nunique(dropna=True) < 2:
            continue
        for x in features:
            rhs = [x] + controls + ["C(conference)", "C(year)"]
            formula = f"{y} ~ " + " + ".join([r for r in rhs if r.split("_")[0] != "C"])
            m = safe_fit(formula, dseg, model="ols")
            t = tidy_model(m, y, x, seg, model_name="OLS")
            if t: ols_rows.append(t)

ols_df = pd.DataFrame(ols_rows)
if not ols_df.empty:
    ols_df.to_csv(OUT / "models_ols_by_segment.csv", index=False)
    print("[OK] models_ols_by_segment.csv", len(ols_df))

# ===== 2) Count models with overdispersion check =====
glm_rows = []
for seg in ["first", "middle", "last"]:
    dseg = df[df["segment"]==seg].copy()
    for y in [c for c in ["num_teams", "num_funded_teams"] if c in dseg]:
        if dseg[y].nunique(dropna=True) < 2:
            continue
        # pick Poisson vs NegBin by dispersion
        stats = check_dispersion(dseg[y])
        fam = "negbin" if stats["var"] > stats["mean"] else "poisson"
        for x in features:
            rhs = [x] + controls + ["C(conference)", "C(year)"]
            formula = f"{y} ~ " + " + ".join([r for r in rhs if r.split("_")[0] != "C"])
            m = safe_fit(formula, dseg, model=fam)
            t = tidy_model(m, y, x, seg, model_name=fam.upper())
            if t: glm_rows.append(t)

glm_df = pd.DataFrame(glm_rows)
if not glm_df.empty:
    glm_df.to_csv(OUT / "models_counts_glm_by_segment.csv", index=False)
    print("[OK] models_counts_glm_by_segment.csv", len(glm_df))

# ===== 3) Logistic: facilitator vs not, with segment interaction
logit_rows = []
if "role_facilitator" in df.columns:
    d = df.dropna(subset=["role_facilitator"]).copy()
    d["role_facilitator"] = (pd.to_numeric(d["role_facilitator"], errors="coerce") > 0).astype(int)
    for x in features:
        # interaction with segment to test temporal differences
        rhs = [f"{x}*C(segment)"] + controls + ["C(conference)", "C(year)"]
        formula = "role_facilitator ~ " + " + ".join([r for r in rhs if r.split("_")[0] != "C"])

```

```

m = safe_fit(formula, d, model="logit")
if m is None:
    continue
# collect the main term + interactions for interpretability
for term in [p for p in m.params.index if p.startswith(x)]:
    ci = m.conf_int().loc[term]
    logit_rows.append({
        "outcome": "role_facilitator", "feature": term, "segment":
        "coef": float(m.params[term]), "se": float(m.bse[term]),
        "ci_low": float(ci.iloc[0]), "ci_high": float(ci.iloc[1])
        "r2_like": float(getattr(m, "rsquared", np.nan)),
        "n": int(m.nobs)
    })

logit_df = pd.DataFrame(logit_rows)
if not logit_df.empty:
    logit_df.to_csv(OUT / "models_logit_facilitator_interactions.csv",
    print("[OK] models_logit_facilitator_interactions.csv", len(logit_

# ===== quick plots: which features matter most per segment? (OLS)
if not ols_df.empty:
    for seg in ["first", "middle", "last"]:
        sub = ols_df[(ols_df["segment"]==seg) & (ols_df["p"]<0.05)].co
        if sub.empty: continue
        # rank by max absolute coef across outcomes
        rank = (sub.pivot_table(index="feature", columns="outcome", va
            .abs().max(axis=1).sort_values(ascending=False).head
        keep = sub[sub["feature"].isin(rank.index)]
        fig, ax = plt.subplots(figsize=(10, 0.5*len(rank)+2))
        for i, feat in enumerate(rank.index[::-1]):
            row = keep[keep["feature"]==feat].sort_values("outcome")
            ax.errorbar(row["coef"], [i]*len(row), xerr=1.96*row["se"]
            ax.axvline(0, color="k", lw=1)
            ax.set_yticks(range(len(rank.index[::-1]))); ax.set_yticklabel
            ax.set_title(f"Significant features (OLS, 95% CI) - {seg}")
            ax.set_xlabel("Coefficient")
            fig.tight_layout()
            fig.savefig(OUT / f"ols_sig_features_{seg}.png", dpi=150)
            plt.close(fig)

print("\n[Done] Outputs written next to the merged CSV:")
print(" - models_ols_by_segment.csv")
print(" - models_counts_glm_by_segment.csv")
print(" - models_logit_facilitator_interactions.csv")
print(" - ols_sig_features_first/middle/last.png")

```

```
[INFO] rows: 3209
[INFO] features: ['ann_knowledge_sharing_score', 'ann_evaluation_practices_score', 'ann_idea_management_score', 'ann_coordination_and_decision_practices_score', 'ann_information_seeking_score', 'ann_relational_climate_score', 'ann_none_score', 'ann_participation_dynamics_score', 'ann_integration_practices_score']
[INFO] controls: ['p_speaking_duration_sec', 'p_turns', 'sessions_total']
```

	num_teams	num_funded_teams	role_facilitator
count	2389.000000	2389.000000	3209.000000
mean	0.790707	0.259104	0.076348
std	1.205333	0.702536	0.265595
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	1.000000	0.000000	0.000000
max	7.000000	5.000000	1.000000

```
num_teams: mean=0.79 var=1.45 zeros=60.2%
```

```
→ Many zeros: consider Zero-Inflated / hurdle model later.
```

```
num_funded_teams: mean=0.26 var=0.49 zeros=84.3%
```

```
→ Many zeros: consider Zero-Inflated / hurdle model later.
```

```
[OK] models_ols_by_segment.csv 54
```

```
[SKIP] num_teams ~ ann_evaluation_practices_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix
```

```
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
warnings.warn("Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.")
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
```

```
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "  
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/  
genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.  
    warnings.warn("Negative binomial dispersion parameter alpha not "
```

[illegible]

```

genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
[SKIP] num_funded_teams ~ ann_knowledge_sharing_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix
[SKIP] num_teams ~ ann_coordination_and_decision_practices_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix
[SKIP] num_teams ~ ann_information_seeking_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix
[SKIP] num_teams ~ ann_relational_climate_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix
[SKIP] num_teams ~ ann_participation_dynamics_score + p_speaking_duration_sec + p_turns + sessions_total + C(conference) + C(year) | Singular matrix

```

```

/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/genmod/families/family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "

```

```
[OK] models_counts_glm_by_segment.csv 48
[OK] models_logit_facilitator_interactions.csv 27
```

[Done] Outputs written next to the merged CSV:

- models_ols_by_segment.csv
- models_counts_glm_by_segment.csv
- models_logit_facilitator_interactions.csv
- ols_sig_features_first/middle/last.png

condensing graphs into one chart

```
In [14]: import os, pandas as pd, numpy as np
import matplotlib.pyplot as plt

BASE = "/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/f

ols = pd.read_csv(os.path.join(BASE, "models_ols_by_segment.csv"))
glm = pd.read_csv(os.path.join(BASE, "models_counts_glm_by_segment.c
logit = pd.read_csv(os.path.join(BASE, "models_logit_facilitator_inter

# --- tidy & one frame ---
for df, kind in [(ols,"OLS"), (glm,"GLM"), (logit,"LOGIT")]:
    df["model"] = kind
    # standardize names if needed
    df["segment"] = pd.Categorical(df["segment"], ["first","middle","l

allm = pd.concat([ols, glm, logit], ignore_index=True, sort=False)

# keep only the columns we need (rename if your CSVs differ)
want = ["model", "outcome", "segment", "feature", "coef", "se", "pval"]
keep = [c for c in want if c in allm.columns]
allm = allm[keep].copy()

# --- make ONE condensed plot: coefficients across first→middle→last -
# We'll draw three small multiples (panels): OLS num_teams, OLS num_fu
panels = [
    ("OLS", "num_teams", "OLS: num_teams"),
    ("OLS", "num_funded_teams", "OLS: num_funded_teams"),
    ("LOGIT", "role_facilitator", "Logit: facilitator (log-odds)"),
]

fig, axes = plt.subplots(1, len(panels), figsize=(16, 5), sharey=True)
if len(panels)==1: axes=[axes]

for ax, (mkind, outc, title) in zip(axes, panels):
    dfp = allm[(allm["model"]==mkind) & (allm["outcome"]==outc)].dropn
    if dfp.empty:
        ax.set_title(f"{title}\n(no rows)")
        ax.axis("off")
        continue
```

```

# pick a manageable set: top |coef| features averaged over segment
order = (dfp.groupby("feature")["coef"].apply(lambda s: np.mean(np
dfp = dfp[dfp["feature"].isin(order)].copy()
dfp["feature"] = pd.Categorical(dfp["feature"], order[::-1], order

# draw a line per feature across segments, with 95% CI
for feat, dd in dfp.groupby("feature"):
    dd = dd.sort_values("segment")
    ax.plot(dd["segment"], dd["coef"], marker="o", linewidth=1.8,
    if "se" in dd:
        ci = 1.96*dd["se"]
        ax.vlines(dd["segment"], dd["coef"]-ci, dd["coef"]+ci, alp

ax.axhline(0, color="k", lw=1)
ax.set_title(title)
ax.set_xlabel("temporal third")
if ax is axes[0]: ax.set_ylabel("coefficient")
ax.tick_params(axis='x', rotation=0)
# Legend outside to keep it tidy
ax.legend(fontsize=8, bbox_to_anchor=(1.02, 1), loc="upper left",

fig.suptitle("Temporal sweep of effects: first → middle → last (coeff±
fig.tight_layout()
plt.show()

```

```

/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_27214/998655
536.py:47: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.

```

```

    for feat, dd in dfp.groupby("feature"):
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_27214/998655
536.py:47: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.

```

```

    for feat, dd in dfp.groupby("feature"):
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_27214/998655
536.py:47: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.

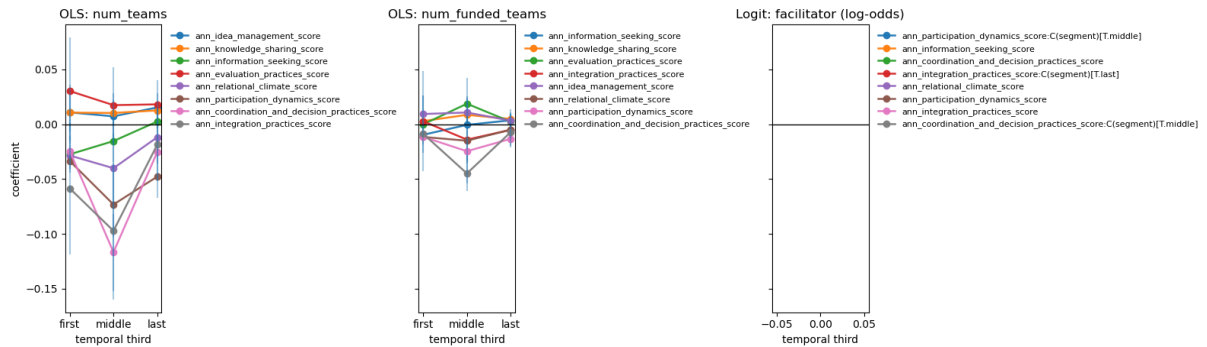
```

```

    for feat, dd in dfp.groupby("feature"):

```

Temporal sweep of effects: first → middle → last (coeff±95% CI)



```
In [16]: # --- tunables ---
N_SHOW = 6                                # how many features per OLS panel
FIGSIZE = (18, 10)                         # bigger canvas
LINE_W = 2.0
MARK_SZ = 5
ALPHA_CI = 0.25

# ----- plotting (reworked layout) -----
import matplotlib.gridspec as gridspec

plt.figure(figsize=FIGSIZE)

# 2 rows x 2 cols; right col spans both rows for logit
gs = gridspec.GridSpec(
    2, 2, width_ratios=[1.3, 1.0], height_ratios=[1, 1],
    wspace=0.35, hspace=0.35
)

# --- A) OLS num_teams (top-left) ---
ax1 = plt.subplot(gs[0, 0])
seg_order = ["first", "middle", "last"]
palette = plt.cm.tab10.colors

feat_rank = (ols_num_teams
              .groupby(ols_map["feature"])[ols_map["coef"]]
              .apply(lambda s: s.abs().mean())
              .sort_values(ascending=False)
              .head(N_SHOW).index.tolist())

handles = []
labels = []
for i, f in enumerate(feat_rank):
    sub = ols_num_teams[ols_num_teams[ols_map["feature"]] == f].copy()
    sub["__segidx"] = sub[ols_map["segment"]].str.lower().map({"first": 0, "middle": 1, "last": 2})
    sub = sub.sort_values("__segidx")
    h = ax1.plot(sub[ols_map["segment"]], sub[ols_map["coef"]],
                 marker="o", ms=MARK_SZ, lw=LINE_W, color=palette[i % 10],
                 errorbar=sub[ols_map["segment"]], sub[ols_map["coef"]],
                 yerr=1.96*sub[ols_map["se"]], fmt="none",
                 ecolor=palette[i % 10], alpha=ALPHA_CI)
```

```

        handles.append(h[0]); labels.append(_nice(f))

ax1.axhline(0, color="k", lw=0.8)
ax1.set_title("OLS: num_teams")
ax1.set_xlabel("temporal third")
ax1.set_ylabel("coefficient")
ax1.set_xticks(seg_order)
ax1.grid(axis="y", alpha=0.2)

# --- B) OLS num_funded_teams (bottom-left) ---
ax2 = plt.subplot(gs[1, 0])

if not ols_num_funded.empty:
    feat_rank2 = (ols_num_funded
                  .groupby(ols_map["feature"])[ols_map["coef"]]
                  .apply(lambda s: s.abs().mean())
                  .sort_values(ascending=False)
                  .head(N_SHOW)).index.tolist()

    for i, f in enumerate(feat_rank2):
        sub = ols_num_funded[ols_num_funded[ols_map["feature"]] == f].
        sub["__segidx"] = sub[ols_map["segment"]].str.lower().map({"fi
        sub = sub.sort_values("__segidx")
        ax2.plot(sub[ols_map["segment"]], sub[ols_map["coef"]],
                 marker="o", ms=MARK_SZ, lw=LINE_W,
                 color=palette[i % 10])
        ax2.errorbar(sub[ols_map["segment"]], sub[ols_map["coef"]],
                     yerr=1.96*sub[ols_map["se"]], fmt="none",
                     ecolor=palette[i % 10], alpha=ALPHA_CI)

ax2.axhline(0, color="k", lw=0.8)
ax2.set_title("OLS: num_funded_teams")
ax2.set_xlabel("temporal third")
ax2.set_ylabel("coefficient")
ax2.set_xticks(seg_order)
ax2.grid(axis="y", alpha=0.2)

# a single legend for both OLS panels, below them
leg = ax2.legend(handles, labels, loc="upper center",
                 bbox_to_anchor=(0.5, -0.22), ncol=2, fontsize=10, fra

# --- C) Logit: facilitator interactions (full right column) ---
ax3 = plt.subplot(gs[:, 1]) # span both rows

L = log.copy()
feat_col = log_map["feature"]

# top interaction terms by |coef|
top_logit = (L.groupby(feat_col)[log_map["coef"]]
             .apply(lambda s: s.abs().mean())
             .sort_values(ascending=False)
             .head(12)).index.tolist()

```

```

plot_df = L[L[feat_col].isin(top_logit)].copy()
# Put "last" above "middle" above "first" if the text includes them,
# so related terms cluster; otherwise just sort by coef for readability
plot_df = plot_df.sort_values([log_map["coef"]])

ylabels = [_nice(t) for t in plot_df[feat_col]]
ypos = np.arange(len(plot_df))

ax3.errorbar(plot_df[log_map["coef"]], ypos,
             xerr=1.96*plot_df[log_map["se"]],
             fmt="o", capsize=3, color="tab:blue")
ax3.axvline(0, color="k", lw=0.8)
ax3.set_yticks(ypos)
ax3.set_yticklabels(ylabels, fontsize=10)
ax3.set_xlabel("log-odds (coef  $\pm$ 95% CI)")
ax3.set_title("Logit: facilitator – segment interactions")
ax3.grid(axis="x", alpha=0.2)

plt.suptitle("Temporal sweep: first  $\rightarrow$  middle  $\rightarrow$  last (coef  $\pm$ 95% CI)", y
plt.tight_layout(rect=[0, 0.05, 1, 0.96]) # leave room for supertitle

plt.savefig(OUT_PNG, dpi=300, bbox_inches="tight")
plt.savefig(OUT_PDF, dpi=300, bbox_inches="tight")
print(f"[OK] Saved:\n – {OUT_PNG}\n – {OUT_PDF}")

```

```

/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_27214/867305
030.py:113: UserWarning: This figure includes Axes that are not compati
ble with tight_layout, so results might be incorrect.

```

```

plt.tight_layout(rect=[0, 0.05, 1, 0.96]) # leave room for supertitl
e + legend

```

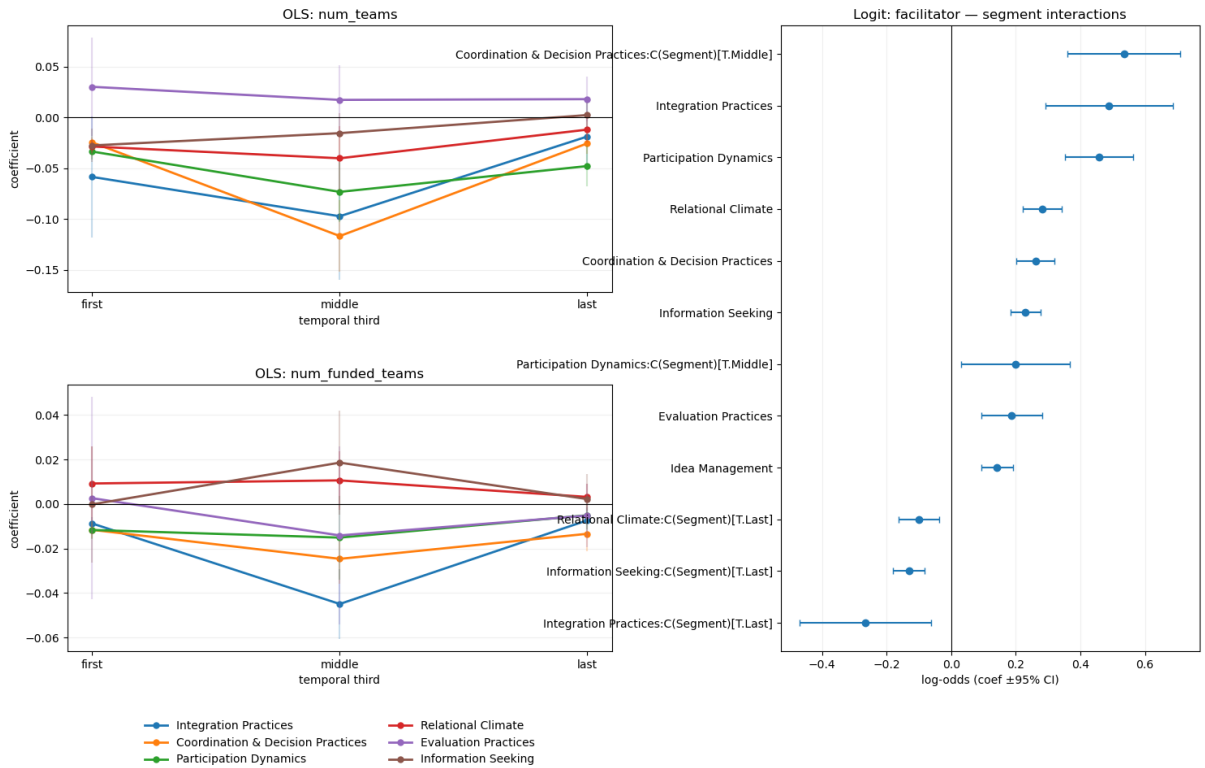
```
[OK] Saved:
```

```

– /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilit
ator-identification/3-reconsolidating-features-temporal/regression-mode
l-output/temporal_models_panel.png
– /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilit
ator-identification/3-reconsolidating-features-temporal/regression-mode
l-output/temporal_models_panel.pdf

```

Temporal sweep: first $\rightarrow$ middle $\rightarrow$ last (coef $\pm$ 95% CI)



```
In [17]: # Lane chart: temporal thirds on x; one horizontal lane per feature (+
# Output: temporal_lanes_num_teams.png (and PDF)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# ---- paths ----
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gem")
CSV = BASE / "models_ols_by_segment.csv"      # your OLS-by-segment file
OUT = BASE / "temporal_lanes_num_teams.png"  # change name for funding
OUTP = BASE / "temporal_lanes_num_teams.pdf"

# ---- settings ----
OUTCOME_KEYWORD = "num_teams"  # change to "num_funded" for the other outcome
N_SHOW = 12                   # top N features by |coef| to keep re-ranking
FIGSIZE = (12, 8)

def nice(s):
    return (str(s).replace("ann_", "")
            .replace("_score", "")
            .replace("_and_", " & ")
            .replace("_", " ")
            .title())

# ---- load & normalize ----
df = pd.read_csv(CSV)
```

```

lc = {c.lower(): c for c in df.columns}
feat = lc.get("feature", "feature")
seg = lc.get("segment", "segment")
coef = lc.get("coef", "coef")
se = lc.get("se", "se")
outc = lc.get("outcome", "outcome") if "outcome" in lc else None

if outc:
    df = df[df[outc].str.contains(OUTCOME_KEYWORD, case=False, na=False)]

# keep consistent order on thirds
order_map = {"first":0, "middle":1, "last":2}
df["__segidx"] = df[seg].str.lower().map(order_map)

# choose top features by average |coef|
top_feats = (df.groupby(feat)[coef]
              .apply(lambda s: s.abs().mean())
              .sort_values(ascending=False)
              .head(N_SHOW).index.tolist())
plot = df[df[feat].isin(top_feats)].copy()

# sort features by their peak magnitude (nice for ordering lanes)
feat_order = (plot.groupby(feat)[coef]
              .apply(lambda s: s.abs().max())
              .sort_values(ascending=True) # small→large so
              .index.tolist())

# ---- plot ----
plt.figure(figsize=FIGSIZE)
ax = plt.gca()

# one lane per feature (y = 0..N-1)
ypos = {f:i for i, f in enumerate(feat_order)}
yticks = []
ytlabs = []

colors = plt.cm.tab20.colors
for i, f in enumerate(feat_order):
    sub = plot[plot[feat]==f].sort_values("__segidx")
    y = np.full(len(sub), ypos[f], dtype=float)

    # thin baseline lane
    ax.hlines(y=y[0], xmin=-0.1, xmax=2.1, color="#999", lw=0.5, alpha=0.5)

    # connect thirds
    ax.plot(sub["__segidx"], y, color=colors[i%20], lw=1.5, alpha=0.8)

    # dots at each third with CI whiskers in the vertical (value) direction
    # In a lane chart we show the **coefficient as x-offset** is tempting
    # but your sketch fixes x = thirds. So we annotate effect size by
    # **dot size + text** next to each point (clean & readable).
    sizes = (sub[coef].abs() / (plot[coef].abs().max() + 1e-9) * 200)

```

```

ax.scatter(sub["__segidx"], y, s=sizes, color=colors[i%20], edgeco

# small text label of coef under/above the dot
for (xi, yi, c, s_err) in zip(sub["__segidx"], y, sub[coef], sub[s
    ax.text(xi, yi + 0.18, f"{c:+.3f}", ha="center", va="bottom",

yticks.append(y[0])
ytlabs.append(nice(f))

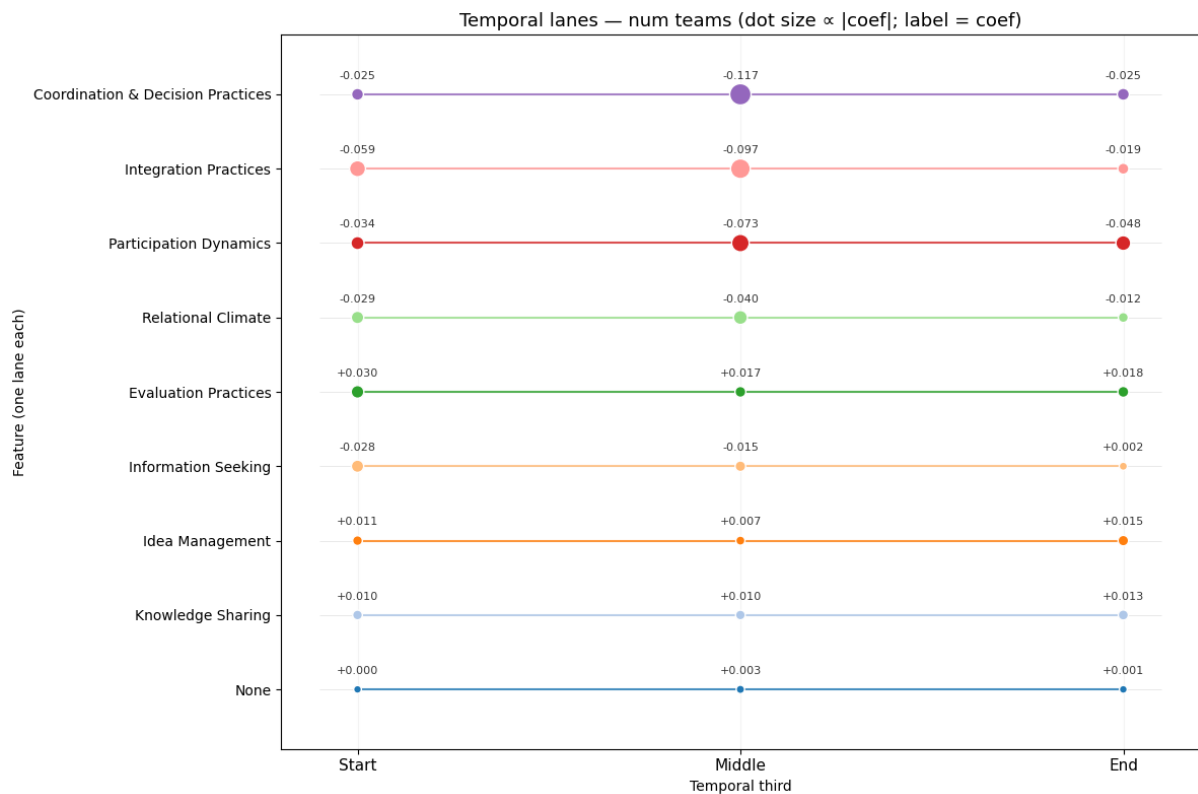
# axes cosmetics
ax.set_yticks(yticks)
ax.set_yticklabels(ytlabs, fontsize=10)
ax.set_xticks([0,1,2])
ax.set_xticklabels(["Start","Middle","End"], fontsize=11)
ax.set_xlim(-0.2, 2.2)
ax.set_ylim(-0.8, len(feat_order)-0.2)
ax.grid(axis="x", alpha=0.15)
ax.set_title(f"Temporal lanes - {OUTCOME_KEYWORD.replace('_', ' ')} (do
    fontsize=13)
ax.set_xlabel("Temporal third")
ax.set_ylabel("Feature (one lane each)")

plt.tight_layout()
plt.savefig(OUT, dpi=300, bbox_inches="tight")
plt.savefig(OUTP, dpi=300, bbox_inches="tight")
print(f"[OK] saved:\n- {OUT}\n- {OUTP}")

```

[OK] saved:

- /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output/temporal_lanes_num_teams.png
- /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output/temporal_lanes_num_teams.pdf



checking outcome distributions - fin

```
In [1]: # == Person-level temporal modeling pipeline (OLS, NegBin, Logit) ==
# Requirements: pandas, numpy, statsmodels, matplotlib
import pandas as pd, numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
from pathlib import Path

# ----- PATHS -----
# 1) Point this to your merged file with outcomes
DATA = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gem/
            "facilitator-identification/3-reconsolidating-features-tem
            "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.csv")

# 2) Output folder for tables & figs
OUT = DATA.parent / "regression-model-output"
OUT.mkdir(parents=True, exist_ok=True)

# ----- LOAD & PREP -----
df = pd.read_csv(DATA)

# keep only rows with a valid temporal "segment"
df = df[df["segment"].isin(["first", "middle", "last"])].copy()

# features = all consolidated annotation sums
features = sorted([c for c in df.columns if c.endswith("_score")])
```

```

# controls (person-level)
controls = ["C(conference)", "p_speaking_duration_sec"] # speaking du

# outcomes that may or may not exist in your merged CSV
has_num_teams = "num_teams" in df.columns
has_num_funded_teams = "num_funded_teams" in df.columns
has_fac = "role_facilitator" in df.columns

# basic cleaning
for col in ["num_teams", "num_funded_teams", "role_facilitator", "p_speak
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# ----- 1) Distribution checks (prints & quick numbers) -----
def dist_report(series, name):
    s = pd.to_numeric(series, errors="coerce")
    z = (s==0).mean() if s.notna().any() else np.nan
    print(f"[{name}] n={s.notna().sum()} mean={s.mean():.3f} var={s.

print("\n[Distribution checks]")
if has_num_teams: dist_report(df["num_teams"], "num_teams")
if has_num_funded_teams: dist_report(df["num_funded_teams"], "num_fund
if has_fac: print(f"[role_facilitator] value counts:\n{df

# ----- Helper: fit model & extract tidy table -----
def tidy_results(res, model_name, outcome, segment=None):
    # statsmodels .params, .bse, conf_int
    ci = res.conf_int()
    tbl = pd.DataFrame({
        "term": res.params.index,
        "coef": res.params.values,
        "se": res.bse.values,
        "p": res.pvalues.values,
        "lower": ci[0].values,
        "upper": ci[1].values
    })
    tbl["model"] = model_name
    tbl["outcome"] = outcome
    if segment is not None:
        tbl["segment"] = segment
    return tbl

# ----- 2) OLS per segment: num_teams / num_funded_teams -----
ols_rows = []

def ols_for(outcome):
    global ols_rows
    for seg in ["first", "middle", "last"]:
        sub = df[df["segment"]==seg].copy()
        # formula: outcome ~ feature1 + ... + featureK + controls
        rhs = " + ".join(features + controls)

```

```

        fml = f"{outcome} ~ {rhs}"
        m = smf.ols(fml, data=sub).fit()
        ols_rows.append(tidy_results(m, "OLS", outcome, seg))
        # (optional) print quick summary line
        print(f"[OLS] {outcome} - {seg}: n={sub.shape[0]} R2={m.rsqua

if has_num_teams:
    ols_for("num_teams")
if has_num_funded_teams:
    ols_for("num_funded_teams")

ols_df = pd.concat(ols_rows, ignore_index=True) if ols_rows else pd.Da
if not ols_df.empty:
    ols_path = OUT / "models_ols_by_segment.csv"
    ols_df.to_csv(ols_path, index=False)
    print(f"[OK] wrote {ols_path} ({ols_df.shape[0]} rows)")

# ----- 3) Negative Binomial robustness (num_funded_teams) -----
nb_df = pd.DataFrame()
if has_num_funded_teams:
    # Use all segments with segment dummies so we don't drop two-third
    rhs_nb = " + ".join(features + controls + ["C(segment)"])
    fml_nb = f"num_funded_teams ~ {rhs_nb}"
    # GLM with NegBin; let alpha be estimated via NB family in GLM
    m_nb = smf.glm(fml_nb, data=df, family=sm.families.NegativeBinomia
    nb_df = tidy_results(m_nb, "NegBin-GLM", "num_funded_teams")
    nb_path = OUT / "model_negbin_num_funded_teams.csv"
    nb_df.to_csv(nb_path, index=False)
    print(f"[OK] wrote {nb_path}")

# ----- 4) Logistic: facilitator with feature×segment interaction
logit_df = pd.DataFrame()
if has_fac:
    # build interactions with segment so effects are time-sensitive
    feat_terms = " + ".join([f"{f}*C(segment)" for f in features])
    rhs_logit = " + ".join(filter(None, [feat_terms] + controls))
    fml_logit = f"role_facilitator ~ {rhs_logit}"
    m_logit = smf.logit(fml_logit, data=df.dropna(subset=["role_facili
    logit_df = tidy_results(m_logit, "Logit", "role_facilitator")
    logit_path = OUT / "models_logit_facilitator_interactions.csv"
    logit_df.to_csv(logit_path, index=False)
    print(f"[OK] wrote {logit_path}")

# ----- 5) Plots -----
# A) Temporal sweep for OLS (coefficients ±95% CI) for each outcome
def plot_ols_sweep(ols_tbl, outcome, fname):
    subset = ols_tbl[ols_tbl["outcome"]==outcome].copy()
    if subset.empty:
        return
    # pick the top 8 by |coef| avg to avoid clutter
    top = (subset.groupby("term")["coef"].apply(lambda s: s.abs().mean
        .sort_values(ascending=False).head(8)).index

```

```

keep = subset[subset["term"].isin(top) & subset["term"].str.contain
if keep.empty: # fallback: allow controls if no features were top
    keep = subset[subset["term"].isin(top)]
# order segments
order = {"first":0,"middle":1,"last":2}
plt.figure(figsize=(8,5))
for term, grp in keep.groupby("term"):
    grp = grp.sort_values("segment", key=lambda s: s.map(order))
    plt.plot(grp["segment"], grp["coef"], marker="o", label=term.r
    plt.errorbar(grp["segment"], grp["coef"], yerr=1.96*grp["se"],
plt.axhline(0, color="k", lw=0.8)
plt.title(f"OLS temporal sweep – {outcome}")
plt.xlabel("temporal third"); plt.ylabel("coefficient (±95% CI)")
plt.legend(bbox_to_anchor=(1.02,1), loc="upper left", fontsize=8,
plt.tight_layout()
outp = OUT / fname
plt.savefig(outp, dpi=300)
plt.close()
print(f"[OK] plot saved: {outp}")

if not ols_df.empty and has_num_teams:
    plot_ols_sweep(ols_df, "num_teams", "ols_sweep_num_teams.png")
if not ols_df.empty and has_num_funded_teams:
    plot_ols_sweep(ols_df, "num_funded_teams", "ols_sweep_num_funded_t

# B) Logistic interactions forest (top 10 by |coef|)
if not logit_df.empty:
    L = logit_df.copy()
    # keep interaction terms only (feature:C(segment)[T.xxx])
    inter = L[L["term"].str.contains("C\\(segment\\)[T\\.]", regex=True)]
    if inter.empty:
        inter = L # fallback
    top_terms = (inter.groupby("term")["coef"].apply(lambda s: s.abs()
        .sort_values(ascending=False).head(10)).index
    P = inter[inter["term"].isin(top_terms)].sort_values("coef")
    y = np.arange(len(P))
    plt.figure(figsize=(6,6))
    plt.errorbar(P["coef"], y, xerr=1.96*P["se"], fmt="o", capsize=3)
    plt.axvline(0, color="k", lw=0.8)
    plt.yticks(y, [t.replace("ann_", "").replace("_score", "").replace("
    plt.xlabel("log-odds (coef ±95% CI)")
    plt.title("Logit: facilitator – top segment interactions")
    plt.tight_layout()
    figp = OUT / "logit_facilitator_interactions.png"
    plt.savefig(figp, dpi=300)
    plt.close()
    print(f"[OK] plot saved: {figp}")

print("\n[Done] Models + tables + figures written to:", OUT)

```

```

[Distribution checks]
[num_teams] n=2389 mean=0.791 var=1.453 zeros=44.8%
[num_funded_teams] n=2389 mean=0.259 var=0.494 zeros=62.8%
[role_facilitator] value counts:
role_facilitator
0    2964
1     245
Name: count, dtype: int64

[OLS] num_teams - first: n=1090 R2=0.066
[OLS] num_teams - middle: n=962 R2=0.068
[OLS] num_teams - last: n=1157 R2=0.126
[OLS] num_funded_teams - first: n=1090 R2=0.041
[OLS] num_funded_teams - middle: n=962 R2=0.060
[OLS] num_funded_teams - last: n=1157 R2=0.060
[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemin
i/facilitator-identification/3-reconsolidating-features-temporal/regres
sion-model-output/models_ols_by_segment.csv (114 rows)
[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemin
i/facilitator-identification/3-reconsolidating-features-temporal/regres
sion-model-output/model_negbin_num_funded_teams.csv
[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemin
i/facilitator-identification/3-reconsolidating-features-temporal/regres
sion-model-output/models_logit_facilitator_interactions.csv
/opt/miniconda3/envs/gem_samp/lib/python3.11/site-packages/statsmodels/
genmod/families/family.py:1367: ValueWarning: Negative binomial dispers
ion parameter alpha not set. Using default value alpha=1.0.
  warnings.warn("Negative binomial dispersion parameter alpha not "
[OK] plot saved: /Users/maxchalekson/Projects/NICO-Research/NICO_human-
gemini/facilitator-identification/3-reconsolidating-features-temporal/r
egression-model-output/ols_sweep_num_teams.png
[OK] plot saved: /Users/maxchalekson/Projects/NICO-Research/NICO_human-
gemini/facilitator-identification/3-reconsolidating-features-temporal/r
egression-model-output/ols_sweep_num_funded_teams.png
[OK] plot saved: /Users/maxchalekson/Projects/NICO-Research/NICO_human-
gemini/facilitator-identification/3-reconsolidating-features-temporal/r
egression-model-output/logit_facilitator_interactions.png

[Done] Models + tables + figures written to: /Users/maxchalekson/Projec
ts/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconso
lidating-features-temporal/regression-model-output

```

evey debrief 10/09 (text)

Maybe also check, using beginning variables as an independent variable, can we predict what a person would do in middle or end. And whether there are differences for this prediction power for people who are facilitators or in teams (and funded teams)

We need to check the possibility that what someone does in the beginning is

actually correlated to what they do later

```
In [2]: import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

# --- paths ---
FEAT = "/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/f

# --- load ---
df = pd.read_csv(FEAT)

# keep only the three thirds we care about
df = df[df["segment"].isin(["first", "middle", "last"])]

# tidy types
df["conference"] = df["conference"].astype(str)
if "role_facilitator" in df.columns:
    df["role_facilitator"] = pd.to_numeric(df["role_facilitator"], err

# feature list (scores; adjust if you switch to counts)
features = [
    "ann_knowledge_sharing_score",
    "ann_evaluation_practices_score",
    "ann_idea_management_score",
    "ann_coordination_and_decision_practices_score",
    "ann_information_seeking_score",
    "ann_relational_climate_score",
    "ann_participation_dynamics_score",
    "ann_integration_practices_score",
    # include 'ann_none_score' if you want
]

# --- pivot to person-level wide: one row per person x conference x ye
# this creates columns like ann_knowledge_sharing_score_first/middle/l
wide_feats = []
for col in features + ["p_speaking_duration_sec"]:
    tmp = df.pivot_table(index=["person_name", "conference", "year"],
                          columns="segment",
                          values=col,
                          aggfunc="sum")
    tmp.columns = [f"{col}_{c}" for c in tmp.columns]
    wide_feats.append(tmp)

wide = pd.concat(wide_feats, axis=1).reset_index()

# bring role_facilitator (if present) – take max across thirds for the
if "role_facilitator" in df.columns:
    rf = (df.groupby(["person_name", "conference", "year"], as_index=False)
          .max())
    wide = wide.merge(rf, on=["person_name", "conference", "year"], how=
```

```

# add a total speaking-time control (optional but handy)
wide["speaking_total"] = wide[[c for c in wide.columns if c.startswith

# --- run per-feature predictive regressions ---
results = [] # collect summaries per feature

for f in features:
    # ensure the columns exist after pivot
    cols_needed = [f"{f}_first", f"{f}_middle", f"{f}_last",
                    "p_speaking_duration_sec_first", "speaking_total",
    for c in cols_needed:
        if c not in wide.columns:
            wide[c] = np.nan

    # modeling dataset
    d = wide.copy()

    # simple imputations to avoid dropping too much; you can also drop
    d[[f"{f}_first", f"{f}_middle", f"{f}_last"]] = d[[f"{f}_first", f
    d["p_speaking_duration_sec_first"] = d["p_speaking_duration_sec_fi
    d["speaking_total"] = d["speaking_total"].fillna(0)

    # build formula with facilitator interaction if available
    if "role_facilitator" in d.columns:
        # middle ~ first (+ interaction) + controls
        fm_mid = (f"{f}_middle ~ {f}_first"
                  f" + role_facilitator"
                  f" + {f}_first:role_facilitator"
                  f" + p_speaking_duration_sec_first"
                  f" + C(conference)")
        m_mid = smf.ols(fm_mid, data=d).fit(cov_type="HC3")

        # last ~ first (+ interaction) + controls
        fm_last = (f"{f}_last ~ {f}_first"
                   f" + role_facilitator"
                   f" + {f}_first:role_facilitator"
                   f" + p_speaking_duration_sec_first"
                   f" + C(conference)")
        m_last = smf.ols(fm_last, data=d).fit(cov_type="HC3")

        # grab the interaction p-values if present
        p_int_mid = m_mid.pvalues.get(f"{f}_first:role_facilitator",
        p_int_last = m_last.pvalues.get(f"{f}_first:role_facilitator",

    results.append({
        "feature": f,
        "R2_middle_from_first": m_mid.rsquared,
        "R2_last_from_first": m_last.rsquared,
        "p_interaction_middle": p_int_mid,
        "p_interaction_last": p_int_last,
        "n": int(m_mid.nobs)
    })

```

```

else:
    # no facilitator column -> run without the interaction
    fm_mid = f"{f}_middle ~ {f}_first + p_speaking_duration_sec_f
    fm_last = f"{f}_last ~ {f}_first + p_speaking_duration_sec_f
    m_mid = smf.ols(fm_mid, data=d).fit(cov_type="HC3")
    m_last = smf.ols(fm_last, data=d).fit(cov_type="HC3")
    results.append({
        "feature": f,
        "R2_middle_from_first": m_mid.rsquared,
        "R2_last_from_first": m_last.rsquared,
        "p_interaction_middle": np.nan,
        "p_interaction_last": np.nan,
        "n": int(m_mid.nobs)
    })

res_df = pd.DataFrame(results).sort_values("R2_middle_from_first", ascending=False)
print("\n[Predictive power: middle/last from first] (higher R² = stronger carryover)")
print(res_df.to_string(index=False, float_format=lambda x: f"{x:.3f}")

# Optional: save it next to your outputs
out_csv = "/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output/predict_middle_last_from_first.csv"
res_df.to_csv(out_csv, index=False)
print(f"\n[OK] wrote {out_csv}")

```

[Predictive power: middle/last from first] (higher R² = stronger carryover)

	feature	R2_middle_from_first	R2_last_from_first	p_interaction_middle	p_interaction_last	n
0.589	ann_participation_dynamics_score	0.544	0.157	1484		0.590
0.601	ann_knowledge_sharing_score	0.381	0.447	1484		0.528
0.551	ann_idea_management_score	0.470	0.145	1484		0.517
0.548	ann_information_seeking_score	0.731	0.638	1484		0.492
0.627	ann_relational_climate_score	0.032	0.000	1484		0.456
0.659	ann_coordination_and_decision_practices_score	0.471	0.000	1484		0.455
0.458	ann_evaluation_practices_score	0.420	0.145	1484		0.395
0.342	ann_integration_practices_score	0.204	0.447	1484		0.369

[OK] wrote /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output/predict_middle_last_from_first.csv

setting up the slides 10/09/2025

In [5]:

```
# =====  
# SLIDE FIGURE FACTORY – person-level temporal analysis (polished+comp  
# =====  
  
import warnings, textwrap, re  
from pathlib import Path  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import statsmodels.api as sm  
import statsmodels.formula.api as smf  
  
warnings.filterwarnings("ignore")  
sns.set_context("talk")  
  
# ----- PATHS -----  
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-g  
FEAT = BASE / "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.cs  
FIGDIR = BASE / "figures"  
REGDIR = BASE / "regression-model-output"  
FIGDIR.mkdir(parents=True, exist_ok=True)  
REGDIR.mkdir(parents=True, exist_ok=True)  
  
# ----- CONFIG -----  
SEG_ORDER = ["first", "middle", "last"]  
PERSON_SCOPE = "across_all" # "per_year" or "across_all"  
DESC_AGG = "sum" # "sum" or "mean"  
ALPHA_P = 0.05  
MAX_PANELS_PER_PNG = 6  
  
REG_FEATURES = [  
    "ann_knowledge_sharing_score",  
    "ann_evaluation_practices_score",  
    "ann_idea_management_score",  
    "ann_coordination_and_decision_practices_score",  
    "ann_information_seeking_score",  
    "ann_relational_climate_score",  
    "ann_participation_dynamics_score",  
    "ann_integration_practices_score",  
]  
  
ALIASES = {  
    "ann_knowledge_sharing": "Knowledge sharing",  
    "ann_evaluation_practices": "Evaluation practices",  
    "ann_idea_management": "Idea management",  
    "ann_coordination_and_decision_practices": "Coordination & decisio  
    "ann_information_seeking": "Information seeking",  
    "ann_relational_climate": "Relational climate",  
    "ann_participation_dynamics": "Participation dynamics",  
    "ann_integration_practices": "Integration practices",  
    "ann_none": "None",
```

```

}

def nice(col: str) -> str:
    base = col.replace("ann_", "").replace("_score", "").replace("_count", "")
    return ALIASES.get(f"ann_{base}", base.replace("_", " ").title())

def wrap(s, w=46):
    import textwrap as tw
    return "\n".join(tw.wrap(str(s), width=w))

# ----- LOAD -----
df = pd.read_csv(FEAT)
df = df[df["segment"].isin(SEG_ORDER)].copy()
df["segment"] = pd.Categorical(df["segment"], categories=SEG_ORDER, ordered=True)
df["conference"] = df["conference"].astype(str)

# If a stable person_id exists, use it; else fall back to person_name
PERSON_ID_COL = "person_id" if "person_id" in df.columns else "person_name"
df[PERSON_ID_COL] = df[PERSON_ID_COL].astype(str)

has_fac = "role_facilitator" in df.columns
has_teams = "num_teams" in df.columns
has_funded = "num_funded_teams" in df.columns

if has_fac:
    df["role_facilitator"] = pd.to_numeric(df["role_facilitator"], errors="coerce")

# handy group flags
df["any_team"] = (df.get("num_teams", 0).fillna(0) > 0).astype(int)
df["any_funded"] = (df.get("num_funded_teams", 0).fillna(0) > 0).astype(int)

# prefer counts for descriptives if available
CANDIDATE_COUNTS = [f.replace("_score", "_count") for f in REG_FEATURES]
DESC_FEATURES = [c for c in CANDIDATE_COUNTS if c in df.columns]
if not DESC_FEATURES:
    DESC_FEATURES = REG_FEATURES[:] # fallback to scores

# ----- PERSON KEYS -----
# per_year => person×conference×year; across_all => person only
PERSON_KEYS = [PERSON_ID_COL] if PERSON_SCOPE == "across_all" else [PERSON_ID_COL, "conference"]

# ----- helper: add C(conference) only if present -----
def _conf_term(df_like):
    return " + C(conference)" if "conference" in df_like.columns else ""

# ----- Helper: WIDE table used by prediction models -----
def to_wide(dfin: pd.DataFrame) -> pd.DataFrame:
    cols = REG_FEATURES + ["p_speaking_duration_sec"]
    blocks = []
    for col in cols:
        pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
        pt.columns = [f"{col}_{c}" for c in pt.columns]

```

```

        blocks.append(pt)
    wide = pd.concat(blocks, axis=1).reset_index()

    if has_fac:
        rf = dfin.groupby(PERSON_KEYS, as_index=False)["role_facilitat
        wide = wide.merge(rf, on=PERSON_KEYS, how="left")
        wide["role_facilitator"] = wide["role_facilitator"].fillna(0).

    for flag, col in [("any_team", "num_teams"), ("any_funded", "num_fun
        if col in dfin.columns:
            tmp = dfin.groupby(PERSON_KEYS, as_index=False)[col].sum()
            tmp[flag] = (tmp[col] > 0).astype(int)
            wide = wide.merge(tmp[PERSON_KEYS + [flag]], on=PERSON_KEY
        else:
            wide[flag] = 0

    speak_cols = [c for c in wide.columns if c.startswith("p_speaking_
    wide["speaking_total"] = wide[speak_cols].sum(axis=1).fillna(0) if
    wide["speaking_first"] = wide.get("p_speaking_duration_sec_first",
    return wide

wide = to_wide(df)

# ----- SLIDE A: outcome distributions -----
def make_outcome_distributions():
    if not (has_teams or has_funded): return
    fig, axes = plt.subplots(1, 2 if has_teams and has_funded else 1,
    if not isinstance(axes, np.ndarray): axes = np.array([axes])

    def one(ax, col):
        if col not in df.columns:
            ax.axis("off"); return
        z = (df[col].fillna(0) == 0).mean()
        ax.hist(df[col].fillna(0), bins=range(int(df[col].max()+2))
        ax.set_title(f"{col} (zeros={z:.1%})")
        ax.set_xlabel(col); ax.set_ylabel("Count")

    if has_teams: one(axes[0], "num_teams")
    if has_funded: one(axes[1 if has_teams else 0], "num_funded_teams")

    fig.suptitle("Outcome distributions (check for zero-inflation)")
    fig.tight_layout(); fig.savefig(FIGDIR/"outcome_distributions.png")

# ----- SLIDE 1: schematic -----
def make_schematic():
    fig, ax = plt.subplots(figsize=(11, 3.2))
    ax.axis("off"); ax.set_xlim(0, 10); ax.set_ylim(0, 3)
    ax.hlines(1.5, 0.7, 9.3, linewidth=2, color="black")
    for x, lbl in zip([2.5, 5.0, 7.5], SEG_ORDER):
        ax.vlines(x, 1.2, 1.8, color="gray", linestyle="--", linewidth
        ax.text(x, 2.0, lbl, ha="center", va="bottom", fontsize=12)
    rng = np.random.default_rng(7)

```

```

for x in sorted(rng.uniform(1.0,9.0,18)):
    ax.scatter([x],[1.5 + rng.normal(0,0.08)], s=60, ec="black", f
ax.text(0.7, 2.55, "Person-level aggregation across sessions", fon
scope_note = "across all conferences when selected" if PERSON_SCOP
ax.text(0.7, 0.35, wrap(
    f"Each dot = utterance with code scores → aggregated per perso
    f"(first/middle/last), then summed within session and summed a
), fontsize=10)
fig.tight_layout(); fig.savefig(FIGDIR/"schematic_person_aggregati

# ----- SLIDE 2: features glossary -----
def make_features_glossary():
    lines = [f"• {nice(f)} – sum of per-utterance scores per person×th
        for f in REG_FEATURES]
    fig, ax = plt.subplots(figsize=(8.6, 6.0))
    ax.axis("off")
    ax.text(0.02, 0.98, "Features (person-level, temporal):\n\n" + "\n
        ha="left", va="top", fontsize=12)
    fig.savefig(FIGDIR/"features_glossary.png", dpi=300, bbox_inches="

# ----- SLIDE 3: temporal correlations -----
def make_temporal_correlation_grid():
    use_feats = DESC_FEATURES # counts if present, else scores

    W_blocks = []
    for col in use_feats:
        pt = df.pivot_table(index=PERSON_KEYS, columns="segment", valu
        pt.columns = [f"{col}_{c}" for c in pt.columns]
        W_blocks.append(pt)
    W = pd.concat(W_blocks, axis=1)

    pages = [use_feats[i:i+MAX_PANELS_PER_PNG] for i in range(0, len(u
    tag = "counts" if use_feats != REG_FEATURES else "scores"

    for pi, feats in enumerate(pages, start=1):
        rows, cols = 2, 3
        fig, axes = plt.subplots(rows, cols, figsize=(12, 7), constrain
        axes = np.asarray(axes).ravel()

        for ax, f in zip(axes, feats):
            sub = W[[f"{f}_first", f"{f}_middle", f"{f}_last"]].fillna
            mat = sub.corr()
            mat.index = ["first","middle","last"]; mat.columns = ["fir
            sns.heatmap(mat, annot=True, cmap="Blues", cbar=False, ax=
                annot_kws={"fontsize":9})
            ax.set_xlabel(""); ax.set_ylabel(""); ax.set_title(nice(f)

        for j in range(len(feats), len(axes)): axes[j].axis("off")

    fig.suptitle(f"Within-person temporal correlations (first/midd
    out = FIGDIR / (f"temporal_correlation_grid_p{pi}.png" if len(
        else "temporal_correlation_grid.png")

```

```

fig.savefig(out, dpi=180, bbox_inches="tight", pil_kwargs={"co
plt.close(fig)

# ----- SLIDE 3b: descriptive volume by third -----
def make_descriptive_volume_panel():
    feats = DESC_FEATURES
    tmp = df.copy()
    tmp["segment"] = pd.Categorical(tmp["segment"], categories=SEG_ORD

    if DESC_AGG == "sum":
        g = (tmp[["segment"] + feats]
            .groupby("segment", as_index=False)
            .sum(numeric_only=True))
        ylab = "Sum per person-scope"
    else:
        g = (tmp[["segment"] + feats]
            .groupby("segment", as_index=False)
            .mean(numeric_only=True))
        ylab = "Mean per person-scope"

    means = g.melt(id_vars="segment", var_name="feature", value_name="
    means["feature_nice"] = means["feature"].map(nice)

    plt.figure(figsize=(11.5, 6))
    sns.lineplot(data=means, x="segment", y="value", hue="feature_nice
    title_tag = "counts" if "count" in feats[0] else "scores"
    plt.title(f"Descriptive volume by temporal third - {title_tag}")
    plt.xlabel("Temporal third"); plt.ylabel(ylab)
    plt.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), frameon=
    plt.tight_layout(); plt.savefig(FIGDIR/"descriptive_volume_by_thir

# ----- SLIDE 4/5: OLS sweeps -----
def fit_ols_per_segment(y_col: str, out_png: str):
    rows = []
    for seg in SEG_ORDER:
        keep = PERSON_KEYS + ["segment", y_col, "p_speaking_duration_s
        dseg = df[df["segment"]==seg][keep].dropna(subset=[y_col]).cop
        if dseg.empty:
            continue
        conf = _conf_term(dseg)
        fml = f"{y_col} ~ " + " + ".join(REG_FEATURES) + " + p_speakin

    # cluster-robust by person, fallback HC3
    cluster = dseg[PERSON_ID_COL] if PERSON_ID_COL in dseg.columns
    if cluster is not None:
        m = smf.ols(fml, data=dseg).fit(cov_type="cluster", cov_kw
    else:
        m = smf.ols(fml, data=dseg).fit(cov_type="HC3")

    for feat in REG_FEATURES:
        if feat in m.params.index:
            rows.append({

```

```

        "outcome": y_col, "segment": seg, "feature": feat,
        "coef": float(m.params[feat]), "se": float(m.bse[f
        "p": float(m.pvalues[feat]), "r2": float(m.rsquare
    })

    if not rows:
        return pd.DataFrame()

    tab = pd.DataFrame(rows)
    tab.to_csv(REGDIR / f"models_ols_by_segment_{y_col}.csv", index=False)

    fig, ax = plt.subplots(figsize=(10.8, 6))
    top = (tab.groupby("feature")["coef"].apply(lambda s: s.abs().mean
        .sort_values(ascending=False).head(8)).index.tolist())

    for feat in top:
        sub = tab[tab["feature"]==feat].copy()
        sub["seg_idx"] = sub["segment"].map({k:i for i,k in enumerate(
        sub = sub.sort_values("seg_idx")
        ax.plot(sub["segment"], sub["coef"], linewidth=1.6, label=nice
        for _, r in sub.iterrows():
            sig = r["p"] < ALPHA_P
            ax.errorbar([r["segment"]], [r["coef"]], yerr=[1.96*r["se"
                fmt="o", mec="black", mfc=("C0" if sig else "w
                mew=1, capsize=0, alpha=1.0 if sig else 0.9)

        ax.axhline(0, color="black", lw=1)
        ax.set_title(f"OLS sweep by temporal third - {y_col}\n(Plot shows
        ax.set_xlabel("Temporal third"); ax.set_ylabel("Coefficient (±95%
        ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), fontsize=
        fig.tight_layout(); fig.savefig(FIGDIR / out_png, dpi=300, bbox_in
    return tab

# ----- SLIDE 5b: Negative binomial for funded counts -----
def fit_negbin_num_funded():
    if not has_funded: return
    d = df.dropna(subset=["num_funded_teams"]).copy()
    conf = _conf_term(d)
    fml = (
        "num_funded_teams ~ "
        + " + ".join(REG_FEATURES)
        + " + p_speaking_duration_sec"
        + conf
        + " + C(segment)"
    )
    m = smf.glm(fml, data=d, family=sm.families.NegativeBinomial()).fi
    pd.DataFrame({"term": m.params.index, "coef": m.params.values,
        "se": m.bse.values, "p": m.pvalues.values}) \
        .to_csv(REGDIR/"model_negbin_num_funded_teams.csv", index=False)

# ----- SLIDE 6: Logistic (facilitator) -----
def pretty_term(t: str) -> str:

```

```

m = re.match(r'^(ann_.+?)_score:C\\(segment\\)\\[T\\. (first|middle|last)')
if m: return f"{nice(m.group(1))} × {m.group(2)}"
if t.startswith("C(segment)[T."):
    return t.replace("C(segment)[T.", "segment = ").replace("]", "")
return t

def fit_logit_facilitator():
    if not has_fac: return pd.DataFrame()
    d = df.dropna(subset=["role_facilitator"]).copy()
    conf = _conf_term(d)
    fml = (
        "role_facilitator ~ ("
        + " + ".join(REG_FEATURES)
        + ") * C(segment) + p_speaking_duration_sec"
        + conf
    )
    m = smf.logit(fml, data=d).fit(dispen=0)
    tab = pd.DataFrame({"term": m.params.index, "coef": m.params.value
                        "se": m.bse.values, "p": m.pvalues.values})
    tab.to_csv(REGDIR / "models_logit_facilitator_interactions.csv", index=False)

    take = tab[tab["term"].str.contains(r"C\\(segment\\)\\[T\\. ", regex=True)]
    if take.empty:
        print("[LOGIT] No segment-interaction terms found (skipping plot)")
    take["abscoef"] = take["coef"].abs()
    take = take.sort_values("abscoef", ascending=False).head(12)
    take["pretty"] = take["term"].apply(pretty_term)

    fig, ax = plt.subplots(figsize=(10.5, 6))
    y = np.arange(len(take))[:-1]
    ax.errorbar(take["coef"], y, xerr=1.96*take["se"], fmt="o", capsize=5)
    ax.axvline(0, color="black", lw=1)
    ax.set_yticks(y); ax.set_yticklabels([wrap(t) for t in take["pretty"]])
    ax.set_xlabel("Log-odds (coef ±95% CI)")
    ax.set_title("Logistic regression – facilitator × segment interactions")
    fig.tight_layout(); fig.savefig(FIGDIR/"logit_facilitator_interactions.png")
    return tab

# ----- SLIDE 7: Begin→middle/last prediction + group comparisons
def temporal_prediction_from_first():
    rows = []
    groups = {
        "all": slice(None),
        "facilitator": (wide.get("role_facilitator", 0) == 1),
        "non_facilitator": (wide.get("role_facilitator", 0) == 0),
        "any_team": (wide.get("any_team", 0) == 1),
        "no_team": (wide.get("any_team", 0) == 0),
        "any_funded": (wide.get("any_funded", 0) == 1),
        "no_funded": (wide.get("any_funded", 0) == 0),
    }
    for gname, gmask in groups.items():
        dG = wide[gmask].copy() if isinstance(gmask, pd.Series) else wide

```

```

for f in REG_FEATURES:
    for col in [f"{f}_first", f"{f}_middle", f"{f}_last"]:
        if col not in dG.columns: dG[col] = 0.0
    dG["speaking_first"] = pd.to_numeric(dG["speaking_first"],
    fm_mid = f"{f}_middle ~ {f}_first + speaking_first"
    fm_last = f"{f}_last ~ {f}_first + speaking_first"
    m_mid = smf.ols(fm_mid, data=dG).fit(cov_type="HC3")
    m_last = smf.ols(fm_last, data=dG).fit(cov_type="HC3")
    rows.append({"group": gname, "feature": f,
                "R2_middle_from_first": m_mid.rsquared,
                "R2_last_from_first": m_last.rsquared,
                "n": int(m_mid.nobs)})
res = pd.DataFrame(rows)
res.to_csv(REGDIR/"predict_middle_last_from_first_by_group.csv", i

# Main bar (ALL)
base = res[res["group"]=="all"].copy().sort_values("R2_last_from_f
fig, ax = plt.subplots(figsize=(10.5, 6))
plot = base.copy(); plot["Feature"] = plot["feature"].map(nice)
plot = plot.set_index("Feature")[["R2_middle_from_first", "R2_last_
plot.plot(kind="bar", ax=ax)
ax.set_ylim(0, 1); ax.set_ylabel("R²")
scope_txt = "across-all person scope" if PERSON_SCOPE=="across_all
ax.set_title(f"Temporal prediction: beginning → middle/last (perso
plt.xticks(rotation=30, ha="right"); fig.tight_layout()
fig.savefig(FIGDIR/"temporal_prediction_bar.png", dpi=300, bbox_in

# Comparison helpers
def comp_plot(groups_pair, fname, title_suffix):
    comp = res[res["group"].isin(groups_pair)].copy()
    if comp.empty: return
    comp["Feature"] = comp["feature"].map(nice)
    fig, ax = plt.subplots(figsize=(11.5, 5.2))
    sns.barplot(data=comp, x="Feature", y="R2_last_from_first", hu
    ax.set_ylim(0, 1); ax.set_ylabel("R² (last ~ first)"); ax.set_
    ax.set_title(f"Prediction power comparison: {title_suffix}")
    plt.xticks(rotation=30, ha="right"); plt.tight_layout()
    plt.savefig(FIGDIR/fname, dpi=300, bbox_inches="tight"); plt.c

comp_plot(["facilitator", "non_facilitator"], "temporal_prediction_
    "facilitators vs non-facilitators")
comp_plot(["any_team", "no_team"], "temporal_prediction_anyteam_vs_
    "any team vs no team")
comp_plot(["any_funded", "no_funded"], "temporal_prediction_anyfund
    "any funded vs none")

# ----- SLIDE 9: findings card -----
def make_findings_card():
    lines = [
        "Findings (person-level, temporal thirds)",
        "",
        "• Early behaviors predict later behaviors strongly (R²≈0.4–0.

```

```

    "• OLS sweeps (MODEL COEFFICIENTS ±95% CI) show how timing rel
    " Descriptive panel shows raw volume by third (counts if avai
    "• Funded teams: Relational climate & Information seeking stan
    "• Facilitators: timing interactions appear for climate/coordi
    "• In OLS sweeps, filled markers denote p<.05 (hollow = n.s.).
    f"• Prediction scope shown here: {PERSON_SCOPE} (toggle in cod
]
fig, ax = plt.subplots(figsize=(10.8, 5.2)); ax.axis("off")
ax.text(0.02, 0.95, "\n".join(lines), ha="left", va="top", fontsiz
fig.savefig(FIGDIR/"findings_text_card.png", dpi=300, bbox_inches=

# ----- RUN ALL -----
print("[A] outcomes"); make_outcome_distributio
print("[1] schematic"); make_schematic()
print("[2] glossary"); make_features_glossary()
print("[3] correlations"); make_temporal_correlatio
print("[3b] descriptive volume"); make_descriptive_volume_
print("[4] OLS num_teams"); fit_ols_per_segment("num
print("[5] OLS num_funded"); fit_ols_per_segment("num
print("[5b] NegBin funded"); fit_negbin_num_funded()
print("[6] Logit facilitator"); fit_logit_facilitator()
print("[7] Pred begin→mid/last + comps"); temporal_prediction_from
print("[9] Findings card"); make_findings_card()
print("\n[Done] Figures:", FIGDIR); print("[Done] Tables:", REGDIR)

```

```

[A] outcomes
[1] schematic
[2] glossary
[3] correlations
[3b] descriptive volume
[4] OLS num_teams
[5] OLS num_funded
[5b] NegBin funded
[6] Logit facilitator
[7] Pred begin→mid/last + comps
[9] Findings card

```

```

[Done] Figures: /Users/maxchalekson/Projects/NICO-Research/NICO_human-g
emini/facilitator-identification/3-reconsolidating-features-temporal/fi
gures

```

```

[Done] Tables: /Users/maxchalekson/Projects/NICO-Research/NICO_human-ge
mini/facilitator-identification/3-reconsolidating-features-temporal/reg
ression-model-output

```

testing whether early behavior (first 1/3) predict later outcome roles (facilitator, team, funded)

```

In [12]: # =====
# EARLY → ROLE PREDICTION (first-third behaviors predicting roles)
# =====

```

```

# ---- Imports (safe to re-import) ----
from typing import List, Tuple
from pathlib import Path # ensure Path is defined before use
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# ---- Load data (self-contained) ----
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-g
FEAT = BASE / "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.cs
df = pd.read_csv(FEAT)
df = df[df["segment"].isin(["first", "middle", "last"])] .copy()
df["segment"] = pd.Categorical(df["segment"], categories=["first", "mid
df["conference"] = df["conference"].astype(str)
assert 'df' in globals(), "Please run your data loading cell first to

# ---- Fallbacks for globals (only used if your pipeline didn't set th
if 'REG_FEATURES' not in globals():
    REG_FEATURES = [
        "ann_knowledge_sharing_score",
        "ann_evaluation_practices_score",
        "ann_idea_management_score",
        "ann_coordination_and_decision_practices_score",
        "ann_information_seeking_score",
        "ann_relational_climate_score",
        "ann_participation_dynamics_score",
        "ann_integration_practices_score",
    ]

if 'ALIASES' not in globals():
    ALIASES = {
        "ann_knowledge_sharing": "Knowledge sharing",
        "ann_evaluation_practices": "Evaluation practices",
        "ann_idea_management": "Idea management",
        "ann_coordination_and_decision_practices": "Coordination & dec
        "ann_information_seeking": "Information seeking",
        "ann_relational_climate": "Relational climate",
        "ann_participation_dynamics": "Participation dynamics",
        "ann_integration_practices": "Integration practices",
        "ann_none": "None",
    }

def _nice_first_label(term: str) -> str:
    if term == "speaking_first":
        return "Speaking time (first)"
    if term.endswith("_first"):
        base = term.replace("_first", "").replace("_score", "").replace(
            human = ALIASES.get("ann_" + base, base.replace("_", " ").titl
        return f"{human} (first)"
    return term

```

```

# PERSON_KEYS: use your pipeline's if present; else per-year default
if 'PERSON_KEYS' not in globals():
    PERSON_KEYS = ["person_id" if "person_id" in df.columns else "pers

# Output directories (create if missing)
if 'FIGDIR' not in globals():
    FIGDIR = Path("./figures"); FIGDIR.mkdir(parents=True, exist_ok=Tr
if 'REGDIR' not in globals():
    REGDIR = Path("./regression-model-output"); REGDIR.mkdir(parents=T

# ---- Wide builder (with observed=False to silence pandas FutureWarni
if 'to_wide' not in globals():
    def to_wide(dfin: pd.DataFrame) -> pd.DataFrame:
        cols = REG_FEATURES + ["p_speaking_duration_sec"]
        blocks = []
        for col in cols:
            pt = dfin.pivot_table(
                index=PERSON_KEYS, columns="segment", values=col,
                aggfunc="sum", observed=False # <-- silence FutureWa
            )
            pt.columns = [f"{col}_{c}" for c in pt.columns]
            blocks.append(pt)
        wide_ = pd.concat(blocks, axis=1).reset_index()

        # speaking_first control
        wide_["speaking_first"] = wide_.get(
            "p_speaking_duration_sec_first", pd.Series(0, index=wide_.
        ).fillna(0)

        # role + group flags if present in df
        if "role_facilitator" in dfin.columns:
            rf = dfin.groupby(PERSON_KEYS, as_index=False)["role_facil
            wide_ = wide_.merge(rf, on=PERSON_KEYS, how="left")
            wide_["role_facilitator"] = pd.to_numeric(
                wide_["role_facilitator"], errors="coerce"
            ).fillna(0).astype(int)

        for flag, col in [("any_team", "num_teams"), ("any_funded", "num
            if col in dfin.columns:
                tmp = dfin.groupby(PERSON_KEYS, as_index=False)[col].s
                tmp[flag] = (tmp[col] > 0).astype(int)
                wide_ = wide_.merge(tmp[PERSON_KEYS + [flag]], on=PERS
            else:
                wide_[flag] = 0

        return wide_

if 'wide' not in globals():
    wide = to_wide(df)

# include C(conference) only if conference is part of PERSON_KEYS (per
def _has_conference_fe_in_scope() -> bool:

```

```

    return "conference" in PERSON_KEYS

def _first_cols(feats: List[str]) -> List[str]:
    return [f"{f}_first" for f in feats]

# ----- Prep first→role dataset -----
def _build_first_df() -> pd.DataFrame:
    keep = []
    for y in ["role_facilitator", "any_team", "any_funded"]:
        if y in wide.columns:
            keep.append(y)
    keep += [c for c in _first_cols(REG_FEATURES) if c in wide.columns]
    if "speaking_first" in wide.columns:
        keep.append("speaking_first")
    keep += [c for c in PERSON_KEYS if c in wide.columns]

    dF = wide[keep].copy()
    # outcomes to int
    for y in ["role_facilitator", "any_team", "any_funded"]:
        if y in dF.columns:
            dF[y] = pd.to_numeric(dF[y], errors="coerce").fillna(0).astype(int)
    # predictors numeric
    for c in _first_cols(REG_FEATURES) + ["speaking_first"]:
        if c in dF.columns:
            dF[c] = pd.to_numeric(dF[c], errors="coerce").fillna(0.0)
    return dF

# ----- Fit one logistic model -----
def _fit_logit_first2role(dfR: pd.DataFrame, outcome: str):
    X_cols = [c for c in _first_cols(REG_FEATURES) if c in dfR.columns]
    if (outcome not in dfR.columns) or (len(X_cols) == 0):
        print(f"[first→role] Skipping {outcome}: predictors or outcome missing")
        return pd.DataFrame(), None

    rhs_terms = X_cols + (["speaking_first"] if "speaking_first" in dfR.columns else [])
    rhs = " + ".join(rhs_terms)
    if _has_conference_fe_in_scope():
        rhs += " + C(conference)"

    fml = f"{outcome} ~ {rhs}"
    m = smf.logit(fml, data=dfR).fit(dispenv=0, maxiter=200)

    tab = pd.DataFrame({
        "term": m.params.index,
        "coef": m.params.values,
        "se": m.bse.values,
        "p": m.pvalues.values
    })
    # odds ratios + 95% CI
    tab["or"] = np.exp(tab["coef"])
    tab["lo95"] = np.exp(tab["coef"] - 1.96*tab["se"])
    tab["hi95"] = np.exp(tab["coef"] + 1.96*tab["se"])

```

```

tab.to_csv(REGDIR / f"logit_first2role_{outcome}.csv", index=False)
return tab, m

# ----- Plot (odds ratios) -----
def _plot_logit_first2role(tab: pd.DataFrame, outcome: str):
    if tab is None or tab.empty:
        return
    keep_mask = tab["term"].str.endswith("_first") | (tab["term"] == "any_funded")
    take = tab[keep_mask].copy()
    if take.empty:
        print(f"[first→role] No first-third rows to plot for {outcome}")
        return
    take = take.sort_values("coef", key=lambda s: s.abs(), ascending=False)
    take = take.sort_values("or")

    y = np.arange(len(take))
    fig, ax = plt.subplots(figsize=(10.8, 6))
    ax.errorbar(
        take["or"], y,
        xerr=[take["or"] - take["lo95"], take["hi95"] - take["or"]],
        fmt="o", capsize=3
    )
    ax.axvline(1.0, color="black", lw=1) # OR=1 baseline
    ax.set_yticks(y)
    ax.set_yticklabels([_nice_first_label(t) for t in take["term"]])
    title_map = {"role_facilitator": "Facilitator (1/0)", "any_team": "Any team", "any_funded": "Any funded"}
    ax.set_xlabel("Odds ratio (first-third predictors, ±95% CI)")
    ax.set_title(f"Beginning (first-third) behaviors → {title_map.get(outcome, outcome)}")
    fig.tight_layout()
    fig.savefig(FIGDIR / f"logit_first2role_{outcome}.png", dpi=300, bbox_inches="tight")
    plt.close(fig)

# ----- Runner -----
def run_first_to_role_models():
    dfR = _build_first_df()
    outcomes = [y for y in ["role_facilitator", "any_team", "any_funded"]]
    for y in outcomes:
        print(f"[8] First→role logistic for: {y}")
        tab, m = _fit_logit_first2role(dfR, y)
        if tab is not None and not tab.empty:
            _plot_logit_first2role(tab, y)
            # optional: save text summary
            try:
                with open(REGDIR / f"logit_first2role_{y}_summary.txt", "w") as f:
                    f.write(m.summary2().as_text())
            except Exception:
                pass

print("[8] First-third behaviors → roles (logit)")
run_first_to_role_models()

```

```
[8] First-third behaviors → roles (logit)
[8] First→role logistic for: role_facilitator
[8] First→role logistic for: any_team
[8] First→role logistic for: any_funded
```

testing whether middle behavior (second 1/3) predict later outcome roles (facilitator, team, funded)

```
In [2]: # =====
# MIDDLE → ROLE PREDICTION (middle-third behaviors predicting roles)
# Self-contained: builds any missing globals (df, wide, etc.) and runs
# Outputs PNG+CSV for: role_facilitator, any_team, any_funded
# =====

# ---- Imports ----
from typing import List, Tuple
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# ---- Ensure data + config exist (light defaults if needed) ----
if 'df' not in globals():
    BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human")
    FEAT = BASE / "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES."
    df = pd.read_csv(FEAT)
    df = df[df["segment"].isin(["first", "middle", "last"])]
    df["segment"] = pd.Categorical(df["segment"], categories=["first",
    if "conference" in df.columns:
        df["conference"] = df["conference"].astype(str)

# Feature list fallback
if 'REG_FEATURES' not in globals():
    REG_FEATURES = [
        "ann_knowledge_sharing_score",
        "ann_evaluation_practices_score",
        "ann_idea_management_score",
        "ann_coordination_and_decision_practices_score",
        "ann_information_seeking_score",
        "ann_relational_climate_score",
        "ann_participation_dynamics_score",
        "ann_integration_practices_score",
    ]

# Aliases fallback (only used for labels later)
if 'ALIASES' not in globals():
    ALIASES = {
        "ann_knowledge_sharing": "Knowledge sharing",
```

```

        "ann_evaluation_practices": "Evaluation practices",
        "ann_idea_management": "Idea management",
        "ann_coordination_and_decision_practices": "Coordination & decision practices",
        "ann_information_seeking": "Information seeking",
        "ann_relational_climate": "Relational climate",
        "ann_participation_dynamics": "Participation dynamics",
        "ann_integration_practices": "Integration practices",
        "ann_none": "None",
    }

    # Person keys fallback (per-year scope by default)
    if 'PERSON_KEYS' not in globals():
        PERSON_KEYS = ["person_id" if "person_id" in df.columns else "person"]

    # Output dirs fallback
    if 'FIGDIR' not in globals():
        FIGDIR = Path("./figures"); FIGDIR.mkdir(parents=True, exist_ok=True)
    if 'REGDIR' not in globals():
        REGDIR = Path("./regression-model-output"); REGDIR.mkdir(parents=True, exist_ok=True)

    # to_wide helper (if not defined yet)
    if 'to_wide' not in globals():
        def to_wide(dfin: pd.DataFrame) -> pd.DataFrame:
            cols = REG_FEATURES + ["p_speaking_duration_sec"]
            blocks = []
            for col in cols:
                pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment",
                                       values=col, aggfunc="sum")
                pt.columns = [f"{col}_{c}" for c in pt.columns]
                blocks.append(pt)
            wide_ = pd.concat(blocks, axis=1).reset_index()

            # speaking controls
            if "p_speaking_duration_sec_first" in wide_.columns:
                wide_["speaking_first"] = pd.to_numeric(wide_["p_speaking_duration_sec_first"], errors="coerce")
            if "p_speaking_duration_sec_middle" in wide_.columns:
                wide_["speaking_middle"] = pd.to_numeric(wide_["p_speaking_duration_sec_middle"], errors="coerce")

            # roles / flags if source columns exist in the long df
            if "role_facilitator" in dfin.columns:
                rf = dfin.groupby(PERSON_KEYS, as_index=False)["role_facilitator"]
                wide_ = wide_.merge(rf, on=PERSON_KEYS, how="left")
                wide_["role_facilitator"] = pd.to_numeric(wide_["role_facilitator"], errors="coerce")

            for flag, col in [("any_team", "num_teams"), ("any_funded", "num_funded")]:
                if col in dfin.columns:
                    tmp = dfin.groupby(PERSON_KEYS, as_index=False)[col].sum()
                    tmp[flag] = (pd.to_numeric(tmp[col], errors="coerce") > 0)
                    wide_ = wide_.merge(tmp[PERSON_KEYS + [flag]], on=PERSON_KEYS, how="left")
                else:
                    wide_[flag] = 0
            return wide_

```

```

# Build wide if missing
if 'wide' not in globals():
    wide = to_wide(df)

# ----- Middle → role helpers -----
def _middle_cols(feats: List[str]) -> List[str]:
    return [f"{f}_middle" for f in feats]

def _build_middle_df() -> pd.DataFrame:
    keep = []
    for y in ["role_facilitator", "any_team", "any_funded"]:
        if y in wide.columns: keep.append(y)

    keep += [c for c in _middle_cols(REG_FEATURES) if c in wide.columns]

    if "speaking_middle" in wide.columns:
        keep.append("speaking_middle")

    keep += [c for c in PERSON_KEYS if c in wide.columns]
    dF = wide[keep].copy()

    # Clean numeric types
    for y in ["role_facilitator", "any_team", "any_funded"]:
        if y in dF.columns:
            dF[y] = pd.to_numeric(dF[y], errors="coerce").fillna(0).as
    for c in _middle_cols(REG_FEATURES) + ["speaking_middle"]:
        if c in dF.columns:
            dF[c] = pd.to_numeric(dF[c], errors="coerce").fillna(0.0)
    return dF

def _fit_logit_middle2role(dfR: pd.DataFrame, outcome: str) -> Tuple[p
    X_cols = [c for c in _middle_cols(REG_FEATURES) if c in dfR.columns]
    if (outcome not in dfR.columns) or (len(X_cols) == 0):
        print(f"[middle→role] Skipping {outcome}: predictors or outcome missing")
        return pd.DataFrame(), None

    rhs_terms = X_cols + (["speaking_middle"] if "speaking_middle" in dfR.columns else [])
    rhs = " + ".join(rhs_terms)

    if "conference" in PERSON_KEYS: # add FE only when per-year scope is available
        rhs += " + C(conference)"

    fml = f"{outcome} ~ {rhs}"
    m = smf.logit(fml, data=dfR).fit(dis=0, maxiter=200)

    tab = pd.DataFrame({
        "term": m.params.index,
        "coef": m.params.values,
        "se": m.bse.values,
        "p": m.pvalues.values
    })
    tab["or"] = np.exp(tab["coef"])
    tab["lo95"] = np.exp(tab["coef"] - 1.96*tab["se"])

```

```

tab["hi95"] = np.exp(tab["coef"] + 1.96*tab["se"])
tab.to_csv(REGDIR / f"logit_middle2role_{outcome}.csv", index=False)
return tab, m

def _pretty_middle_label(t: str) -> str:
    if t == "speaking_middle":
        return "Speaking time (middle)"
    if t.endswith("_middle"):
        base = t.replace("_middle", "").replace("_score", "").replace(
            human = ALIASES.get("ann_"+base, base.replace("_", " ").title()
        return f"{human} (middle)"
    return t

def _plot_logit_middle2role(tab: pd.DataFrame, outcome: str):
    if tab is None or tab.empty:
        return
    take = tab[tab["term"].str.endswith("_middle") | (tab["term"]=="sp
    if take.empty:
        print(f"[middle→role] No middle-third rows to plot for {outcom

    take = take.sort_values("coef", key=lambda s: s.abs(), ascending=False)
    take = take.sort_values("or")

    y = np.arange(len(take))
    fig, ax = plt.subplots(figsize=(10.8, 6))
    ax.errorbar(take["or"], y,
                xerr=[take["or"] - take["lo95"], take["hi95"] - take["
                fmt="o", capsize=3)
    ax.axvline(1.0, color="black", lw=1)
    ax.set_yticks(y); ax.set_yticklabels([_pretty_middle_label(t) for
    title_map = {"role_facilitator": "Facilitator (1/0)", "any_team": "A
    ax.set_xlabel("Odds ratio (middle-third predictors, ±95% CI)")
    ax.set_title(f"Middle (second-third) behaviors → {title_map.get(ou
    fig.tight_layout()
    fig.savefig(FIGDIR / f"logit_middle2role_{outcome}.png", dpi=300,
    plt.close(fig)

def run_middle_to_role_models():
    dfR = _build_middle_df()
    outcomes = [y for y in ["role_facilitator", "any_team", "any_funded"]
    for y in outcomes:
        print(f"[9] Middle→role logistic for: {y}")
        tab, m = _fit_logit_middle2role(dfR, y)
        if tab is not None and not tab.empty:
            _plot_logit_middle2role(tab, y)
        try:
            with open(REGDIR / f"logit_middle2role_{y}_summary.txt
                f.write(m.summary2().as_text())
        except Exception:
            pass

print("[9] Middle-third behaviors → roles (logit)")

```

```
run_middle_to_role_models()
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

```
/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated
```

ed and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
pt = df.in.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

/var/folders/17/lphw45ds14n5t74zwjdfybx40000gn/T/ipykernel_87681/2774640.py:68: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
pt = df.in.pivot_table(index=PERSON_KEYS, columns="segment", values=col, aggfunc="sum")
```

[9] Middle-third behaviors → roles (logit)

[9] Middle→role logistic for: role_facilitator

[9] Middle→role logistic for: any_team

[9] Middle→role logistic for: any_funded

looking at it from the overall perspective (person-level)

```
In [2]: # =====
# BEHAVIORAL DELTAS → ROLES (logit)
#   - Δ_mid  = middle - first
#   - Δ_end  = last   - middle
# Predictors: all feature deltas + speaking deltas (+ C(conference) if
# Outcomes : role_facilitator (1/0), any_team (1/0), any_funded (1/0)
# Outputs  : CSVs + forest plots (odds ratios ±95% CI)
# =====

# ----- LOAD DATA IF NEEDED -----
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from typing import List, Tuple
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Define base + load long-format file (only if df missing)
if 'df' not in globals():
    BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human
    FEAT = BASE / "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.
    df = pd.read_csv(FEAT)
    df = df[df["segment"].isin(["first", "middle", "last"])]
    df["segment"] = pd.Categorical(df["segment"], categories=["first",
    if "conference" in df.columns:
        df["conference"] = df["conference"].astype(str)
    print("✅ Loaded df with shape:", df.shape)

# ----- Expected globals with safe fallbacks -----
```

```

assert 'df' in globals(), "Please load your long data into `df` first.

# REG_FEATURES fallback
if 'REG_FEATURES' not in globals():
    REG_FEATURES = [
        "ann_knowledge_sharing_score",
        "ann_evaluation_practices_score",
        "ann_idea_management_score",
        "ann_coordination_and_decision_practices_score",
        "ann_information_seeking_score",
        "ann_relational_climate_score",
        "ann_participation_dynamics_score",
        "ann_integration_practices_score",
    ]

# Aliases fallback
if 'ALIASES' not in globals():
    ALIASES = {
        "ann_knowledge_sharing": "Knowledge sharing",
        "ann_evaluation_practices": "Evaluation practices",
        "ann_idea_management": "Idea management",
        "ann_coordination_and_decision_practices": "Coordination & decision",
        "ann_information_seeking": "Information seeking",
        "ann_relational_climate": "Relational climate",
        "ann_participation_dynamics": "Participation dynamics",
        "ann_integration_practices": "Integration practices",
        "ann_none": "None",
    }

# PERSON_KEYS fallback
if 'PERSON_KEYS' not in globals():
    PERSON_KEYS = ["person_id" if "person_id" in df.columns else "person_id"]

# Output dirs
if 'FIGDIR' not in globals():
    FIGDIR = Path("./figures"); FIGDIR.mkdir(parents=True, exist_ok=True)
if 'REGDIR' not in globals():
    REGDIR = Path("./regression-model-output"); REGDIR.mkdir(parents=True, exist_ok=True)

# ----- Build wide -----
if 'to_wide' not in globals():
    def to_wide(dfin: pd.DataFrame) -> pd.DataFrame:
        cols = REG_FEATURES + ["p_speaking_duration_sec"]
        blocks = []
        for col in cols:
            pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment",
                                   values=col, aggfunc="sum", observed=True)
            pt.columns = [f"{col}_{c}" for c in pt.columns]
            blocks.append(pt)
        wide_ = pd.concat(blocks, axis=1).reset_index()

    # speaking per segment for deltas

```

```

for seg in ["first", "middle", "last"]:
    c = f"p_speaking_duration_sec_{seg}"
    if c not in wide_.columns:
        wide_[c] = 0.0

# role flags
if "role_facilitator" in dfin.columns:
    rf = dfin.groupby(PERSON_KEYS, as_index=False)["role_facil
    wide_ = wide_.merge(rf, on=PERSON_KEYS, how="left")
    wide_["role_facilitator"] = pd.to_numeric(wide_["role_faci

for flag, col in [("any_team", "num_teams"), ("any_funded", "num
    if col in dfin.columns:
        tmp = dfin.groupby(PERSON_KEYS, as_index=False)[col].s
        tmp[flag] = (tmp[col] > 0).astype(int)
        wide_ = wide_.merge(tmp[PERSON_KEYS + [flag]], on=PERS
    else:
        wide_[flag] = 0

return wide_

if 'wide' not in globals():
    wide = to_wide(df)

# ----- Make deltas -----
def _delta_name(base: str, which: str) -> str:
    return f"{base}_d{which}"

def _base_from_feature_col(f: str) -> str:
    return f.replace("_score", "")

def make_deltas(w: pd.DataFrame) -> pd.DataFrame:
    out = w.copy()
    # speaking deltas
    out["speaking_dmid"] = out["p_speaking_duration_sec_middle"] - out
    out["speaking_dend"] = out["p_speaking_duration_sec_last"] - out

    for f in REG_FEATURES:
        c_first, c_middle, c_last = f"{f}_first", f"{f}_middle", f"{f}
        if all(c in w.columns for c in [c_first, c_middle, c_last]):
            out[_delta_name(f, "mid")] = w[c_middle].fillna(0) - w[c_fi
            out[_delta_name(f, "end")] = w[c_last].fillna(0) - w[c_mi
        else:
            out[_delta_name(f, "mid")] = 0.0
            out[_delta_name(f, "end")] = 0.0

    return out

wide_delta = make_deltas(wide)

# ----- Helpers -----
def _has_conference_fe() -> bool:
    return "conference" in PERSON_KEYS

```

```

def _delta_cols(which: str) -> List[str]:
    return [_delta_name(f, which) for f in REG_FEATURES]

def _nice_delta_label(term: str) -> str:
    if term == "speaking_dmid": return "Speaking time (Δ middle)"
    if term == "speaking_dend": return "Speaking time (Δ end)"
    if term.endswith("_dmid") or term.endswith("_dend"):
        base = term.replace("_score", "").replace("_dmid", "").replace("_dend", "")
        human = ALIASES.get("ann_" + base, base.replace("_", " ").title)
        tail = "Δ middle" if term.endswith("_dmid") else "Δ last"
        return f"{human} ({tail})"
    return term

def _fit_delta_logit(dfR: pd.DataFrame, outcome: str, which: str) -> T:
    preds = [c for c in _delta_cols(which) if c in dfR.columns]
    if len(preds) == 0 or outcome not in dfR.columns:
        print(f"[delta-{which}] Skipping {outcome}: predictors/outcome")
        return pd.DataFrame(), None

    rhs_terms = preds + ([f"speaking_d{which}"] if f"speaking_d{which}" in dfR.columns else [])
    rhs = " + ".join(rhs_terms)
    if _has_conference_fe():
        rhs += " + C(conference)"

    fml = f"{outcome} ~ {rhs}"
    m = smf.logit(fml, data=dfR).fit(dis=0, maxiter=200)

    tab = pd.DataFrame({
        "term": m.params.index,
        "coef": m.params.values,
        "se": m.bse.values,
        "p": m.pvalues.values
    })
    tab["or"] = np.exp(tab["coef"])
    tab["lo95"] = np.exp(tab["coef"] - 1.96*tab["se"])
    tab["hi95"] = np.exp(tab["coef"] + 1.96*tab["se"])
    tab["which_delta"] = which
    return tab, m

def _plot_delta_forest(tab: pd.DataFrame, outcome: str, which: str):
    keep = tab["term"].str.contains("_dmid|_dend") | tab["term"].isin(["speaking_dmid", "speaking_dend"])
    take = tab[keep].copy()
    if take.empty:
        print(f"[delta-{which}] No rows to plot for {outcome}.");
        return

    take = take.sort_values("coef", key=lambda s: s.abs(), ascending=False)
    take = take.sort_values("or")

    y = np.arange(len(take))
    fig, ax = plt.subplots(figsize=(11, 6))

```

```

ax.errorbar(take["or"], y,
            xerr=[take["lo95"] - take["lo95"], take["hi95"] - take["lo95"]],
            fmt="o", capsize=3)
ax.axvline(1.0, color="black", lw=1)
ax.set_yticks(y)
ax.set_yticklabels([_nice_delta_label(t) for t in take["term"]])
title_map = {"role_facilitator": "Facilitator (1/0)", "any_team": "Any team (1/0)", "any_funded": "Any funded (1/0)"}
ax.set_xlabel("Odds ratio (delta predictors,  $\pm 95\%$  CI)")
ax.set_title(f"Behavioral change ({'middle-first' if which=='mid' else 'end'})")
fig.tight_layout()
fig.savefig(FIGDIR / f"logit_delta_{which}_{outcome}.png", dpi=300)
plt.close(fig)

# ----- Runner -----
def run_delta_role_models():
    dfR = wide_delta.copy()
    for y in ["role_facilitator", "any_team", "any_funded"]:
        if y in dfR.columns:
            dfR[y] = pd.to_numeric(dfR[y], errors="coerce").fillna(0)

    outcomes = [y for y in ["role_facilitator", "any_team", "any_funded"] if y in dfR.columns]

    for which in ["mid", "end"]:
        for y in outcomes:
            print(f"[{which}] Logistic for {y} ...")
            tab, m = _fit_delta_logit(dfR, y, which)
            if tab is None or tab.empty:
                continue
            tab.to_csv(REGDIR / f"logit_delta_{which}_{y}.csv", index=False)
            try:
                with open(REGDIR / f"logit_delta_{which}_{y}_summary.txt", "w") as f:
                    f.write(m.summary2().as_text())
            except Exception:
                pass
            _plot_delta_forest(tab, y, which)

# ---- Call it ----
print("[10] Behavioral deltas  $\rightarrow$  roles (logit)")
run_delta_role_models()

```

```

✓ Loaded df with shape: (3209, 20)
[10] Behavioral deltas  $\rightarrow$  roles (logit)
[Δmid] Logistic for role_facilitator ...
[Δmid] Logistic for any_team ...
[Δmid] Logistic for any_funded ...
[Δend] Logistic for role_facilitator ...
[Δend] Logistic for any_team ...
[Δend] Logistic for any_funded ...

```

transition matrix

In [4]:

```
# =====
# COMBINED TRANSITION PANELS (one PNG per feature × group)
# Panels (left→right): first→middle | middle→last | implied first→last
# Each is a 3×3 Low/Mid/High row→stochastic matrix (rows sum to 1.0)
# Files saved as: tmatrix_combo_{group}_{featurecore}.png in FIGDIR
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

# ----- Safe fallbacks / assumptions -----
try:
    df # your long data
except NameError:
    raise AssertionError("Please define `df` (long data with segment E")

if 'REG_FEATURES' not in globals():
    REG_FEATURES = [
        "ann_knowledge_sharing_score",
        "ann_evaluation_practices_score",
        "ann_idea_management_score",
        "ann_coordination_and_decision_practices_score",
        "ann_information_seeking_score",
        "ann_relational_climate_score",
        "ann_participation_dynamics_score",
        "ann_integration_practices_score",
    ]

if 'ALIASES' not in globals():
    ALIASES = {
        "ann_knowledge_sharing": "Knowledge sharing",
        "ann_evaluation_practices": "Evaluation practices",
        "ann_idea_management": "Idea management",
        "ann_coordination_and_decision_practices": "Coordination & decision",
        "ann_information_seeking": "Information seeking",
        "ann_relational_climate": "Relational climate",
        "ann_participation_dynamics": "Participation dynamics",
        "ann_integration_practices": "Integration practices",
        "ann_none": "None",
    }

if 'PERSON_KEYS' not in globals():
    PERSON_KEYS = ["person_id" if "person_id" in df.columns else "pers"]

if 'FIGDIR' not in globals():
    FIGDIR = Path("./figures")
    FIGDIR.mkdir(parents=True, exist_ok=True)

# Ensure segment filter + category
```

```

df = df[df["segment"].isin(["first", "middle", "last"])] .copy()
df["segment"] = pd.Categorical(df["segment"], ["first", "middle", "last"])
if "conference" in df.columns:
    df["conference"] = df["conference"].astype(str)

# Build / reuse wide
if 'to_wide' not in globals():
    def to_wide(dfin: pd.DataFrame) -> pd.DataFrame:
        blocks = []
        for col in REG_FEATURES:
            pt = dfin.pivot_table(index=PERSON_KEYS, columns="segment",
                                   values=col, aggfunc="sum")
            pt.columns = [f"{col}_{c}" for c in pt.columns]
            blocks.append(pt)
        wide_ = pd.concat(blocks, axis=1).reset_index()

        # speaking per segment (optional)
        if "p_speaking_duration_sec" in dfin.columns:
            sp = dfin.pivot_table(index=PERSON_KEYS, columns="segment",
                                   values="p_speaking_duration_sec", aggfunc="sum")
            sp.columns = [f"p_speaking_duration_sec_{c}" for c in sp.columns]
            wide_ = wide_.merge(sp, on=PERSON_KEYS, how="left")
        else:
            for c in ["first", "middle", "last"]:
                wide_[f"p_speaking_duration_sec_{c}"] = 0.0

        # group flags if present
        if "role_facilitator" in dfin.columns:
            rf = dfin.groupby(PERSON_KEYS, as_index=False)["role_facilitator"]
            wide_ = wide_.merge(rf, on=PERSON_KEYS, how="left")
            wide_["role_facilitator"] = pd.to_numeric(wide_["role_facilitator"], errors="coerce")

        for flag, col in [("any_team", "num_teams"), ("any_funded", "num_funded")]:
            if col in dfin.columns:
                tmp = dfin.groupby(PERSON_KEYS, as_index=False)[col].sum()
                tmp[flag] = (tmp[col] > 0).astype(int)
                wide_ = wide_.merge(tmp[PERSON_KEYS + [flag]], on=PERSON_KEYS, how="left")
            else:
                wide_[flag] = 0
        return wide_

try:
    wide = to_wide(df)
except NameError:
    wide = df

# ----- Helpers -----
def _nice_feature_name(f: str) -> str:
    base = f.replace("_score", "").replace("ann_", "")
    return ALIASES.get("ann_" + base, base.replace("_", " ").title())

def _robust_tertiles(x: pd.Series) -> pd.Series:
    x = pd.to_numeric(x, errors="coerce")

```

```

if x.notna().sum() == 0:
    return pd.Series(pd.Categorical(["mid"]*len(x), categories=["l

uniq = np.unique(x.dropna())
if len(uniq) < 3:
    r = x.rank(method="average", pct=True)
    return pd.cut(r, bins=[-0.001, 1/3, 2/3, 1.001], labels=["low"

try:
    return pd.qcut(x, q=3, labels=["low","mid","high"])
except Exception:
    std = x.std()
    jitter = (std if std > 0 else 1e-6) * 1e-6
    xr = x + np.random.default_rng(0).normal(0, jitter, size=len(x)
    return pd.qcut(xr, q=3, labels=["low","mid","high"])

def _role_masks(w: pd.DataFrame):
    masks = {"all": pd.Series(True, index=w.index)}
    if "role_facilitator" in w.columns:
        masks["facilitator"] = (w["role_facilitator"] == 1)
    if "any_team" in w.columns:
        masks["any_team"] = (w["any_team"] == 1)
    if "any_funded" in w.columns:
        masks["any_funded"] = (w["any_funded"] == 1)
    return masks

def _cat_cols_for_feature(w: pd.DataFrame, feature: str):
    cols = {}
    for seg in ["first","middle","last"]:
        raw = f"{feature}_{seg}"
        if raw not in w.columns:
            w[raw] = 0.0
        cols[seg] = _robust_tertiles(w[raw])
    return cols # dict: seg -> categorical series (low/mid/high)

def _transition_tbl(cats_src: pd.Series, cats_dst: pd.Series) -> pd.Da
tbl = pd.crosstab(cats_src, cats_dst, normalize="index").reindex(
    index=["low","mid","high"], columns=["low","mid","high"]
)
return tbl.fillna(0.0)

def _plot_combo(feature_name: str, group_name: str, T_fm: pd.DataFrame
fig, axes = plt.subplots(1, 3, figsize=(15.6, 5.0), constrained_la
for ax, T, title in zip(
    axes,
    [T_fm, T_ml, T_fl],
    ["first -> middle", "middle -> last", "implied first -> last"]
):
    sns.heatmap(T, annot=True, fmt=".2f", cmap="Blues", vmin=0, vm
        cbar=False, linewidths=.5, linecolor="white", ax=a
        annot_kws={"fontsize":10})
    ax.set_xlabel("To state"); ax.set_ylabel("From state")

```

```

        ax.set_title(title)

    fig.suptitle(f"Transition matrices – {feature_name} – {group_name}")
    fig.savefig(outpath, dpi=300, bbox_inches="tight")
    plt.close(fig)

# ----- Main -----
def make_transition_panels_combined(features=None):
    feats = features if features is not None else REG_FEATURES
    masks = _role_masks(wide)

    for f in feats:
        feat_nice = _nice_feature_name(f)
        fname_core = feat_nice.lower().replace(" & ", "_").replace(" "
        # Categorize per segment (on the full sample once, then mask s
        cat_dict = _cat_cols_for_feature(wide, f) # {'first':..., 'mi

        for gname, gmask in masks.items():
            idx = gmask[gmask].index
            if len(idx) == 0:
                continue

            c_first = cat_dict["first"].loc[idx]
            c_middle = cat_dict["middle"].loc[idx]
            c_last = cat_dict["last"].loc[idx]

            # Build 3x3 transitions
            T_fm = _transition_tbl(c_first, c_middle) # first → midd
            T_ml = _transition_tbl(c_middle, c_last) # middle → las

            # Implied first → last (Markov composition): T_fl = T_fm @
            # Cast to numpy for clean matmul, then back to DataFrame w
            T_fl_np = T_fm.values @ T_ml.values
            T_fl = pd.DataFrame(T_fl_np, index=T_fm.index, columns=T_m

            out = FIGDIR / f"tmatrix_combo_{gname}_{fname_core}.png"
            _plot_combo(feat_nice, gname, T_fm, T_ml, T_fl, out)

# ----- Call -----
print("[T*] Building combined transition panels (first→middle | middle
make_transition_panels_combined()
print("[T*] Done. See combined PNGs in:", FIGDIR)

```

```

[T*] Building combined transition panels (first→middle | middle→last |
implied first→last)
[T*] Done. See combined PNGs in: figures

```

combining into one png file

```

In [5]: # =====
# Combine all transition-matrix PNGs into a single grid image
# =====

```

```

from PIL import Image
import numpy as np
import math
from pathlib import Path

FIGDIR = Path("./figures") # adjust if different
OUTFILE = FIGDIR / "tmatrix_all_combined.png"

# --- Gather all PNGs that match your transition naming pattern ---
pngs = sorted(FIGDIR.glob("tmatrix_combo_*.png"))

if not pngs:
    raise FileNotFoundError("No transition PNGs found. Run make_transi

# --- Load all images ---
imgs = [Image.open(p).convert("RGB") for p in pngs]

# --- Resize to uniform size (based on first image) ---
base_w, base_h = imgs[0].size
imgs = [img.resize((base_w, base_h)) for img in imgs]

# --- Define grid layout ---
cols = 3 # number of images per row (adjust as you like)
rows = math.ceil(len(imgs) / cols)

# --- Create composite canvas ---
gap = 20 # pixel gap between images
canvas_w = cols * base_w + (cols + 1) * gap
canvas_h = rows * base_h + (rows + 1) * gap
combined = Image.new("RGB", (canvas_w, canvas_h), color=(255, 255, 255))

# --- Paste images into grid ---
for i, img in enumerate(imgs):
    r, c = divmod(i, cols)
    x = gap + c * (base_w + gap)
    y = gap + r * (base_h + gap)
    combined.paste(img, (x, y))

# --- Save combined image ---
combined.save(OUTFILE)
print(f"✅ Combined transition image saved to: {OUTFILE}")
print(f"Included {len(imgs)} images in a {rows}x{cols} grid.")

```

✅ Combined transition image saved to: figures/tmatrix_all_combined.png
Included 32 images in a 11x3 grid.

one mega matrix

```

In [7]: # =====
# MEGA TRANSITION MATRIX (person-level, temporal thirds)
# - Robust terciles via rank percentiles (handles all-zeros safely)

```

```

# - Aligns each person across segments; computes  $P(\text{state}_{t+1} \mid \text{state}_t)$ 
# - Groups: All, Facilitator/Non-, AnyTeam/NoTeam, AnyFunded/NoFunded
# - Saves one combined PNG: figures/mega_transition_matrix.png
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from typing import List, Tuple

# ----- INPUTS / FALLBACKS -----
assert 'df' in globals(), "Please define `df` (long table) before running"

# Ensure segment is ordered
df = df[df['segment'].isin(['first', 'middle', 'last'])].copy()
df['segment'] = pd.Categorical(df['segment'], categories=['first', 'mid

# Figure dir (fallback)
if 'FIGDIR' not in globals():
    FIGDIR = Path('./figures')
FIGDIR.mkdir(parents=True, exist_ok=True)
OUTFILE = FIGDIR / 'mega_transition_matrix.png'

# If you already have REG_FEATURES, we'll prefer their *_count version
if 'REG_FEATURES' in globals():
    _counts = [f.replace('_score', '_count') for f in REG_FEATURES]
    FEATURES = [c for c in _counts if c in df.columns]
    if not FEATURES:
        FEATURES = [f for f in REG_FEATURES if f in df.columns]
else:
    # autodetect ann_* features (score or count)
    cand = [c for c in df.columns if c.startswith('ann_') and (c.endswith
    # keep a stable order
    FEATURES = sorted(list(dict.fromkeys(cand)))

# Pretty names
if 'ALIASES' not in globals():
    ALIASES = {
        "ann_knowledge_sharing": "Knowledge sharing",
        "ann_evaluation_practices": "Evaluation practices",
        "ann_idea_management": "Idea management",
        "ann_coordination_and_decision_practices": "Coordination & dec
        "ann_information_seeking": "Information seeking",
        "ann_relational_climate": "Relational climate",
        "ann_participation_dynamics": "Participation dynamics",
        "ann_integration_practices": "Integration practices",
    }

def nice_feature(col: str) -> str:
    base = col.replace('ann_', '').replace('_score', '').replace('_count

```

```

        return ALIASES.get('ann_'+base, base.replace('_', ' ').title())

# Keys to align the same person across segments
if 'PERSON_KEYS' in globals():
    PERSON_KEYS = [k for k in PERSON_KEYS if k in df.columns]
else:
    PERSON_KEYS = []
    PERSON_KEYS.append('person_id' if 'person_id' in df.columns else '
    for k in ['conference', 'year']:
        if k in df.columns:
            PERSON_KEYS.append(k)

# ----- TERCILES (robust via rank percentiles) -----
STATES = ["Low", "Mid", "High"]

def tercile_states_rank(x: pd.Series) -> pd.Series:
    """Rank-based terciles: never fails on flat/zero series."""
    x_num = pd.to_numeric(x, errors="coerce").fillna(0.0)
    r = x_num.rank(method="average", pct=True) # in [0,1]
    lab = pd.cut(r, [0.0, 1/3, 2/3, 1.0], labels=STATES, include_lowes
    return lab.astype(str).fillna("Low")

def build_state_df(df_long: pd.DataFrame, features: List[str]) -> pd.D
    """Create *_state (Low/Mid/High) columns per feature×segment using
    take = df_long.copy()
    take = take[take['segment'].isin(['first', 'middle', 'last'])].copy(
    take['segment'] = pd.Categorical(take['segment'], categories=['fir

    keep_keys = [c for c in (PERSON_KEYS + ['segment']) if c in take.c
    out = take[keep_keys].copy()

    for f in features:
        if f not in take.columns:
            continue
        out[f + '_state'] = (
            take.groupby('segment', observed=True)[f]
                .transform(tercile_states_rank)
                .astype(str)
                .fillna("Low")
        )

    # carry group flags if present
    for flag in ['role_facilitator', 'any_team', 'any_funded']:
        if flag in take.columns:
            out[flag] = take[flag].values

    return out

# ----- TRANSITIONS -----
def cross_transition(S: pd.DataFrame, feature: str, start_seg: str, en
    """P(state_end | state_start) with proper person alignment."""
    if row_mask is not None:

```

```

        S = S[row_mask].copy()
        cols_needed = [feature + '_state', 'segment'] + [k for k in PERSON
        miss = [c for c in cols_needed if c not in S.columns]
        if miss:
            return pd.DataFrame(np.zeros((3,3)), index=STATES, columns=STA

# Pivot person×segment → feature_state
idx_keys = PERSON_KEYS
wide = S.pivot_table(index=idx_keys, columns='segment', values=fea
if start_seg not in wide.columns or end_seg not in wide.columns:
    return pd.DataFrame(np.zeros((3,3)), index=STATES, columns=STA

a = wide[start_seg]
b = wide[end_seg]
valid = a.notna() & b.notna()
N = int(valid.sum())
if N == 0:
    return pd.DataFrame(np.zeros((3,3)), index=STATES, columns=STA

ct = pd.crosstab(a[valid], b[valid], dropna=False)
ct = ct.reindex(index=STATES, columns=STATES, fill_value=0)
prob = ct.div(ct.sum(axis=1).replace(0, np.nan), axis=0).fillna(0)
return prob, N

# ----- MAIN PLOT -----
def plot_mega_transition(df_long: pd.DataFrame, features: List[str]=No
    """
    Renders a grid: rows=features, columns = [first→middle, middle→las
    Groups shown if available: All, Facilitator/Non-, AnyTeam/NoTeam
    Each cell is a 3×3 heatmap of P(next_state | current_state) with N
    """
    feats = [f for f in (features or FEATURES) if f in df_long.columns
    if max_features and len(feats) > max_features:
        feats = feats[:max_features]

    S = build_state_df(df_long, feats)

    # define groups present in data
    groups = [('All', np.ones(len(S), dtype=bool))]
    if 'role_facilitator' in S.columns:
        groups += [('Facilitator=1', S['role_facilitator'].astype(int)
                    ('Facilitator=0', S['role_facilitator'].astype(int)
    if 'any_team' in S.columns:
        groups += [('Any team=1', S['any_team'].astype(int)==1),
                    ('Any team=0', S['any_team'].astype(int)==0)]
    if 'any_funded' in S.columns:
        groups += [('Any funded=1', S['any_funded'].astype(int)==1),
                    ('Any funded=0', S['any_funded'].astype(int)==0)]

    transitions = [('first', 'middle', 'F→M'), ('middle', 'last', 'M→L')]

    n_rows = len(feats)

```

```

n_cols = len(groups) * len(transitions)

# sizing: keep compact but readable
cell_w, cell_h = 1.9, 1.9
fig_w = max(8, cell_w * n_cols + 1.0)
fig_h = max(6, cell_h * n_rows + 1.5)

fig, axes = plt.subplots(n_rows, n_cols, figsize=(fig_w, fig_h), s
cmap = 'Blues'

for i, f in enumerate(feats):
    for jg, (gname, gmask) in enumerate(groups):
        for jt, (s1, s2, tag) in enumerate(transitions):
            j = jg*len(transitions) + jt
            ax = axes[i, j]
            mat, N = cross_transition(S, f, s1, s2, row_mask=gmask
sns.heatmap(mat, ax=ax, cmap=cmap, vmin=0, vmax=1, cba
            annot=True, fmt=".2f", linewidths=0.5, lin
            annot_kws={'fontsize':8})

            if i == 0:
                ax.set_title(f"{tag} • {gname}\nN={N}", fontsize=9
            else:
                ax.set_title(f"N={N}", fontsize=8)
            ax.set_xlabel('Next state'); ax.set_ylabel('Current st
            ax.set_xticklabels(STATES, rotation=0, fontsize=8)
            ax.set_yticklabels(STATES, rotation=0, fontsize=8)

# row label on the left margin
axes[i,0].text(-0.8, 1.15, nice_feature(f), fontsize=10, fontw
            transform=axes[i,0].transAxes, ha='left', va='c

fig.suptitle("Within-person behavior transitions: P(next tercile |
plt.tight_layout()
fig.savefig(outfile, dpi=200, bbox_inches='tight')
plt.close(fig)
print(f"[mega-transition] saved -> {outfile}")

# ----- RUN -----
plot_mega_transition(df, max_features=8) # increase/decrease max_fea
[mega-transition] saved -> figures/mega_transition_matrix.png

```

five analyses at once

```

In [10]: # =====
# TRAJECTORY CLUSTERING – FIXED OUTPUT PATH SETUP
# =====
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# ----- 1. Define stable base path -----
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gen

# define absolute folders
FIGDIR = BASE / "figures"
REGDIR = BASE / "regression-model-output"

# create folders if they don't exist
FIGDIR.mkdir(parents=True, exist_ok=True)
REGDIR.mkdir(parents=True, exist_ok=True)

print("✅ Output directories ready:")
print("FIGDIR:", FIGDIR.resolve())
print("REGDIR:", REGDIR.resolve())

# quick write test
(FIGDIR / "_write_test.txt").write_text("ok")
assert (FIGDIR / "_write_test.txt").exists(), "Could not write to FIGD

# =====
# TRAJECTORY CLUSTERING FUNCTION
# =====
def run_trajectory_clustering(k=3):
    """
    Cluster people by their temporal feature trajectories across third
    Creates a trajectory_clusters.png figure and saves cluster assignm
    """
    global df # assumes df already loaded (long-form)

    # make sure segment ordering is correct
    df["segment"] = pd.Categorical(df["segment"], categories=["first",

    # aggregate to person × segment
    PERSON_KEYS = ["person_id" if "person_id" in df.columns else "pers
    feats = [
        "ann_knowledge_sharing_score",
        "ann_evaluation_practices_score",
        "ann_idea_management_score",
        "ann_coordination_and_decision_practices_score",
        "ann_information_seeking_score",
        "ann_relational_climate_score",
        "ann_participation_dynamics_score",
        "ann_integration_practices_score",
    ]

    # pivot: wide table (person × [feature_segment])
    blocks = []
    for f in feats:

```

```

        pt = df.pivot_table(index=PERSON_KEYS, columns="segment", values=
        pt.columns = [f"{f}_{c}" for c in pt.columns]
        blocks.append(pt)
    wide = pd.concat(blocks, axis=1).reset_index()

    # standardize and cluster
    X = wide.drop(columns=PERSON_KEYS)
    X_scaled = StandardScaler().fit_transform(X)
    km = KMeans(n_clusters=k, random_state=42)
    wide["traj_cluster"] = km.fit_predict(X_scaled)

    # compute cluster-mean trajectories
    cluster_means = []
    for f in feats:
        cluster_mean = (
            wide.groupby("traj_cluster")[f"{f}_first", f"{f}_middle",
            .mean()
            .reset_index()
        )
        cluster_mean["feature"] = f
        cluster_means.append(cluster_mean)
    cluster_means = pd.concat(cluster_means, ignore_index=True)

    # ---- Plot ----
    fig, ax = plt.subplots(figsize=(10, 6))
    for cid, grp in cluster_means.groupby("traj_cluster"):
        for f in feats:
            sub = grp[grp["feature"] == f]
            if not sub.empty:
                ax.plot(
                    ["first", "middle", "last"],
                    sub[[f"{f}_first", f"{f}_middle", f"{f}_last"]].mean(),
                    marker="o", label=f"Cluster {cid}" if f == feats[0]
                    alpha=0.5,
                )
    ax.set_xlabel("Temporal third")
    ax.set_ylabel("Feature mean (standardized)")
    ax.set_title("Trajectory clusters (average paths across temporal t
    ax.legend(title="Cluster", loc="center left", bbox_to_anchor=(1.02
    fig.tight_layout()

    # ---- Save outputs ----
    fig.savefig(FIGDIR / "trajectory_clusters.png", dpi=300, bbox_inches=
    plt.close(fig)
    wide[PERSON_KEYS + ["traj_cluster"]].to_csv(REGDIR / "trajectory_c

    print(f"[T1] ✅ trajectory clustering saved → {FIGDIR / 'trajector
    print(f"[T1] ✅ cluster assignments saved → {REGDIR / 'trajectory_

    # ----- Run -----
    print("[T1] trajectory clustering ...")

```

```
run_trajectory_clustering(k=3)
```

✅ Output directories ready:

FIGDIR: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/figures

REGDIR: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output

[T1] trajectory clustering ...

[T1] ✅ trajectory clustering saved → /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/figures/trajectory_clusters.png

[T1] ✅ cluster assignments saved → /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/regression-model-output/trajectory_clusters_assignments.csv

interpreting clusters ?

```
In [6]: from pathlib import Path
import pandas as pd

# Base project dir
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/figures")

# Candidate locations for the assignments CSV
candidates = [
    BASE / "figures-five-figs" / "regression-model-output" / "trajectory_clusters_assignments.csv",
    BASE / "regression-model-output" / "trajectory_clusters_assignments.csv",
    BASE / "trajectory_clusters_assignments.csv", # just in case
]

assign_path = next((p for p in candidates if p.exists()), None)
if assign_path is None:
    raise FileNotFoundError(
        "Could not find trajectory_clusters_assignments.csv in any expected location: "
        + "\n".join(str(p) for p in candidates)
    )

print(f"✅ Using: {assign_path}")
assign = pd.read_csv(assign_path)

# Load the long/wide data you already used before
wide_path = BASE / "ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.csv"
print(f"✅ Loading roles from: {wide_path}")
df = pd.read_csv(wide_path)

# Collapse to person-level (max across segments)
role_flags = (
    df.groupby(["person_name", "conference", "year"], as_index=False)[
        ["role_facilitator", "num_teams", "num_funded_teams"]
    ]
)
```

```

        .max()
    )
    role_flags["any_team"] = (role_flags["num_teams"] > 0).astype(int)
    role_flags["any_funded"] = (role_flags["num_funded_teams"] > 0).astype(int)

    # Merge cluster assignments with roles (by person_name; adjust if you
    merged = assign.merge(role_flags, on="person_name", how="left")

    # Cluster composition summary
    summary = (
        merged.groupby("traj_cluster")[["role_facilitator", "any_team", "any_funded"]
        .mean()
        .round(2)
    )
    summary["n_people"] = merged.groupby("traj_cluster").size()

    print("\n=== Cluster composition by role ===")
    print(summary)

```

[✓] Using: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/figure-s-five-figs/regression-model-output/trajectory_clusters_assignments.csv
 [✓] Loading roles from: /Users/maxchalekson/Projects/NICO-Research/NICO_human-gemini/facilitator-identification/3-reconsolidating-features-temporal/ALL_person_year_segment_CONSOLIDATED_WITH_OUTCOMES.csv

```

=== Cluster composition by role ===
               role_facilitator  any_team  any_funded  n_people
traj_cluster
0                   0.01         0.16         0.06         1128
1                   0.16         0.48         0.20          331
2                   0.76         0.00         0.00          25

```

network perspective (taken from Patrick Park's 10/15/2025 NICO Seminar)

```

In [4]: # =====
# TEMPORAL INTERACTION NETWORKS (person-level)
# - Pseudo meeting id = conference + year
# - Edge if two people are co-active on any feature (top tercile)
# - Compute network metrics per meeting×segment + person×segment
# - Deltas (middle-first, last-middle) and simple logit links to roles
# Outputs:
# <BASE>/figures/network_*.png
# <BASE>/regression-model-output/network_*.csv
# =====

import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
from pathlib import Path

```

```

import statsmodels.formula.api as smf

# ----- EXPECTED INPUT -----
# df: long table with columns shown in your printout
assert "df" in globals(), "Load your long dataframe into `df` first."

# ----- PATHS -----
BASE = Path("/Users/maxchalekson/Projects/NICO-Research/NICO_human-g
FIGDIR = BASE / "figures"; FIGDIR.mkdir(parents=True, exist_ok=True)
REGDIR = BASE / "regression-model-output"; REGDIR.mkdir(parents=True,

# ----- CONFIG -----
REG_FEATURES = [
    "ann_knowledge_sharing_score",
    "ann_evaluation_practices_score",
    "ann_idea_management_score",
    "ann_coordination_and_decision_practices_score",
    "ann_information_seeking_score",
    "ann_relational_climate_score",
    "ann_participation_dynamics_score",
    "ann_integration_practices_score",
]

PERSON_COL = "person_name" if "person_name" in df.columns else "person

# Build a pseudo meeting id if you don't have meeting/session ids
df = df.copy()
df["conference"] = df["conference"].astype(str)
df["year"] = pd.to_numeric(df["year"], errors="coerce").fillna(-1).ast
df["conference_year"] = df["conference"] + "_" + df["year"]
MEET_COL = "conference_year"

# Roles / outcomes
df["any_team"] = (pd.to_numeric(df.get("num_teams",0), errors="coerc
df["any_funded"] = (pd.to_numeric(df.get("num_funded_teams",0), errors
df["role_facilitator"] = pd.to_numeric(df.get("role_facilitator",0), e

# Segment ordering
SEG_ORDER = ["first", "middle", "last"]
df = df[df["segment"].isin(SEG_ORDER)].copy()
df["segment"] = pd.Categorical(df["segment"], SEG_ORDER, ordered=True)

# ----- 1) DEFINE ACTIVITY STATES (top tercile within meeting×seg
def mark_active_by_tercile(dfin: pd.DataFrame) -> pd.DataFrame:
    d = dfin.copy()

    def _flag_top_tercile(x: pd.Series) -> pd.Series:
        # Handle degenerate groups (all equal, usually all zero)
        if x.notna().sum() == 0 or np.nanmax(x) == np.nanmin(x):
            return pd.Series(False, index=x.index)
        q = x.quantile([1/3, 2/3]).values
        hi = q[1]

```

```

        # If both terciles collapse (e.g., many zeros), fall back to m
        if not np.isfinite(hi) or hi == q[0]:
            hi = x.median()
        return x > hi

    for f in REG_FEATURES:
        d[f+"_active"] = (
            d.groupby([MEET_COL, "segment"], observed=True)[f]
            .transform(_flag_top_tercile)
            .astype(bool)
        )
    act_cols = [f"{f}_active" for f in REG_FEATURES]
    d["any_active"] = d[act_cols].any(axis=1)
    return d

dfA = mark_active_by_tercile(df)

# ----- 2) BUILD GRAPHS PER MEETING×SEGMENT -----
def build_graph_for_group(block: pd.DataFrame) -> nx.Graph:
    """Edge between two people if they are co-active on ANY feature in
    people = block[PERSON_COL].unique().tolist()
    G = nx.Graph()
    G.add_nodes_from(people)

    A = block[[PERSON_COL] + [f"{f}_active" for f in REG_FEATURES]].co
    A = A.groupby(PERSON_COL, as_index=True).max() # dedupe
    if A.shape[0] < 2:
        return G

    active_people = A.index[A.any(axis=1)].tolist()
    for i, p in enumerate(active_people):
        feats_i = A.loc[p].values
        for q in active_people[i+1:]:
            feats_q = A.loc[q].values
            shared = int(np.sum(feats_i & feats_q))
            if shared > 0:
                G.add_edge(p, q, weight=shared)
    return G

def compute_graph_metrics(G: nx.Graph) -> dict:
    n = G.number_of_nodes()
    m = G.number_of_edges()
    dens = nx.density(G) if n > 1 else 0.0
    trans = nx.transitivity(G) if n > 2 else 0.0
    clust = nx.average_clustering(G) if n > 2 else 0.0
    giant = 0
    if n > 0:
        comps = sorted(nx.connected_components(G), key=len, reverse=True)
        giant = len(comps[0]) if comps else 0
    return dict(n=n, m=m, density=dens, transitivity=trans, avg_cluste

# Meeting×segment metrics

```

```

records = []
for (mtg, seg), block in dfA.groupby([MEET_COL, "segment"], observed=True):
    G = build_graph_for_group(block)
    met = compute_graph_metrics(G)
    met.update({MEET_COL: mtg, "segment": seg})
    records.append(met)

net_meeting_seg = pd.DataFrame(records)
net_meeting_seg.to_csv(REGDIR/"network_meeting_segment_metrics.csv", index=False)

# ----- 3) PERSON×SEGMENT METRICS -----
node_rows = []
for (mtg, seg), block in dfA.groupby([MEET_COL, "segment"], observed=True):
    G = build_graph_for_group(block)
    if G.number_of_nodes() == 0:
        continue
    deg = dict(G.degree())
    cl = nx.clustering(G) if G.number_of_nodes() > 2 else {n: 0.0 for n in G.nodes()}
    bt = nx.betweenness_centrality(G, normalized=True) if G.number_of_nodes() > 2 else {}
    for p in G.nodes():
        node_rows.append({
            MEET_COL: mtg, "segment": seg, PERSON_COL: p,
            "deg": deg.get(p, 0), "clustering": cl.get(p, 0.0), "betweenness": bt.get(p, 0)
        })

net_person_seg = pd.DataFrame(node_rows)

# Join roles (max across appearances)
roles = (df[[PERSON_COL, "role_facilitator", "any_team", "any_funded"]]
         .groupby(PERSON_COL, as_index=False).max())
net_person_seg = net_person_seg.merge(roles, on=PERSON_COL, how="left")
net_person_seg.to_csv(REGDIR/"network_person_segment_metrics.csv", index=False)

# ----- 4) DELTAS (middle-first, last-middle) per person -----
def pivot_person_seg(dfin, value):
    p = (dfin.pivot_table(index=PERSON_COL, columns="segment", values=value)
         .reindex(columns=SEG_ORDER))
    return p

metric_list = ["deg", "clustering", "betweenness"]
delta_rows = []
for met in metric_list:
    W = pivot_person_seg(net_person_seg, met)
    if W.isna().all().all():
        continue
    D_mid = (W["middle"] - W["first"]).fillna(0)
    D_end = (W["last"] - W["middle"]).fillna(0)
    tmp = pd.DataFrame({
        PERSON_COL: W.index, f"{met}_first": W["first"],
        f"{met}_middle": W["middle"], f"{met}_last": W["last"],
        f"d_mid_{met}": D_mid, f"d_end_{met}": D_end
    })

```

```

    delta_rows.append(tmp.set_index(PERSON_COL))
net_person_delta = pd.concat(delta_rows, axis=1).reset_index()
net_person_delta = net_person_delta.merge(roles, on=PERSON_COL, how="l")
net_person_delta.to_csv(REGDIR/"network_person_deltas.csv", index=False)

# ----- 5) DESCRIPTIVE PLOTS -----
def line_metric_across_segments(metric="avg_clustering"):
    gp = net_meeting_seg.pivot_table(index=MEET_COL, columns="segment")
    means = gp.mean().reindex(SEG_ORDER)
    cis = gp.apply(lambda s: 1.96*s.std()/np.sqrt(s.notna().sum()),

    x = np.arange(3)
    fig, ax = plt.subplots(figsize=(7,4))
    ax.plot(x, means.values, marker="o")
    ax.fill_between(x, (means-cis).values, (means+cis).values, alpha=0.5)
    ax.set_xticks(x); ax.set_xticklabels(SEG_ORDER)
    ax.set_title(f"{metric} (meeting-level)"); ax.set_ylabel(metric)
    fig.tight_layout(); fig.savefig(FIGDIR/f"network_{metric}_by_segme

for m in ["density", "transitivity", "avg_clustering", "giant"]:
    line_metric_across_segments(m)

# Simple role contrasts for person-level clustering (mean bars by segm
def bars_by_role(metric="clustering"):
    take = net_person_seg[[PERSON_COL, "segment", metric, "role_facilitat
    segs = SEG_ORDER
    x = np.arange(len(segs)); width = 0.35
    for role in ["role_facilitator", "any_team", "any_funded"]:
        fig, ax = plt.subplots(figsize=(7,4))
        tmp = (take.groupby(["segment", role])[metric].mean().unstack(
        ax.bar(x-width/2, tmp[0].values, width, label=f"{role}=0")
        ax.bar(x+width/2, tmp[1].values, width, label=f"{role}=1")
        ax.set_xticks(x); ax.set_xticklabels(segs)
        ax.set_title(f"{metric} by segment x {role}")
        ax.legend()
        fig.tight_layout(); fig.savefig(FIGDIR/f"network_{metric}_by_

bars_by_role("clustering")

# ----- 6) LINK NETWORK CHANGE → OUTCOMES (logit; illustrative) -----
for outcome in ["any_funded", "any_team", "role_facilitator"]:
    if outcome not in net_person_delta.columns:
        continue
    cols = [c for c in ["d_mid_clustering", "d_end_clustering", "cluster
    if not cols:
        continue
    fml = f"{outcome} ~ " + " + ".join(cols)
    m = smf.logit(fml, data=net_person_delta.dropna()).fit(dis=0, max
    tab = pd.DataFrame({
        "term": m.params.index, "coef": m.params.values, "se": m.bse.v
    })
    tab["or"] = np.exp(tab["coef"])

```

```

tab["lo95"] = np.exp(tab["coef"] - 1.96*tab["se"])
tab["hi95"] = np.exp(tab["coef"] + 1.96*tab["se"])
tab.to_csv(REGDIR/f"logit_network_change_{outcome}.csv", index=False)
print(f"[logit] {outcome}\n", tab[["term", "or", "lo95", "hi95", "p"]])

```

```

[logit] any_funded
      term      or      lo95      hi95      p
0      Intercept  0.088040  0.054876  0.141245  7.097645e-24
1  d_mid_clustering  3.509868  1.385959  8.888552  8.085365e-03
2  d_end_clustering  1.676870  0.692679  4.059446  2.518034e-01
3  clustering_first  5.974105  2.561700  13.932127  3.513122e-05
[logit] any_team
      term      or      lo95      hi95      p
0      Intercept  0.215475  0.151794  0.305872  8.878584e-18
1  d_mid_clustering  4.300932  2.073206  8.922420  8.918173e-05
2  d_end_clustering  1.691593  0.841184  3.401738  1.402676e-01
3  clustering_first  12.622433  6.320372  25.208295  6.743103e-13
[logit] role_facilitator
      term      or      lo95      hi95      p
0      Intercept  0.043140  0.023111  0.080529  5.582400e-23
1  d_mid_clustering  5.680191  1.788547  18.039542  3.217793e-03
2  d_end_clustering  4.009591  1.277785  12.581790  1.730628e-02
3  clustering_first  7.264093  2.509919  21.023406  2.549023e-04

```

visual network graph

```

In [6]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Assume you already have net_person_seg with columns:
# ['person_name', 'segment', 'deg', 'clustering', 'betweenness',
#  'role_facilitator', 'any_team', 'any_funded']

def plot_network_metric_by_role(net_person_seg, metric="clustering"):
    seg_order = ["first", "middle", "last"]
    fig, axes = plt.subplots(1, 3, figsize=(12, 4), sharey=True)
    roles = {
        "role_facilitator": "Facilitators",
        "any_team": "On a Team",
        "any_funded": "Funded Teams"
    }

    for ax, (col, title) in zip(axes, roles.items()):
        tmp = (net_person_seg
               .groupby(["segment", col])[metric]
               .mean()
               .reset_index()
               .pivot(index="segment", columns=col, values=metric)
               .reindex(seg_order))
        tmp.plot(kind="line", marker="o", ax=ax)

```

```

ax.set_title(title)
ax.set_xlabel("Segment")
ax.set_ylabel(f"Mean {metric}")
ax.legend(["No", "Yes"], title=title.split()[0])
ax.grid(alpha=0.3)

```

```

plt.suptitle(f"Network {metric.title()} Over Time by Role", fontsize=12)
plt.tight_layout()
plt.savefig(FIGDIR / f"network_{metric}_by_role_temporal.png", dpi=300)
plt.show()

```

```

plot_network_metric_by_role(net_person_seg, metric="clustering")

```

